

Chapter 3: Dimensionality Reduction

Prof. Dr.-Ing. Thomas Schultz

URL: <http://cg.cs.uni-bonn.de/schultz/>

E-Mail: schultz@cs.uni-bonn.de

Office: Friedrich-Hirzebruch-Allee 6, Room 2.117

April 22/28/29, 2026

Dimensionality Reduction

Goal: Embed high-dimensional data into a low-dimensional space, in a way that important structure is preserved

We will cover eight methods:

- **PCA:** Brief intro / recap
- **Kernel PCA:** Deal with nonlinear structures
- **MDS:** Based on distances alone
- **ISOMAP / Laplacian Eigenmaps:** “Unroll” nonlinear manifolds
- **LDA:** Supervised = account for class labels
- **t-SNE / UMAP:** Focus on preserving local neighbors

Motivating Example

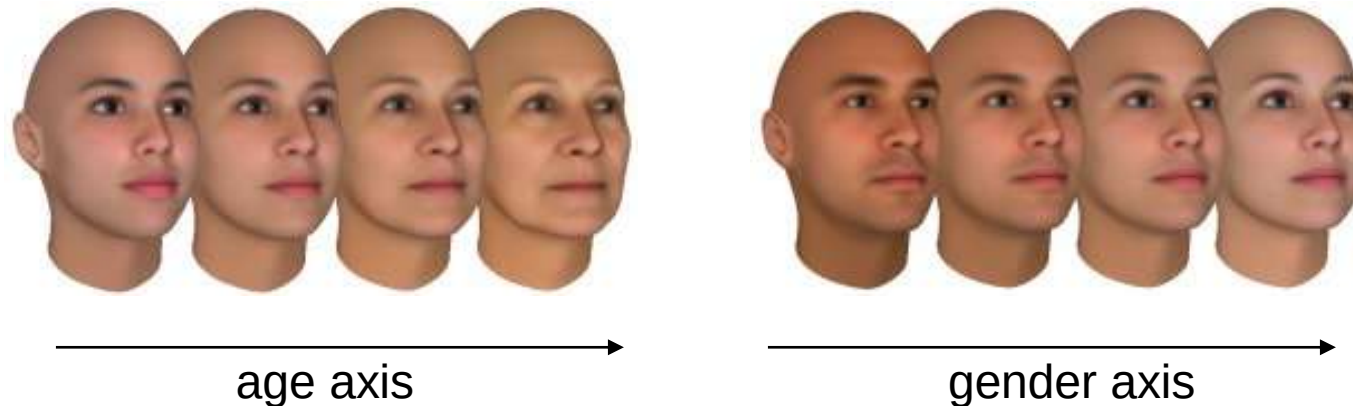
- Example: Data base of face scans (3D geometry + texture)



- 10,000 points in each scan
- x, y, z, R, G, B – 6 numbers for each point
- Thus, each scan is a $10,000 \times 6 = \mathbf{60,000}$ -dimensional vector!

Motivating Example

- How to find interesting axes in this 60,000-dimensional space?
 - axes that measure age, gender, etc...
 - There is hope: the faces are likely to be governed by a small set of parameters (much fewer than 60,000...)

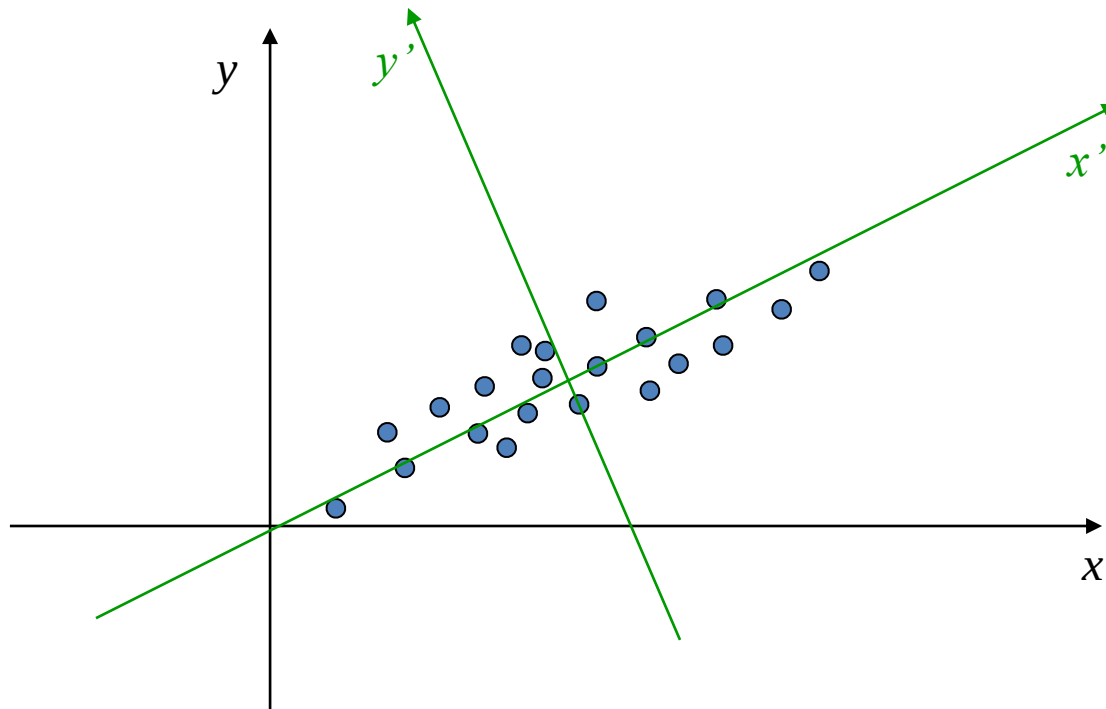


Section 3.1:

(Kernel) Principal Component Analysis

PCA – the general idea

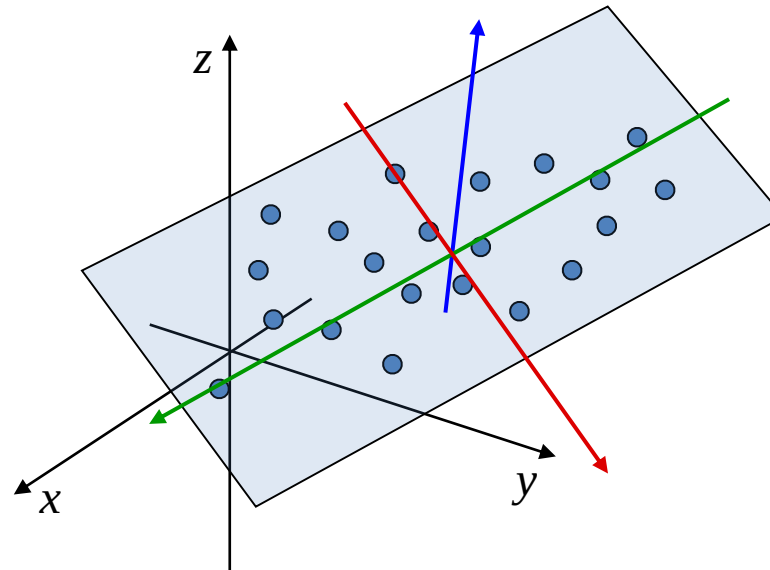
- PCA finds an orthogonal basis that best represents a given data set.



The sum of squared distances from the x' axis is minimized.

PCA – the general idea

- PCA finds an orthogonal basis that best represents a given data set.



3D point set in
standard basis

PCA finds a best approximating plane
(again, in terms of $\sum \text{distances}^2$)

Goals of Principal Component Analysis

- Given input $\mathbf{x}_i \in \mathbb{R}^p$, $i = 1, 2, \dots, n$, compute a mapping from $\mathbf{x}$ to $\mathbf{y} = U_k^T(\mathbf{x} - \bar{\mathbf{x}})$ that can be used for
 - **Approximation:** $U_k \in \mathbb{R}^{p \times k}$ with $k < p$
 - Select U_k such that the overall approximation error $\sum_i \|(I - U_k U_k^T)(\mathbf{x}_i - \bar{\mathbf{x}})\|^2$ is minimal
 - For the same U_k , the variance in the projected data remains maximal
 - **Decorrelation:**
 - Coefficients of $\mathbf{y}$ are uncorrelated
 - If $k = p$, pure rotation, no approximation

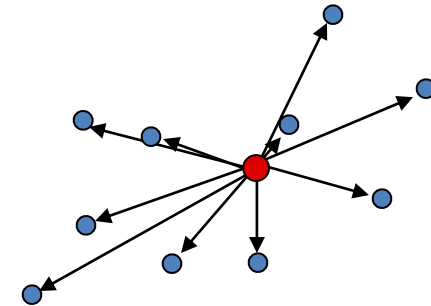
The origin of the new axes

- The origin is a zero-order approximation of our data set

$$\mathbf{m} = \operatorname{argmin}_{\mathbf{x}} \sum_{i=1}^n \|\mathbf{x}_i - \mathbf{x}\|^2$$

- It will be the center of mass:

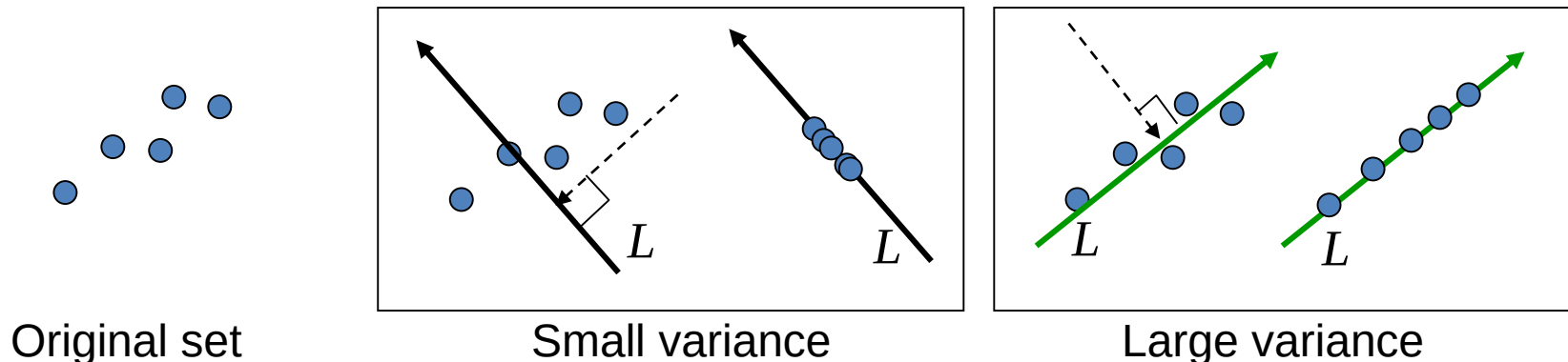
$$\mathbf{m} = \frac{1}{n} \sum_{i=1}^n \mathbf{x}_i$$



Variance of projected points

- Let's look at a **line** L through the center of mass $\mathbf{m}$, and project our points $\mathbf{x}_i$ onto it. The *sample variance* of the *projected* points $\mathbf{x}'_i$ is:

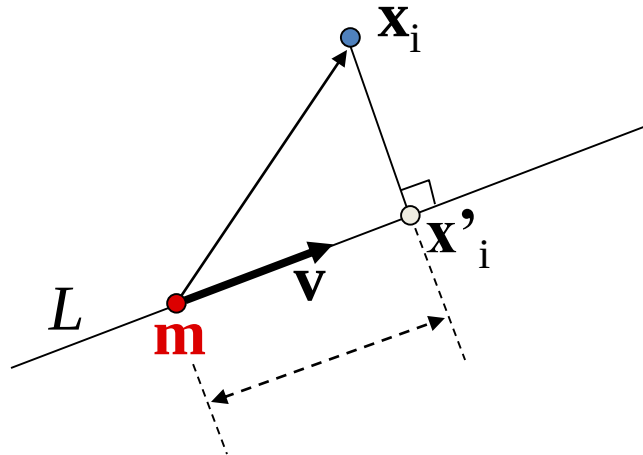
$$\text{var}(L) = \frac{1}{n-1} \sum_{i=1}^n \|\mathbf{x}'_i - \mathbf{m}\|^2$$



Variance of projected points

- Since **line** L goes through the center of mass $\mathbf{m}$, it can be written as $\mathbf{m} + t \mathbf{v}$ with direction $\mathbf{v}$, $\|\mathbf{v}\| = 1$
- After projecting $\mathbf{x}_i$ onto L , the distance from $\mathbf{m}$ is:

$$\|\mathbf{x}'_i - \mathbf{m}\| = \langle \mathbf{v}, \mathbf{x}_i - \mathbf{m} \rangle = \langle \mathbf{v}, \mathbf{z}_i \rangle = \mathbf{v}^T \mathbf{z}_i$$



Scatter matrix

- Given $\mathbf{z}_i = \mathbf{x}_i - \mathbf{m}$, $i = 1, 2, \dots, n$,
 - form $p \times n$ matrix Z with $\mathbf{z}_i$ as columns
 - define $p \times p$ **scatter matrix**

$$S = ZZ^T$$

$$S = \underbrace{\begin{pmatrix} \begin{matrix} z_1^1 \\ z_1^2 \\ \vdots \\ z_1^p \end{matrix} & \begin{matrix} z_2^1 \\ z_2^2 \\ \vdots \\ z_2^p \end{matrix} & \cdots & \begin{matrix} z_n^1 \\ z_n^2 \\ \vdots \\ z_n^p \end{matrix} \end{pmatrix}}_Z \underbrace{\begin{pmatrix} \begin{matrix} z_1^1 & z_1^2 & \cdots & z_1^p \end{matrix} \\ \begin{matrix} z_2^1 & z_2^2 & \cdots & z_2^p \end{matrix} \\ \vdots & \vdots & & \vdots \\ \begin{matrix} z_n^1 & z_n^2 & \cdots & z_n^p \end{matrix} \end{pmatrix}}_{Z^T}$$

Variance of projected points

Scatter matrix S determines the sample variance in direction $\mathbf{v}$:

$$\begin{aligned}
 \text{var}(L) &= \frac{1}{n-1} \sum_{i=1}^n \|\mathbf{x}'_i - \mathbf{m}\|^2 = \frac{1}{n-1} \sum_{i=1}^n (\mathbf{v}^T \mathbf{z}_i)^2 = \frac{1}{n-1} \|\mathbf{v}^T \mathbf{Z}\|^2 = \frac{1}{n-1} \|\mathbf{Z}^T \mathbf{v}\|^2 \\
 &= \frac{1}{n-1} \langle \mathbf{Z}^T \mathbf{v}, \mathbf{Z}^T \mathbf{v} \rangle = \frac{1}{n-1} \mathbf{v}^T \mathbf{Z} \mathbf{Z}^T \mathbf{v} = \frac{1}{n-1} \mathbf{v}^T \mathbf{S} \mathbf{v} \\
 &= \frac{1}{n-1} \sum_{i=1}^n (\mathbf{v}^T \mathbf{z}_i)^2 = \frac{1}{n-1} \sum_{i=1}^n \left((v^1 \ v^2 \ \dots \ v^p) \begin{pmatrix} z_i^1 \\ z_i^2 \\ \vdots \\ z_i^p \end{pmatrix} \right)^2 \\
 &= \left\| (v^1 \ v^2 \ \dots \ v^p) \begin{pmatrix} z_1^1 & z_1^2 & \dots & z_1^p \\ z_2^1 & z_2^2 & \dots & z_2^p \\ \vdots & \vdots & \ddots & \vdots \\ z_n^1 & z_n^2 & \dots & z_n^p \end{pmatrix} \right\|^2 = \|\mathbf{v}^T \mathbf{Z}\|^2
 \end{aligned}$$

Directions of maximal variance

- So, we have:

$$\text{var}(L) = \frac{1}{n-1} \mathbf{v}^T S \mathbf{v}$$

- Min-Max Theorem (Linear Algebra):

Let $f: \{\mathbf{v} \in \mathbb{R}^p \mid \|\mathbf{v}\| = 1\} \rightarrow \mathbb{R}$,

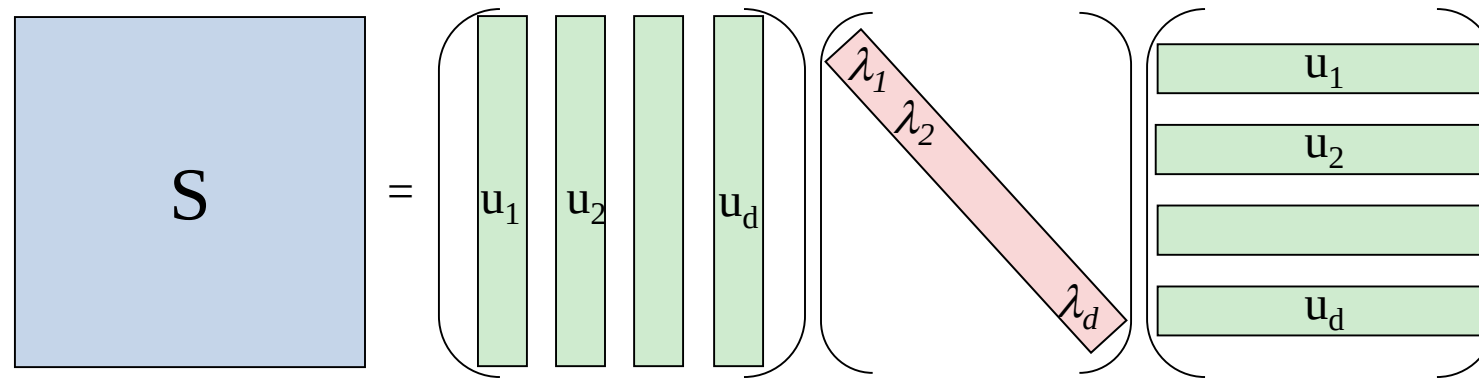
$f(\mathbf{v}) = \mathbf{v}^T S \mathbf{v}$ (and S is symmetric).

Then, the extrema of f are attained at the eigenvectors of S .

Eigenvectors of S are directions of maximal/minimal variance!

Scatter matrix - eigendecomposition

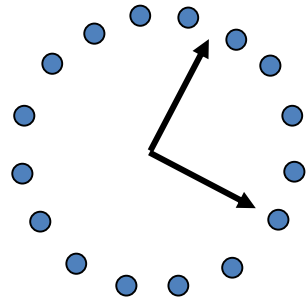
- Scatter matrix S is symmetric
 - **Reminder:** S has eigendecomposition: $S = U\Lambda U^T$



The eigenvectors form
an orthogonal basis

Principal Components

- Eigenvectors that correspond to **big** eigenvalues are the directions in which the data has strong components (= large variance).
- If the eigenvalues are more or less the same, there is no main direction.

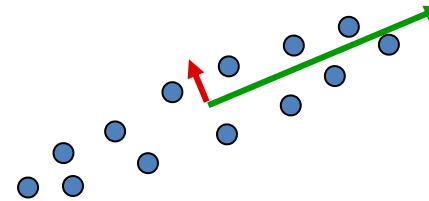


There is no preferred direction.

S looks like this:

$$\begin{pmatrix} \lambda & \\ & \lambda \end{pmatrix}$$

Any vector is an eigenvector



There is a clear preferred direction.

S looks like this:

$$U \begin{pmatrix} \lambda & \\ & \mu \end{pmatrix} U^T$$

μ is close to zero, much smaller than λ .

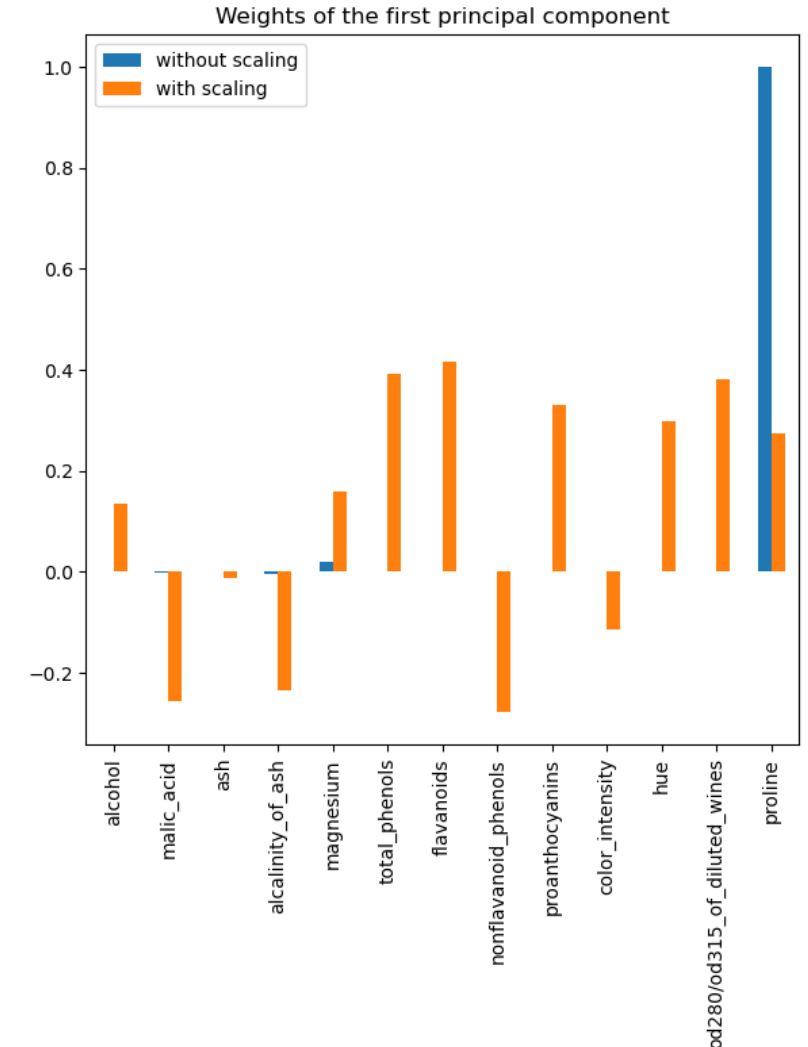
Feature Standardization

- *Caveat:* For attributes on different scales, principle components often make little sense
 - Attributes with large numerical ranges dominate
 - Results depend on the units used (e.g., mm vs m)
- In such cases, **standardization** should precede PCA:

$$Z_i = \frac{x_i - \mu}{\sigma}$$

with sample mean μ , standard deviation σ

- When attributes are on a common scale, it often makes sense *not* to standardize them
 - avoids amplifying noise / unrealistic variations



Algorithm: PCA

- **Input:** $\mathbf{x}_i \in \mathbb{R}^p, i = 1, 2, \dots, n$
- **Output:** $U_k \in \mathbb{R}^{p \times k}$ with $k \leq p$ such that $\mathbf{y} = U_k^T(\mathbf{x} - \bar{\mathbf{x}})$ with $\bar{\mathbf{x}} = \frac{1}{n} \sum_i \mathbf{x}_i$
- **Algorithm:**
 - **Center** the points, $\mathbf{z}_i = \mathbf{x}_i - \bar{\mathbf{x}}$, standardize if needed
 - Form $p \times n$ **centered data matrix** $Z = [\mathbf{z}_1, \mathbf{z}_2, \dots, \mathbf{z}_n]$
 - Form $p \times p$ **scatter matrix** $S = ZZ^T$
 - Compute spectral decomposition $S = U\Lambda U^T$
 - Sort coefficients of Λ in decreasing order
 - Form U_k from k leading columns of U

PCA: Intrinsic Dimensionality

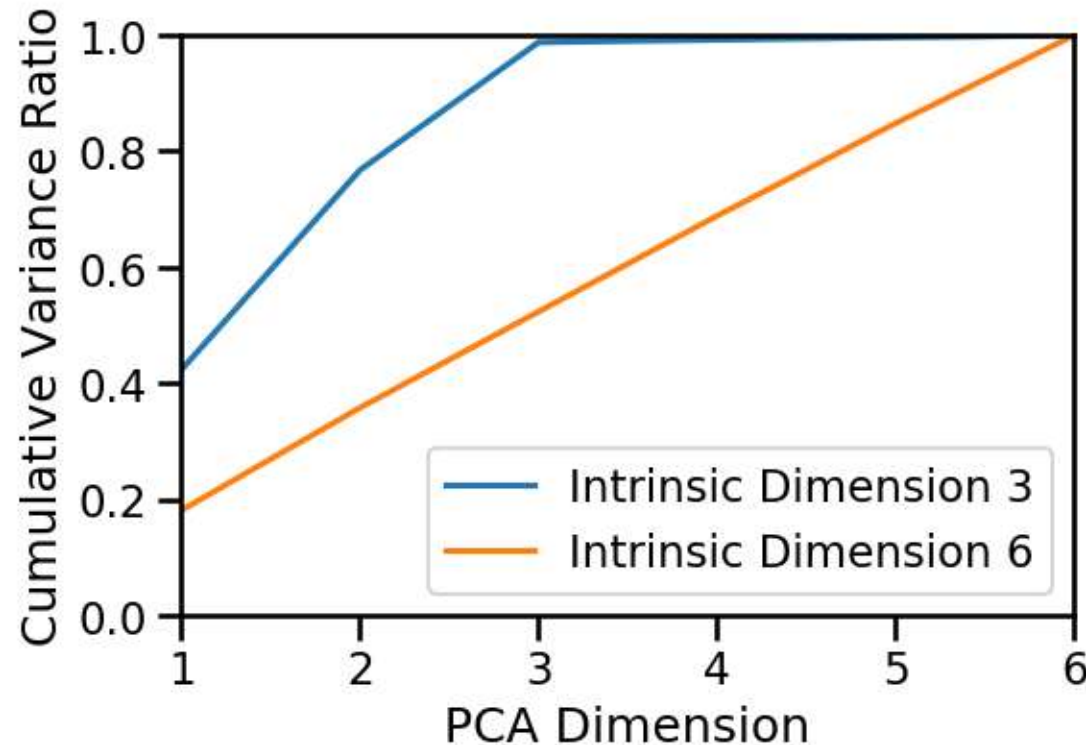
- **Reminder:** The scatter matrix is proportional to a variance-covariance matrix estimate
- We can compute the **cumulative ratio of variance** explained by the first k components from its sorted eigenvalues λ_i as

$$r_{\sigma^2}^k = \frac{\sum_{i=1}^k \lambda_i}{\sum_{i=1}^p \lambda_i}$$

- It is common to plot $r_{\sigma^2}^k$ over k and look for an “elbow” in the curve, or to pick smallest k for which $r_{\sigma^2}^k$ exceeds a given threshold

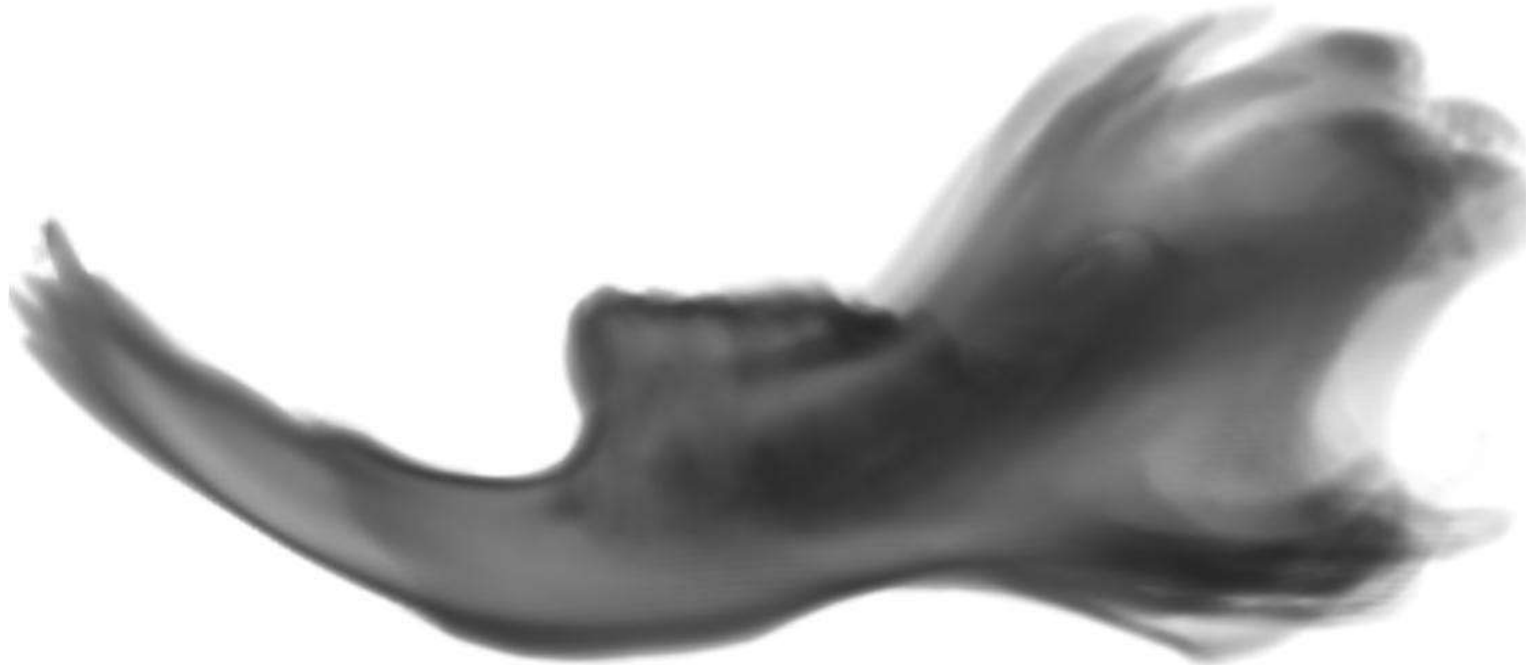
Example: Intrinsic Dimensionality

- Results on two simulated 6D datasets:
 - Three dimensions contain low-magnitude noise
 - All six dimensions contain the same variance



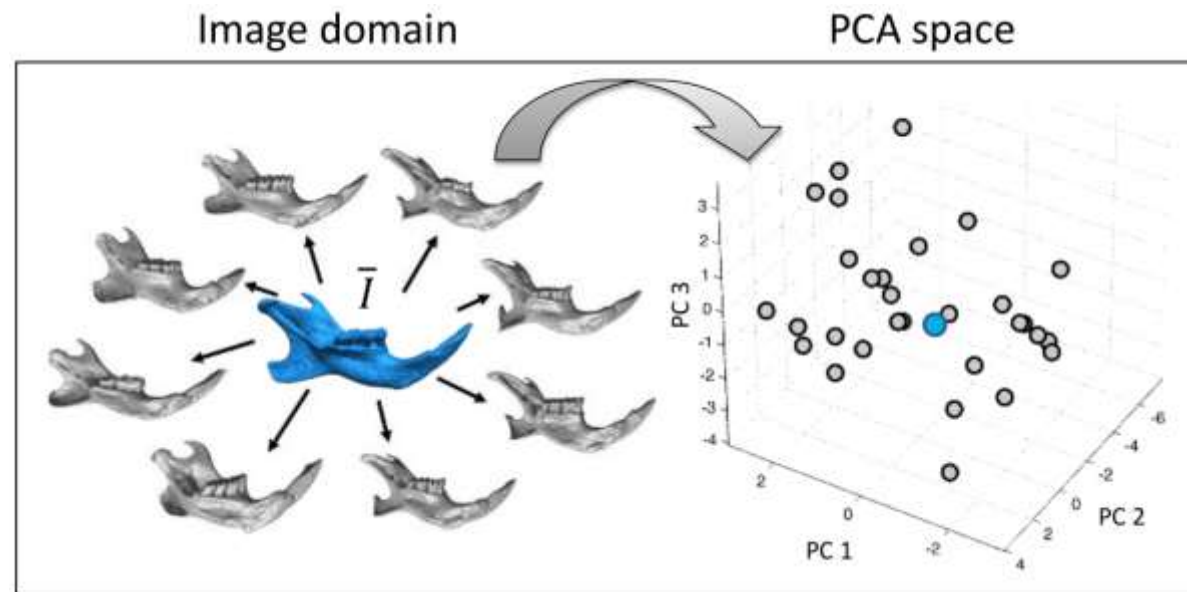
Example: Biological Shape Analysis

- **Data:** CT images of 48 mouse mandibles
- **Scientific question:** Impact of factors such as evolutionary history, diet, or geography on skeleton shape



Linear Shape Spaces

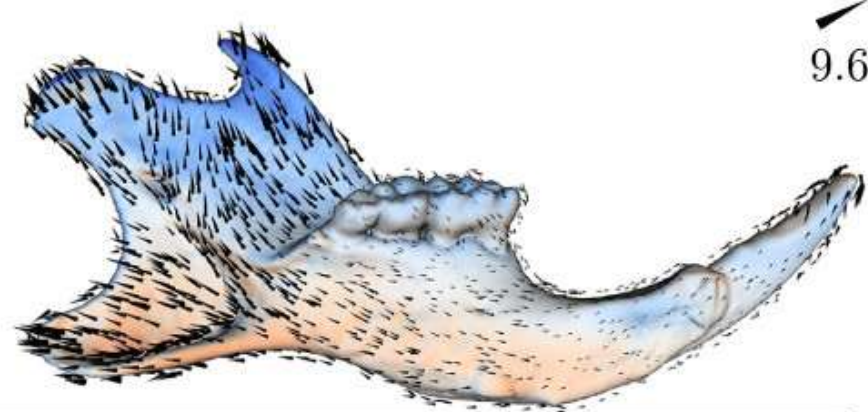
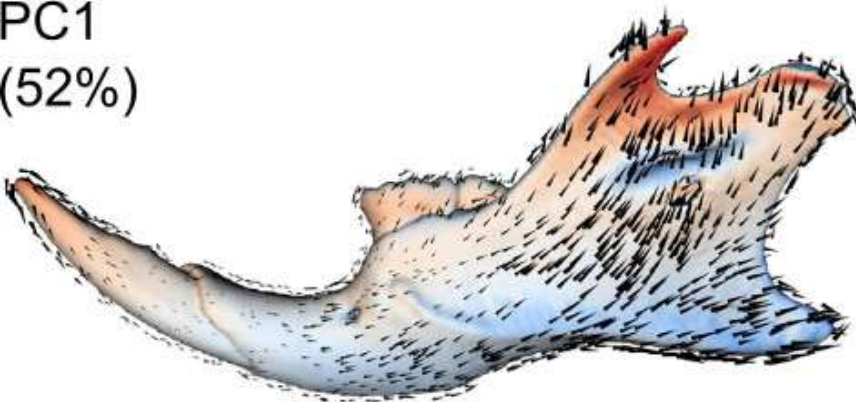
- **Align** all shapes, compute a **mean shape**
- Express each shape as transformation of the mean
 - **Shape vector**: Concatenation of displacement vectors
- Perform PCA on shape vectors to identify **principal modes** of shape variation



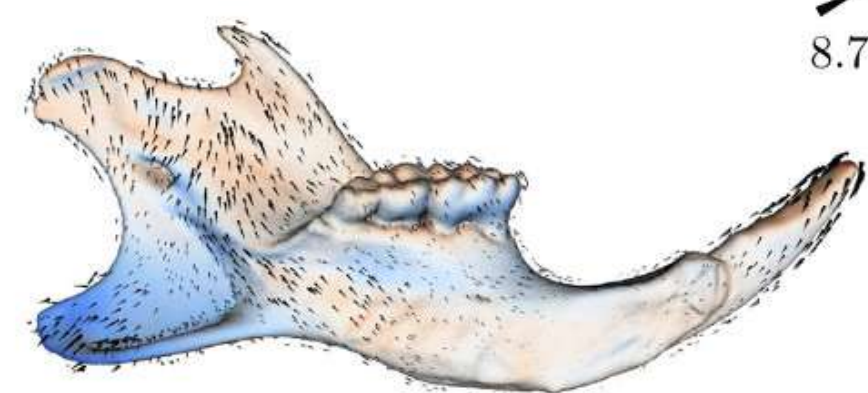
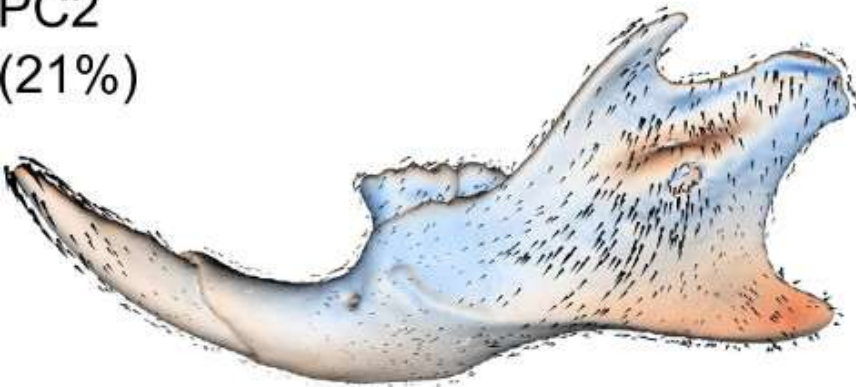
Linear Shape Space: Example

- Normal component: Color
- Tangential component: Arrow glyph

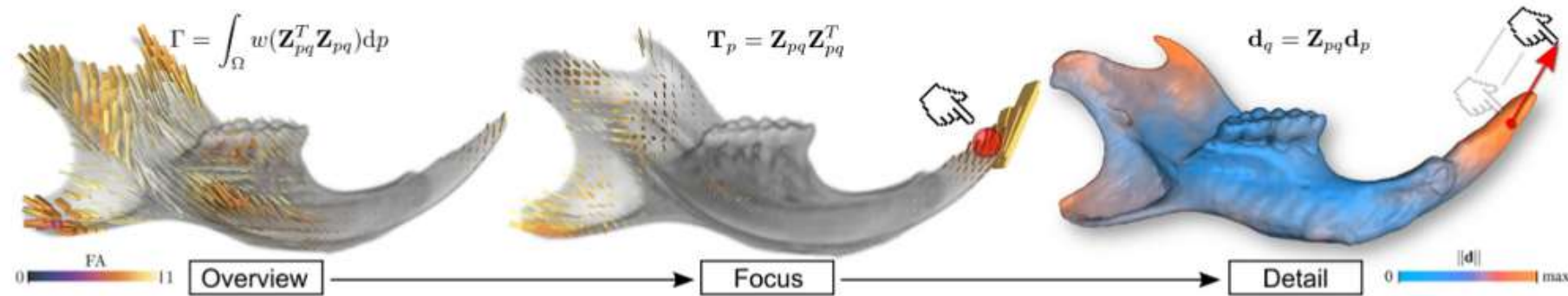
PC1
(52%)



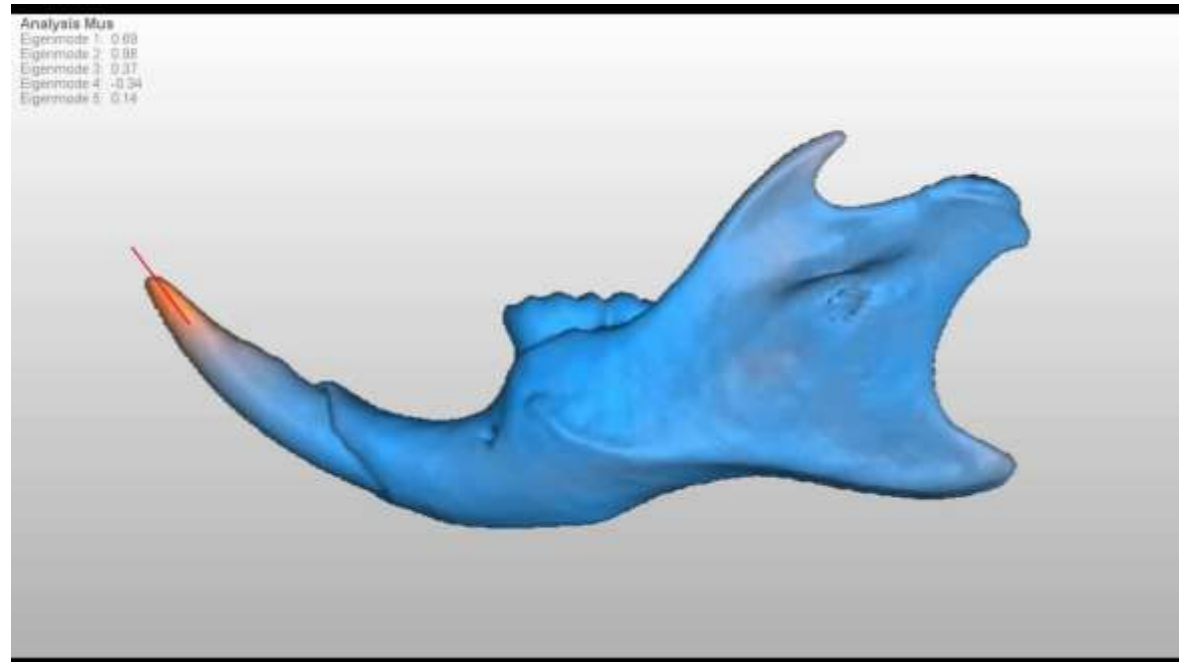
PC2
(21%)



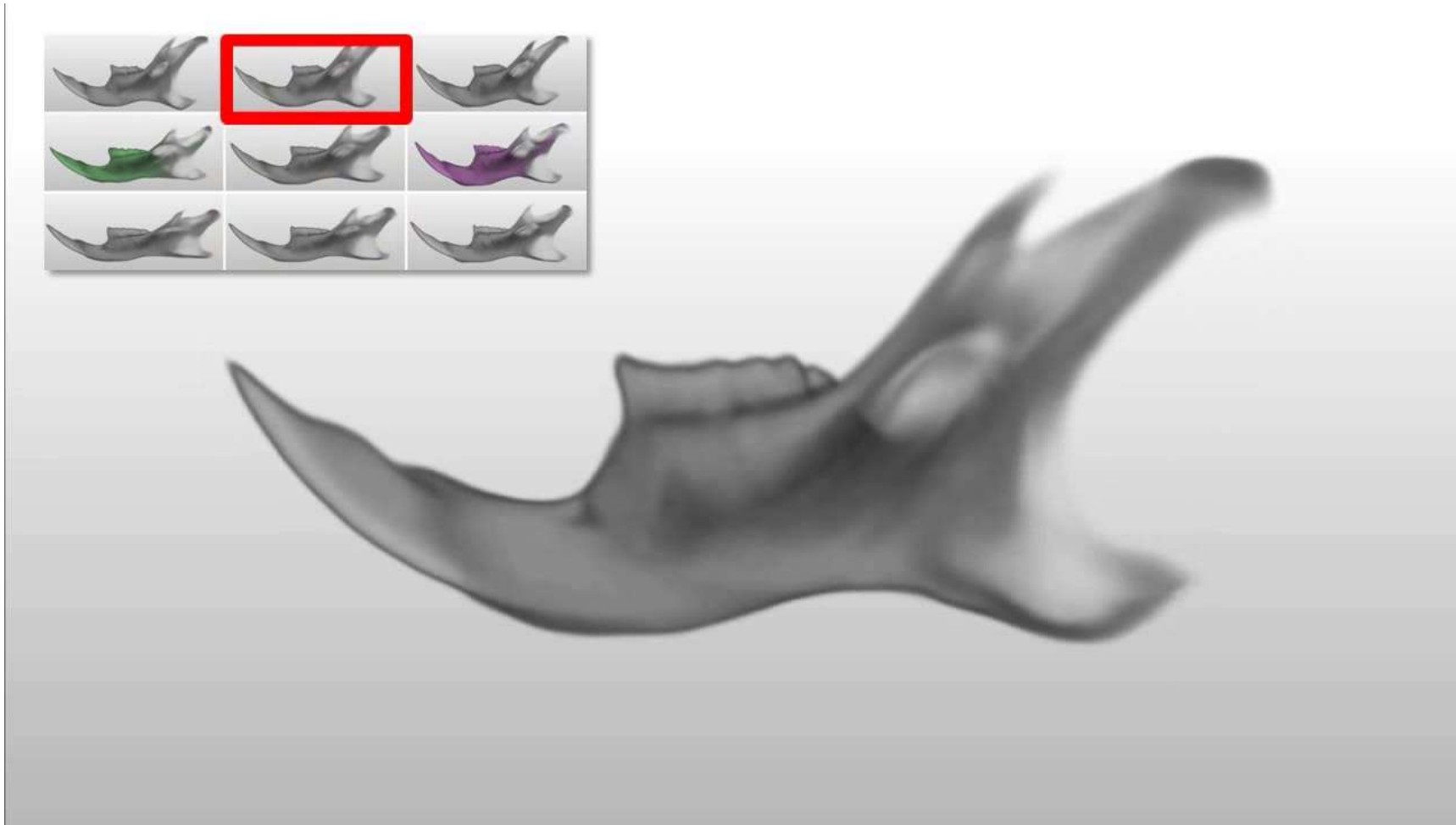
Exploring Shape Spaces



Hermann et al.:
„A Visual Analytics
Approach to Study
Anatomic
Covariation.”
PacificVis 2014



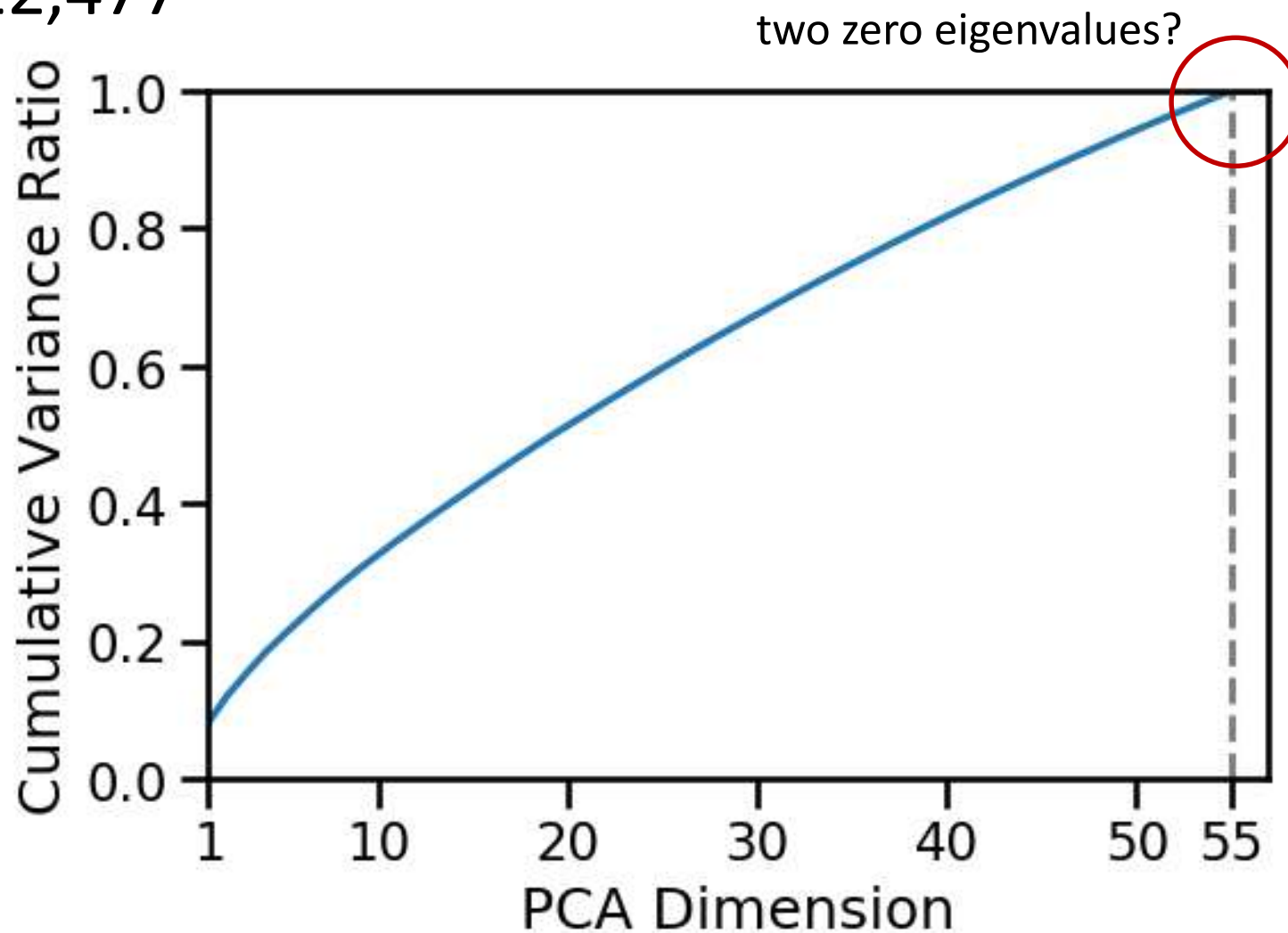
Group Analysis



Hermann et al.: „Accurate Interactive Visualization of Large Deformations and Variability in Biomedical Image Ensembles.“
IEEE SciVis 2015

Quiz: Interpreting PCA Result

$n=57, p=122,477$



What if $n < p$?

- **Maximum rank** of Z and S is $\min(n - 1, p)$
 - Linear dependence introduced by centering
 - It does not make sense to use $k > n - 1$
- All eigenvectors of S are in the span of $\mathbf{z}_i$:

$$\begin{aligned}\lambda_a \mathbf{u}_a &= S \mathbf{u}_a = \sum_i \mathbf{z}_i \mathbf{z}_i^T \mathbf{u}_a = \sum_i (\mathbf{z}_i^T \mathbf{u}_a) \mathbf{z}_i \\ \Rightarrow \mathbf{u}_a &= \sum_i \frac{\mathbf{z}_i^T \mathbf{u}_a}{\lambda_a} \mathbf{z}_i =: \sum_i \alpha_i^a \mathbf{z}_i\end{aligned}$$

Efficient computation for $n < p$

- We can replace the $p \times p$ with an $n \times n$ eigenvalue problem

- Based on $S\mathbf{u} = \lambda\mathbf{u} \Leftrightarrow \mathbf{z}_i^T S\mathbf{u} = \lambda \mathbf{z}_i^T \mathbf{u}$ for all i

$$\mathbf{z}_i^T S\mathbf{u}_a = \lambda_a \mathbf{z}_i^T \mathbf{u}_a$$

$$\mathbf{z}_i^T \sum_l \mathbf{z}_l \mathbf{z}_l^T \sum_j \alpha_j^a \mathbf{z}_j = \lambda_a \mathbf{z}_i^T \sum_j \alpha_j^a \mathbf{z}_j$$

$$\sum_{j,l} \alpha_j^a [\mathbf{z}_i^T \mathbf{z}_l] [\mathbf{z}_l^T \mathbf{z}_j] = \lambda_a \sum_j \alpha_j^a [\mathbf{z}_i^T \mathbf{z}_j]$$

- With $n \times n$ centered inner product („Gram“) matrix $K_{ij}^c := [\mathbf{z}_i^T \mathbf{z}_j]$:

$$(K^c)^2 \boldsymbol{\alpha}^a = \lambda_a K^c \boldsymbol{\alpha}^a \Leftrightarrow K^c \boldsymbol{\alpha}^a = \lambda_a \boldsymbol{\alpha}^a$$

Proper Normalization and Projecting New Data

- **Normalization:** $\mathbf{u}_a^T \mathbf{u}_a = 1$ translates into

$$\sum_{i,j} \alpha_i^a \alpha_j^a [\mathbf{z}_i^T \mathbf{z}_j] = (\boldsymbol{\alpha}^a)^T \mathbf{K}^c \boldsymbol{\alpha}^a \\ = \lambda_a (\boldsymbol{\alpha}^a)^T \boldsymbol{\alpha}^a = 1$$

– Therefore:

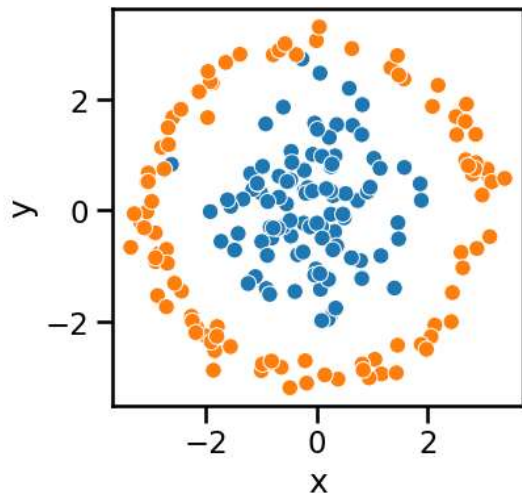
$$\|\boldsymbol{\alpha}^a\| = \frac{1}{\sqrt{\lambda_a}}$$

- **Projection** of a new point $\mathbf{x}$:

$$\mathbf{u}_a^T (\mathbf{x} - \bar{\mathbf{x}}) = \sum_i \alpha_i^a \mathbf{z}_i^T (\mathbf{x} - \bar{\mathbf{x}})$$

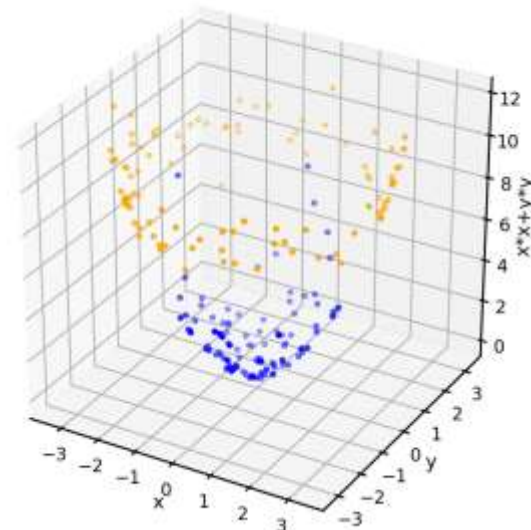
Preserving Nonlinear Structures

- PCA works well for **linear structures**
 - Straight lines, planes, etc.
- Can we preserve **nonlinear / curved structure**?
- **Idea:** Nonlinearly map data to a higher-dimensional feature space, apply PCA there



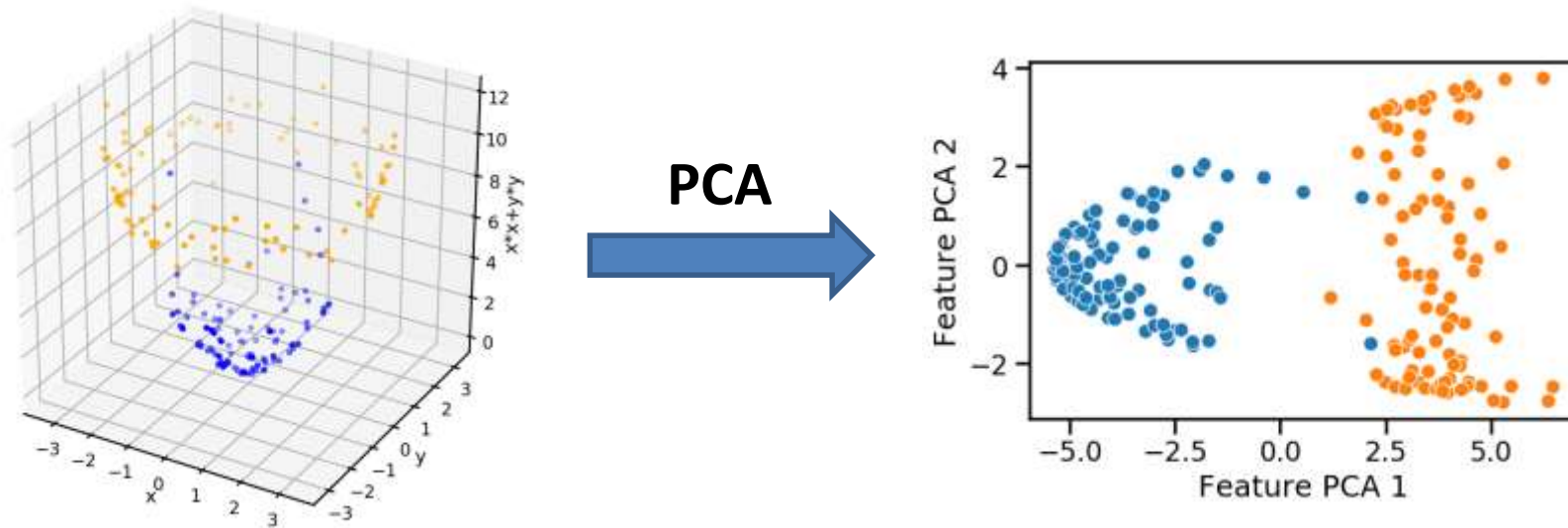
$$\begin{aligned} (x_1, x_2) &\mapsto \\ (x_1, x_2, x_1^2 + x_2^2) \end{aligned}$$

$\mathbf{x} \rightarrow \phi(\mathbf{x})$



The Kernel Trick

- PCA in feature space can lead to a more meaningful embedding of nonlinear structures



- **Remaining concern:** Explicitly computing $\mathbf{x} \rightarrow \phi(\mathbf{x})$ can be cumbersome and expensive
- **Kernel trick:** In the dual form, the PCA does not require $\phi(\mathbf{x})$. Instead, we simply replace all inner products with nonlinear kernel $K(\mathbf{x}_i, \mathbf{x}_j) = \phi(\mathbf{x}_i)^\top \phi(\mathbf{x}_j)$

Kernel Functions

- **Example:** $K(\mathbf{x}_i, \mathbf{x}_j) = (1 + \mathbf{x}_i^\top \mathbf{x}_j)^2$
 - Easy to verify that, for $\mathbf{x} = [x_1 \ x_2]^\top$ it corresponds to $\phi(\mathbf{x}_i)^\top \phi(\mathbf{x}_j)$ with $\phi(\mathbf{x}) = [1, \sqrt{2}x_1, \sqrt{2}x_2, x_1^2, \sqrt{2}x_1x_2, x_2^2]^\top$
- **Intuition:** $K(\mathbf{x}_i, \mathbf{x}_j)$ is a, potentially nonlinear, measure of similarity between $\mathbf{x}_i$ and $\mathbf{x}_j$
 - A valid kernel K produces positive definite Gram matrices
- **Examples** of valid kernels (here without proofs):
 - Linear: $K(\mathbf{x}_i, \mathbf{x}_j) = \mathbf{x}_i^\top \mathbf{x}_j$
 - Polynomial of power a : $K(\mathbf{x}_i, \mathbf{x}_j) = (1 + \mathbf{x}_i^\top \mathbf{x}_j)^a$
 - Squared Exponential (RBF, “Gaussian”):

$$K(\mathbf{x}_i, \mathbf{x}_j) = e^{-\gamma \|\mathbf{x}_i - \mathbf{x}_j\|^2}$$

Centering in Feature Space

- We require the centered kernel matrix $K_{ij}^c = (\Phi_i - \bar{\Phi})^T (\Phi_j - \bar{\Phi})$, but evaluating the kernel function gives us

$$K_{ij} = K(\mathbf{x}_i, \mathbf{x}_j) = \Phi_i^T \Phi_j$$

- Applying K to centered data $\mathbf{z}_i$ is *not* equivalent to centering feature vectors Φ_i
- Centering can be performed as follows:
$$K_{ij}^c = (\Phi_i - \bar{\Phi})^T (\Phi_j - \bar{\Phi}) = \Phi_i^T \Phi_j - \bar{\Phi}^T \Phi_j - \Phi_i^T \bar{\Phi} + \bar{\Phi}^T \bar{\Phi}$$
 - $\bar{\Phi}^T \Phi_j = \frac{1}{n} \sum_i \Phi_i^T \Phi_j = \frac{1}{n} \sum_i K_{ij}$
 - $\Phi_i^T \bar{\Phi} = \frac{1}{n} \sum_j \Phi_i^T \Phi_j = \frac{1}{n} \sum_j K_{ij}$
 - $\bar{\Phi}^T \bar{\Phi} = \frac{1}{n^2} \sum_{i,j} \Phi_i^T \Phi_j = \frac{1}{n^2} \sum_{i,j} K_{ij}$

Notation: Centering in Feature Space

- Let $\mathbf{1} \in \mathbb{R}^n$ denote the column vector in which each coefficient equals 1.
 - Then, $\frac{1}{n} \mathbf{1}\mathbf{1}^T$ is a matrix with $\frac{1}{n}$ everywhere
 - Multiplying $\frac{1}{n} \mathbf{1}\mathbf{1}^T$ from the left/right computes column/row averages
- Easy to verify that, given $H = I - \frac{1}{n} \mathbf{1}\mathbf{1}^T$
$$K^c = HKH$$

Algorithm: Kernel PCA

- **Input:** $\mathbf{x}_i \in \mathbb{R}^p, i = 1, 2, \dots, n$ and kernel function $K: \mathbb{R}^p \times \mathbb{R}^p \rightarrow \mathbb{R}$
- **Output:** $V_k \in \mathbb{R}^{n \times k}$ with $k \leq n$ and $D_k \in \mathbb{R}^{k \times k}$ such that $\mathbf{y} = D_k^{-1} V_k^T H \left(\mathbf{k} - \frac{1}{n} K \mathbf{1} \right)$ with $k_i = K(\mathbf{x}_i, \mathbf{x})$
- **Algorithm:**
 - Standardize $\mathbf{x}_i$ if needed
 - Compute **kernel matrix** $K_{ij} = K(\mathbf{x}_i, \mathbf{x}_j)$
 - **Center kernel matrix** $K^c = HKH$
 - Compute **spectral decomposition** $K^c = V\Lambda V^T$
 - Sort coefficients of Λ in decreasing order
 - Form V_k from k leading columns of V
 - Form D_k from top-left $k \times k$ block of $\sqrt{\Lambda}$

Projection Step in Kernel PCA

- Projection on the a^{th} principal component:

$$y_a = \sum_i \alpha_i^a \mathbf{z}_i^T (\mathbf{x} - \bar{\mathbf{x}})$$

- In feature space, this turns into

$$\begin{aligned} y_a &= \sum_i \alpha_i^a \langle \phi(\mathbf{x}_i) - \bar{\phi}, \phi(\mathbf{x}) - \bar{\phi} \rangle \\ &= \sum_i \alpha_i^a \left[K(\mathbf{x}_i, \mathbf{x}) - \frac{1}{n} \sum_j K(\mathbf{x}_j, \mathbf{x}) - \frac{1}{n} \sum_j K(\mathbf{x}_i, \mathbf{x}_j) + \frac{1}{n^2} \sum_{i,j} K(\mathbf{x}_i, \mathbf{x}_j) \right] \end{aligned}$$

- Expression in brackets is the i 'th coefficient of $\mathbf{H} \left(\mathbf{k} - \frac{1}{n} \mathbf{K} \mathbf{1} \right)$
- V_k contains (unnormalized) $\boldsymbol{\alpha}^a$ as its a 'th column, so multiplication with V_k^T amounts to the sum, yields a vector of (unnormalized) y_a
- D_k^{-1} correctly normalizes the a 'th principal axis

Kernel PCA: Choice of Kernel

- Results of Kernel PCA can depend significantly on the **choice of kernel** and its parameters
 - When **class labels** are available, we might optimize a measure such as DSC / DC (Chapter 2)
 - We could also train a supervised classifier on kernel PCA output and use its cross-validation accuracy
 - **RBF kernel** $K(\mathbf{x}_i, \mathbf{x}_j) = e^{-\gamma \|\mathbf{x}_i - \mathbf{x}_j\|^2}$ has been found to work well in many applications, often used as a default choice
 - If attributes have been standardized, $\gamma = 1/p$ can be used as a rule of thumb
 - For exploratory data analysis, we might vary γ to see if “interesting” structures emerge

Kernel PCA: Intrinsic Dimensionality

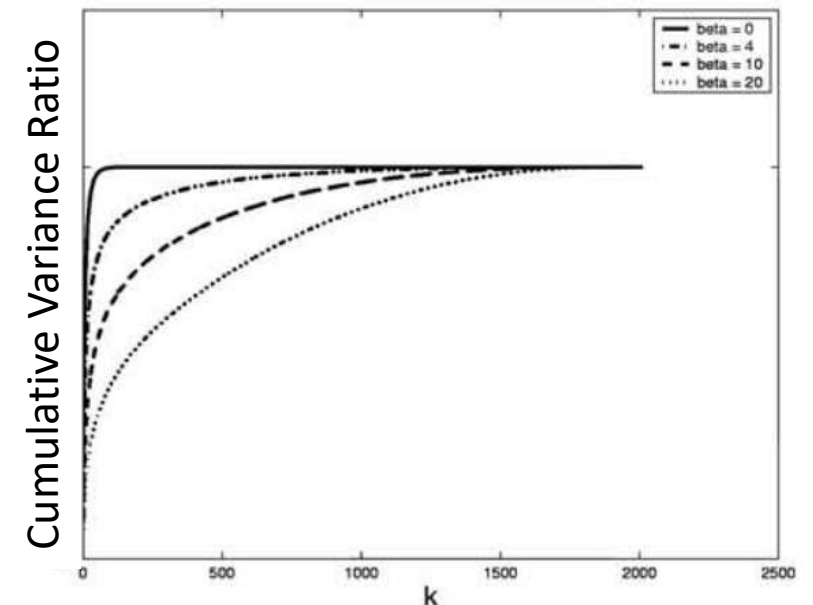
- **Observation:** The spectra of the outer and inner product matrices coincide.

- Consider SVD $Z = U\Sigma V^T$:

$$ZZ^T = U\Sigma V^T V \Sigma U^T = U\Sigma^2 U^T$$

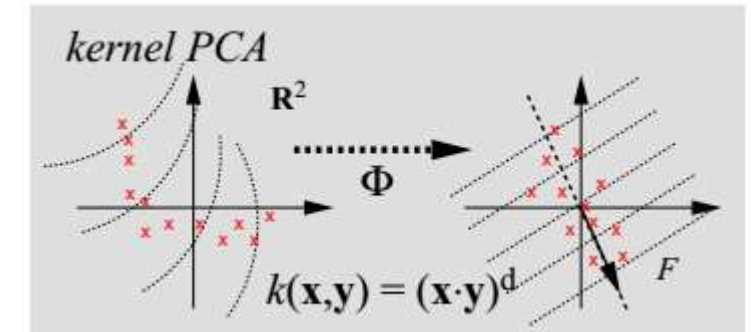
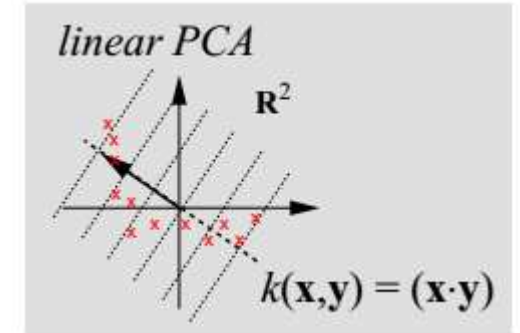
$$Z^T Z = V\Sigma U^T U \Sigma V^T = V\Sigma^2 V^T$$

- We can plot the ratio of explained variance, but it will refer to *the abstract feature space*
- “Elbow” in this plot will depend on the kernel



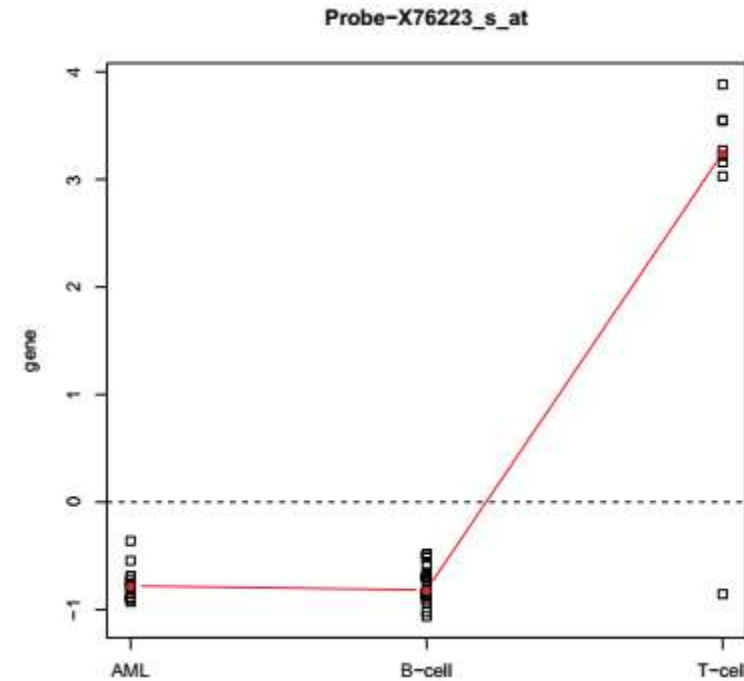
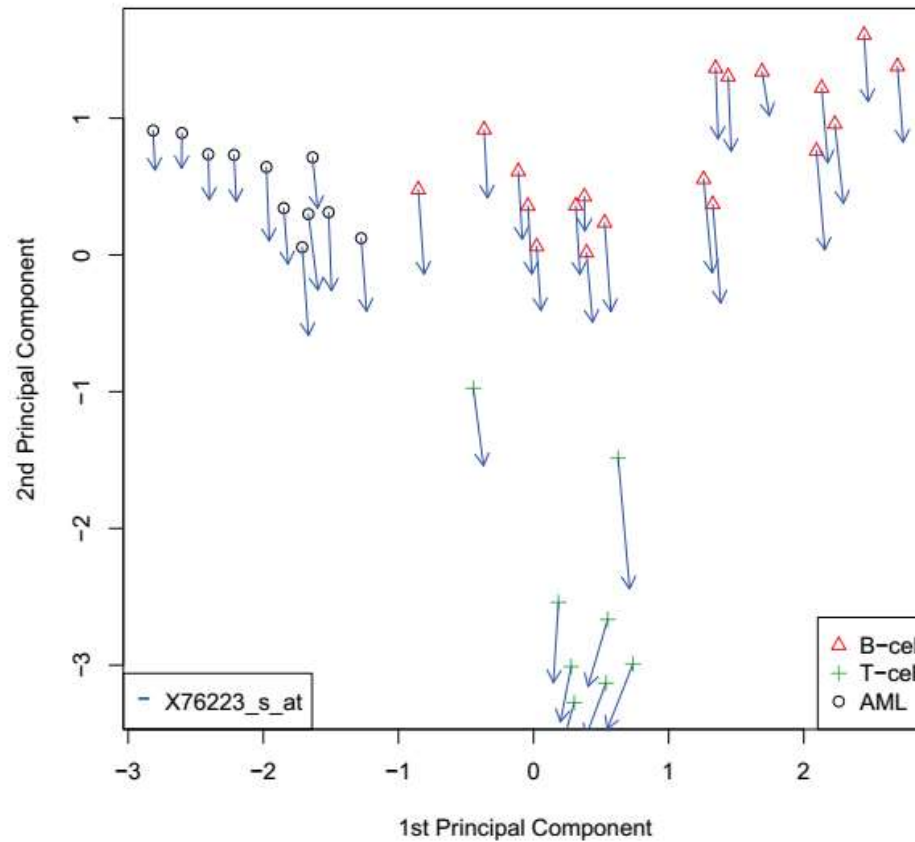
Kernel PCA: Advantages and Disadvantages

- **Advantages of Kernel PCA:**
 - Better reflects nonlinear structures
 - Given suitable kernels, can be applied to more abstract objects or to unfold manifolds (see next sections)
- **Disadvantages of Kernel PCA:**
 - Less interpretable: Principal modes are no longer a fixed combination of input variables
 - Unlike with linear PCA, it is not easy to find a vector that corresponds to given Kernel PCA coordinates („pre-image problem“)
- **Advantage / Disadvantage:**
 - Complexity grows with number of samples
 - If needed, cluster as a pre-process (e.g., k-means)



Example: Kernel PCA of Gene Expression Data

- Leukemia dataset with $n=19$, $p=3051$
- Kernel PCA with RBF kernel



Summary: (Kernel) PCA

- **Principal Component Analysis (PCA)**
 - finds orthogonal principal components
 - linear combinations of the original axes
 - data uncorrelated along principal components
 - maximize variance in projected data points
 - minimize approximation errors
 - can be computed efficiently in a dual form if $n < p$
- **Kernel PCA**
 - implicitly performs nonlinear mapping to a feature space to better capture nonlinear structures
 - can be efficiently implemented using the kernel trick
 - is less interpretable than linear PCA

Section 3.2:

Multidimensional Scaling

Multidimensional scaling (MDS)

Multidimensional scaling (MDS)

- A dimensionality reduction technique
- Maps pairwise dissimilarities to coordinates
- Provides model of non-geometric data
- Useful for
 - visualizing high-dimensional data
 - preprocessing data before clustering

Example: Multidimensional scaling (MDS)

Example: map of the US

Given a list of cities...

Chicago

Raleigh

Boston

Seattle

San Francisco

Austin

Orlando

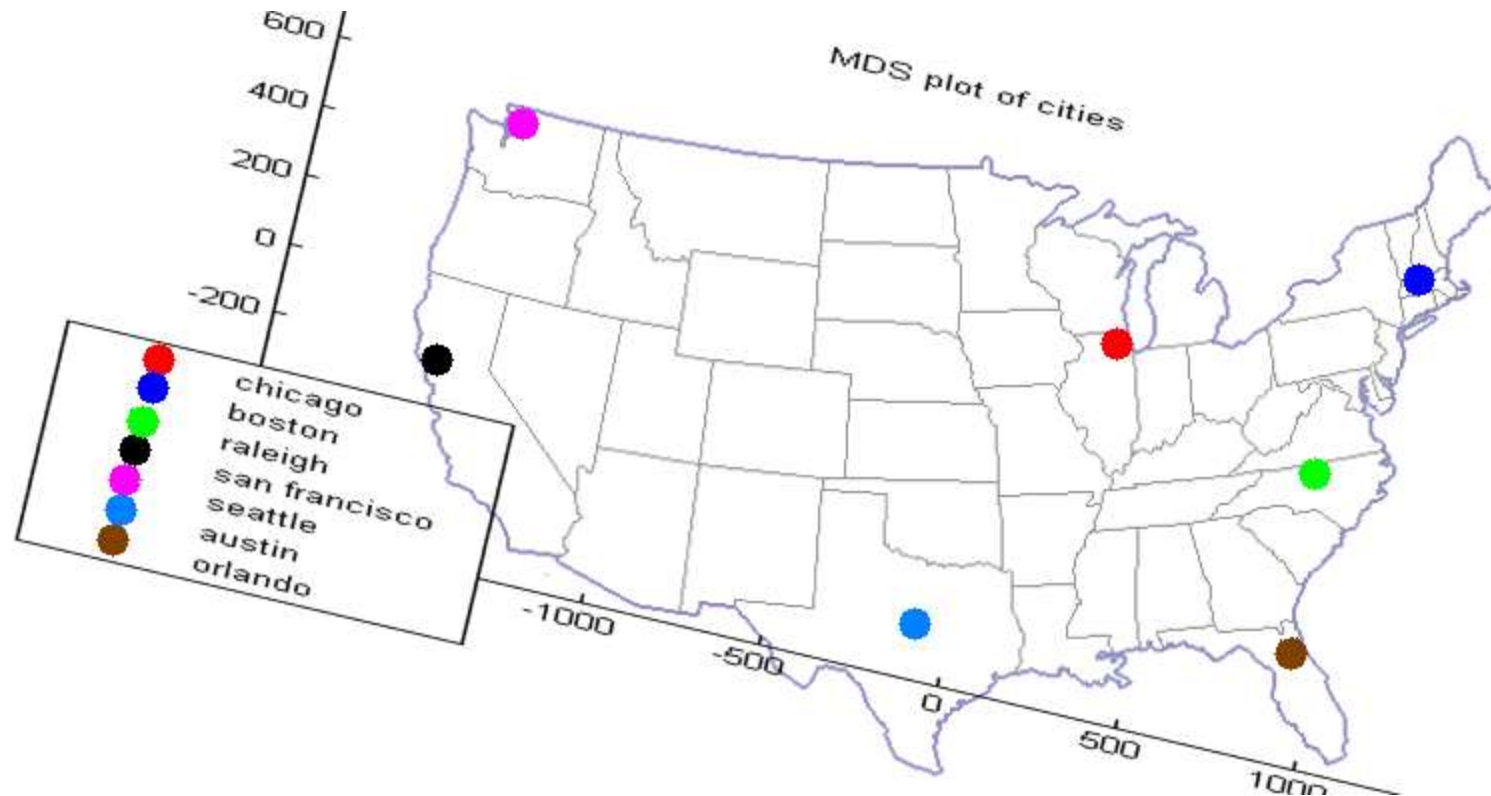
Example: Multidimensional scaling (MDS)

Knowing only the distances between them...

	Chicago	Raleigh	Boston	Seattle	San Francisco	Austin	Orlando
Chicago	0						
Raleigh	641	0					
Boston	851	608	0				
Seattle	1733	2363	2488	0			
San Francisco	1855	2406	2696	684	0		
Austin	972	1167	1691	1764	1495	0	
Orlando	994	520	1105	2565	2458	1015	0

Example: Multidimensional scaling (MDS)

Result: Given only the distances between points, MDS finds suitable coordinates for each point, of the specified dimension



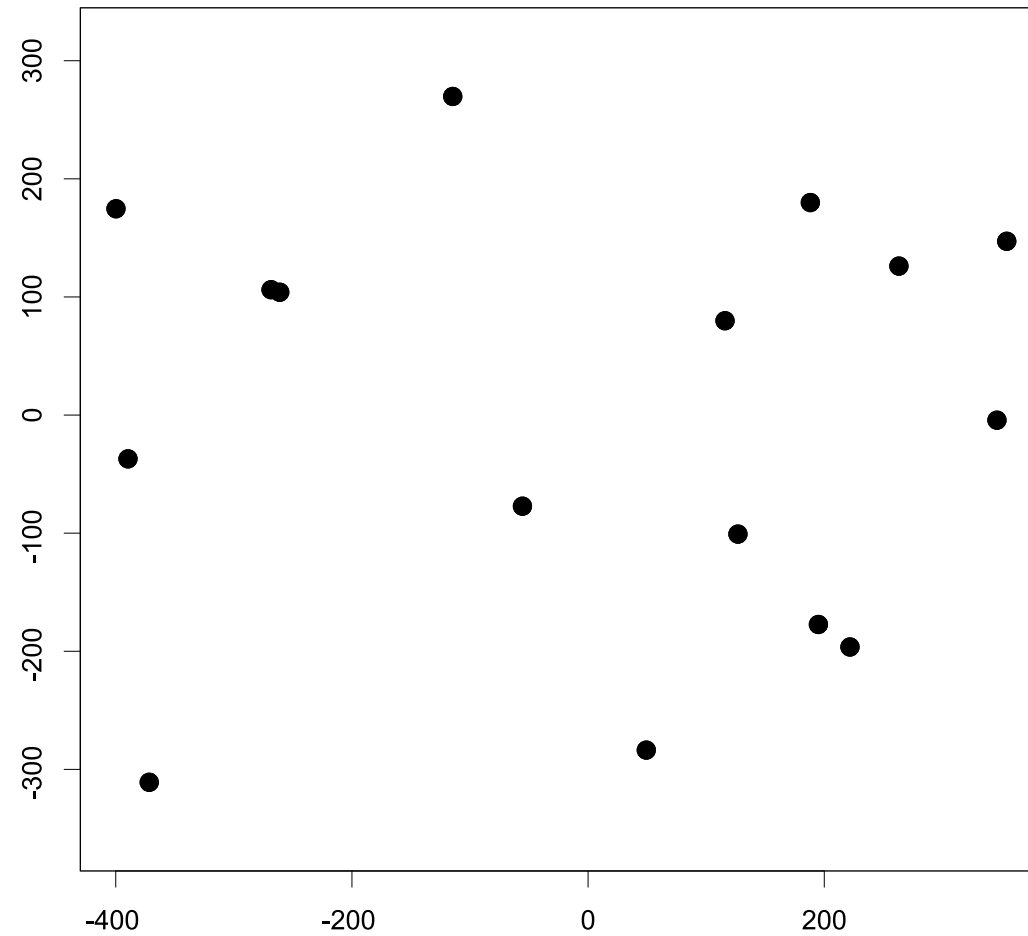
Example with non-Geodesic Dissimilarity Matrix

- Given: *street* distances between 16 German cities (according to a route planner)

	B	HB	DD	D	EF	HH	H	KI	MD	MZ	M	P	SB	SN	S	WI
Berlin (B)	0	392	193	559	303	289	286	353	156	580	585	36	722	211	633	570
Bremen (HB)	392	0	472	287	334	124	131	208	253	476	749	363	575	224	629	466
Dresden (DD)	193	472	0	615	218	500	367	542	232	496	465	211	638	400	510	486
Düsseldorf (D)	559	287	615	0	400	411	277	495	419	216	611	529	298	511	404	201
Erfurt (EF)	303	334	218	400	0	362	219	458	235	289	414	274	431	475	339	279
Hamburg (HH)	289	124	500	411	362	0	157	97	280	528	775	283	670	109	655	518
Hannover (H)	286	131	367	277	219	157	0	248	147	384	632	256	526	252	512	374
Kiel (KI)	353	208	542	495	458	97	248	0	376	624	872	348	766	175	752	614
Magdeburg (MD)	156	253	232	419	235	280	147	376	0	465	519	125	607	309	567	455
Mainz (MZ)	580	476	496	216	289	528	384	624	465	0	430	544	146	621	210	14
München (M)	585	749	465	611	414	775	632	872	519	430	0	555	487	757	232	424
Potsdam (P)	36	363	211	529	274	283	256	348	125	544	555	0	693	207	604	541
Saarbrücken (SB)	722	575	638	298	431	670	526	766	607	146	487	693	0	771	262	159
Schwerin (SN)	211	224	400	511	475	109	252	175	309	621	757	207	771	0	756	619
Stuttgart (S)	633	629	510	404	339	655	512	752	567	210	232	604	262	756	0	220
Wiesbaden (WI)	570	466	486	201	279	518	374	614	455	14	424	541	159	619	220	0

MDS: Non-Geodesic Example

- MDS result



MDS: Non-Geodesic Example

- MDS result rotated and overlaid on a geographic map



Multidimensional scaling: Basic Concepts

- **Input**

- Symmetric dissimilarity matrix, containing pair-wise dissimilarities $\delta_{ij} = \delta(x_i, x_j)$ between data samples $x_1, \dots, x_n$ that may or may not be from a Euclidean space
- Desired dimensionality k (often $k = 2, 3$)

- **Output**

- Image of data in a Euclidean frame $y_1, \dots, y_n$ such that pair-wise distances $d_{ij} = d(y_i, y_j)$ approximate the original dissimilarities $\delta(x_i, x_j)$ as accurately as possible in k dimensions

- **Embedding errors**

- Account for $n(n - 1)/2$ errors between the individual distances
- Invariant against rigid transformations (translation, rotation, mirroring)

Find $y_i \in \mathbb{R}^k$ such that $d(y_i, y_j) \approx \delta_{ij}$

MDS: What is a good approximation?

- Possible error functionals (“Stress”):

$$J_{ee} = \frac{\sum_{i < j} (d_{ij} - \delta_{ij})^2}{\sum_{i < j} \delta_{ij}^2} \quad \text{(punishes large absolute deviations)}$$

$$J_{ff} = \sum_{i < j} \left(\frac{d_{ij} - \delta_{ij}}{\delta_{ij}} \right)^2 \quad \text{(punishes large relative deviations)}$$

$$J_{ef} = \frac{1}{\sum_{i < j} \delta_{ij}} \sum_{i < j} \frac{(d_{ij} - \delta_{ij})^2}{\delta_{ij}} \quad \text{Compromise between } J_{ee} \text{ and } J_{ff} \\ \text{ („Sammon mapping“)}$$

(Commonly L2 distance measure $d_{ij} = \|y_i - y_j\|_2$ is used here.)

MDS: What is a good approximation?

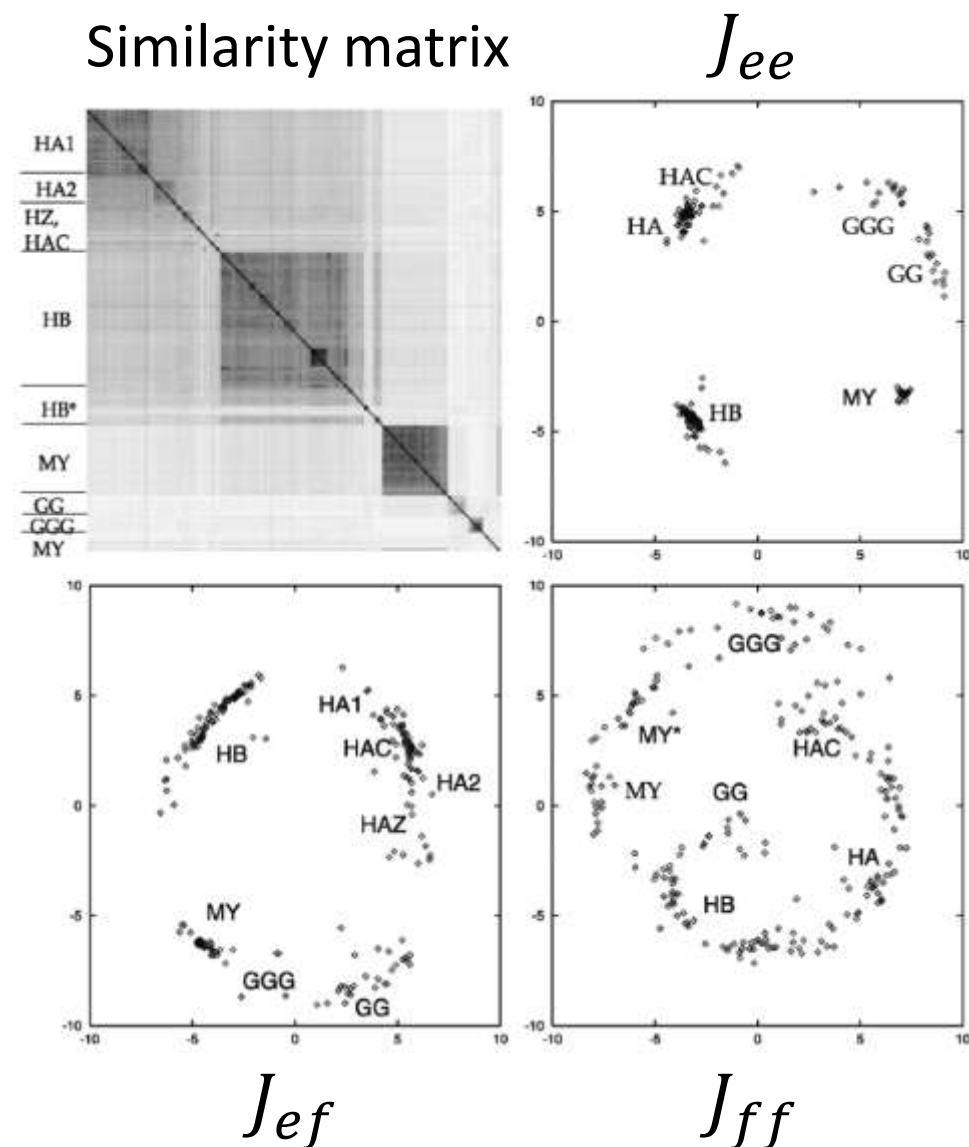
- Example: global vs. local
 - Similarities between 226 proteins as determined by sequence alignment

$$J_{ee} = \frac{\sum_{i < j} (d_{ij} - \delta_{ij})^2}{\sum_{i < j} \delta_{ij}^2}$$

$$J_{ff} = \sum_{i < j} \left(\frac{d_{ij} - \delta_{ij}}{\delta_{ij}} \right)^2$$

$$J_{ef} = \frac{1}{\sum_{i < j} \delta_{ij}} \sum_{i < j} \frac{(d_{ij} - \delta_{ij})^2}{\delta_{ij}}$$

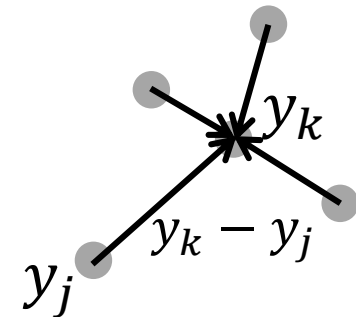
Image from [Klock et al. 2000]



MDS: Solution by Iterative Methods

- **Common solution:** Gradient descent
 - Initialize embedded points in random positions
 - Iteratively shift points to reduce the stress, until overall configuration stabilizes
 - Shifts are obtained by some constant times the negative derivatives of stress with respect to point positions
 - Can imagine points being connected by springs that attempt to keep them at the given distance; points will move until forces balance out

$$\begin{aligned}\frac{\partial J_{ee}}{\partial y_k} &= \frac{2}{\sum_{i < j} \delta_{ij}^2} \sum_{j \neq k} (d_{kj} - \delta_{kj}) \frac{y_k - y_j}{d_{kj}} \\ \frac{\partial J_{ff}}{\partial y_k} &= 2 \sum_{j \neq k} \frac{d_{kj} - \delta_{kj}}{\delta_{kj}^2} \cdot \frac{y_k - y_j}{d_{kj}} \\ \frac{\partial J_{ef}}{\partial y_k} &= \frac{2}{\sum_{i < j} \delta_{ij}} \sum_{j \neq k} \frac{d_{kj} - \delta_{kj}}{\delta_{kj}} \cdot \frac{y_k - y_j}{d_{kj}}\end{aligned}$$



Alternative: Distance-based Kernel PCA

- Assume dissimilarity matrix $\Delta_{SE} = (\delta_{ij}^2)$ contains squared Euclidean distances between points in an unknown feature space:

$$\{\Phi_i \in \mathbb{R}^q, i = 1, \dots, n\}$$

- In this case, distances relate to inner products as:

$$\delta_{ij}^2 = \|\Phi_i - \Phi_j\|^2 = \|\Phi_i\|^2 + \|\Phi_j\|^2 - 2\langle \Phi_i, \Phi_j \rangle$$

- *Idea*: If we can transform Δ_{SE} into an inner product matrix, we can find an embedding using kernel PCA
 - *Note*: Similar to MDS, this computes an embedding from distances. However, it yields a different layout, since maximizing preserved variance is not the same as optimizing MDS stress.

Centered Inner Products from Squared Distances

- Kernel PCA requires inner product matrix of centered features Φ_i

$$K^c = [\langle \Phi_i, \Phi_j \rangle]$$

whose coefficients relate to squared distances as

$$\delta_{ij}^2 = K_{ii} + K_{jj} - 2K_{ij}$$

- Centered features $\sum_i \Phi_i = 0$ yield $\sum_i K_{ij} = 0$ and thus:

$$\frac{1}{N} \sum_i \delta_{ij}^2 = \frac{1}{N} \sum_i K_{ii} + K_{jj} \quad := \delta_{\oplus j}^2$$

$$\frac{1}{N} \sum_j \delta_{ij}^2 = K_{ii} + \frac{1}{N} \sum_j K_{jj} \quad := \delta_{i \oplus}^2$$

$$\frac{1}{N^2} \sum_i \sum_j \delta_{ij}^2 = \frac{2}{N} \sum_i K_{ii} \quad := \delta_{\oplus \oplus}^2$$

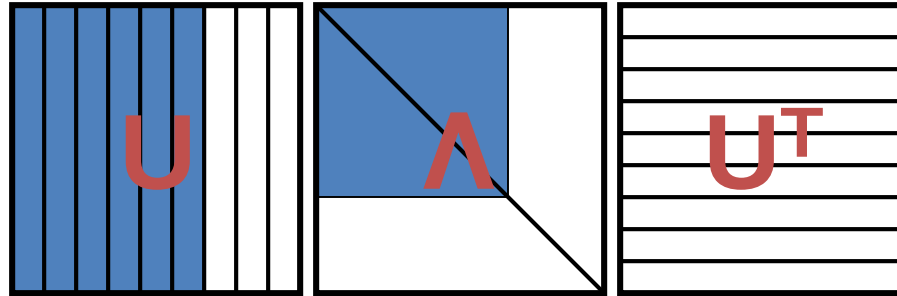
- Conclusion:* We should transform squared distances according to

$$K_{ij} = -\frac{1}{2} (\delta_{ij}^2 - \delta_{i \oplus}^2 - \delta_{\oplus j}^2 + \delta_{\oplus \oplus}^2)$$

Distance-based Kernel PCA: Computation

1. Compute $K^c = -\frac{1}{2}H\Delta_{SE}H$ with $H = I - \frac{1}{N}\mathbf{1}\mathbf{1}^T$
 - If Δ_{SE} conforms to a metric, this defines K^c such that $K^c = [\langle \Phi_i, \Phi_j \rangle]$
2. Compute spectral decomposition of K^c

– $K^c = U\Lambda U^T$



3. Lower dimensional embedding:

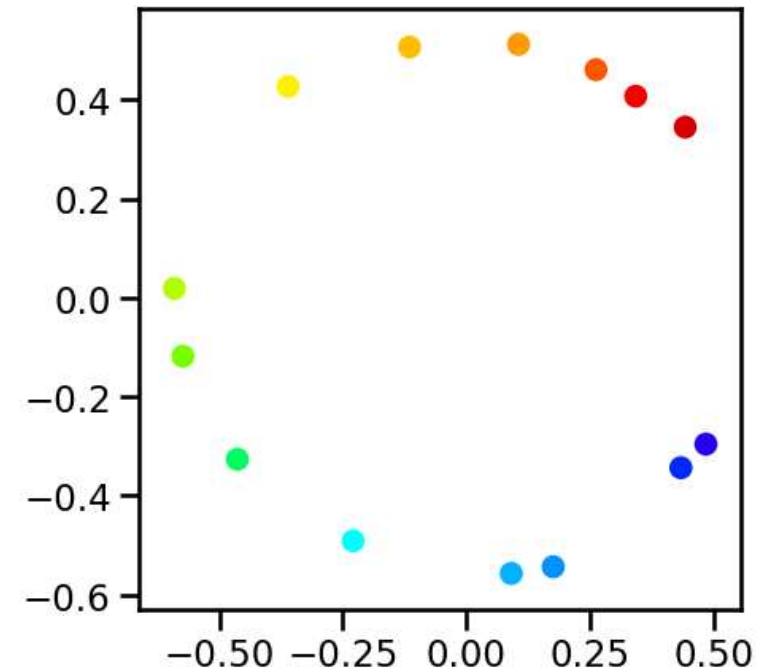
- Want k -dimensional coordinates
- U_k = first k columns of U
- Λ_k = $k \times k$ submatrix of Λ
- Desired coordinates = $U_k \Lambda_k^{1/2}$ (follows from equation for kernel PCA projection)

Quiz: Where could this algorithm fail when given a non metric dissimilarity matrix as input?

Example: Color Perception

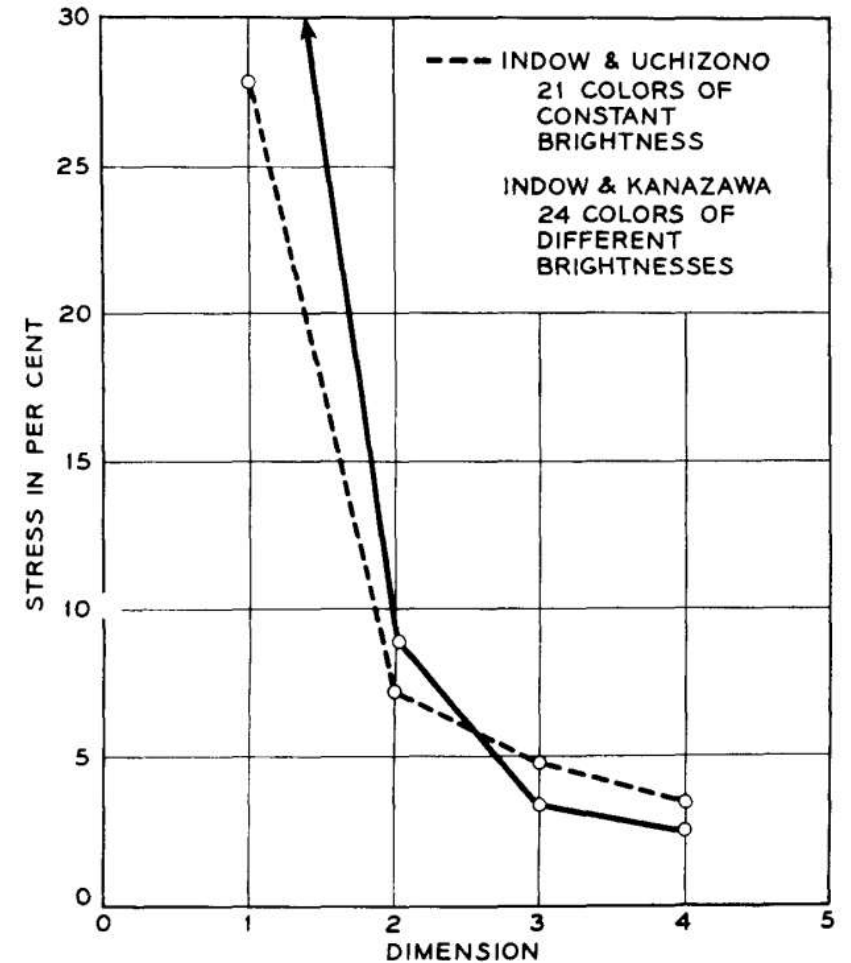
- Experimental data collected by [Ekman 1954], visualized with MDS:

	434	445	465	472	490	504	537	555	584	600	610	628	651	674
434	0	0.14	0.58	0.58	0.82	0.94	0.93	0.96	0.98	0.93	0.91	0.88	0.87	0.84
445	0.14	0	0.5	0.56	0.78	0.91	0.93	0.93	0.98	0.96	0.93	0.89	0.87	0.86
465	0.58	0.5	0	0.19	0.53	0.83	0.9	0.92	0.98	0.99	0.98	0.99	0.95	0.97
472	0.58	0.56	0.19	0	0.46	0.75	0.9	0.91	0.98	0.99	1	0.99	0.98	0.96
490	0.82	0.78	0.53	0.46	0	0.39	0.69	0.74	0.93	0.98	0.98	0.99	0.98	1
504	0.94	0.91	0.83	0.75	0.39	0	0.38	0.55	0.86	0.92	0.98	0.98	0.98	0.99
537	0.93	0.93	0.9	0.9	0.69	0.38	0	0.27	0.78	0.86	0.95	0.98	0.98	1
555	0.96	0.93	0.92	0.91	0.74	0.55	0.27	0	0.67	0.81	0.96	0.97	0.98	0.98
584	0.98	0.98	0.98	0.98	0.93	0.86	0.78	0.67	0	0.42	0.63	0.73	0.8	0.77
600	0.93	0.96	0.99	0.99	0.98	0.92	0.86	0.81	0.42	0	0.26	0.5	0.59	0.72
610	0.91	0.93	0.98	1	0.98	0.98	0.95	0.96	0.63	0.26	0	0.24	0.38	0.45
628	0.88	0.89	0.99	0.99	0.99	0.98	0.98	0.97	0.73	0.5	0.24	0	0.15	0.32
651	0.87	0.87	0.95	0.98	0.98	0.98	0.98	0.98	0.8	0.59	0.38	0.15	0	0.24
674	0.84	0.86	0.97	0.96	1	0.99	1	0.98	0.77	0.72	0.45	0.32	0.24	0



MDS: Intrinsic Dimensionality

- **Rules of thumb** for selecting dimensionality of the embedding space [Kruskal 1964]:
 1. “Elbow” when plotting stress vs. dimension
 2. Prefer interpretable embeddings
 3. Consider expected level of noise in the data



Example: Full-Brain Tractography

Problem: Neural tracts present themselves as a dense set of entangled curves. How to interact with them (e.g., make a selection) in a simple way?

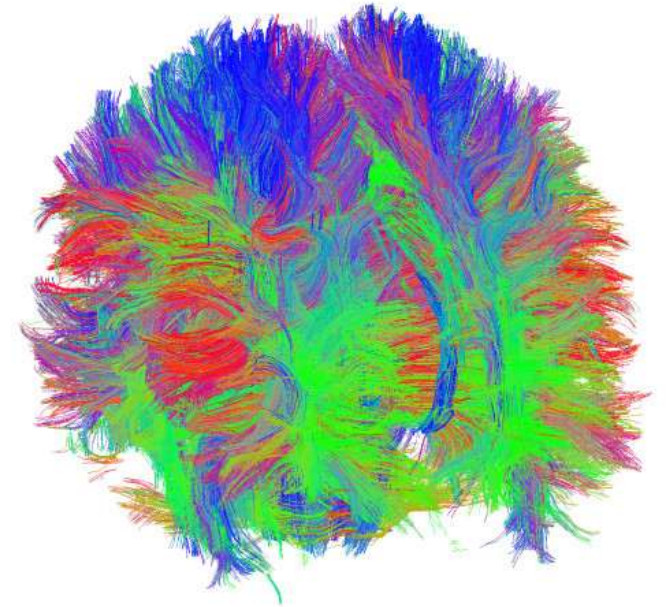


Image by Tobias Isenberg

Idea: It is much simpler to interact with 2D views! Can't we represent each streamline as a point in 2D image space, such that similar streamlines are placed nearby?

Linked 2D Views

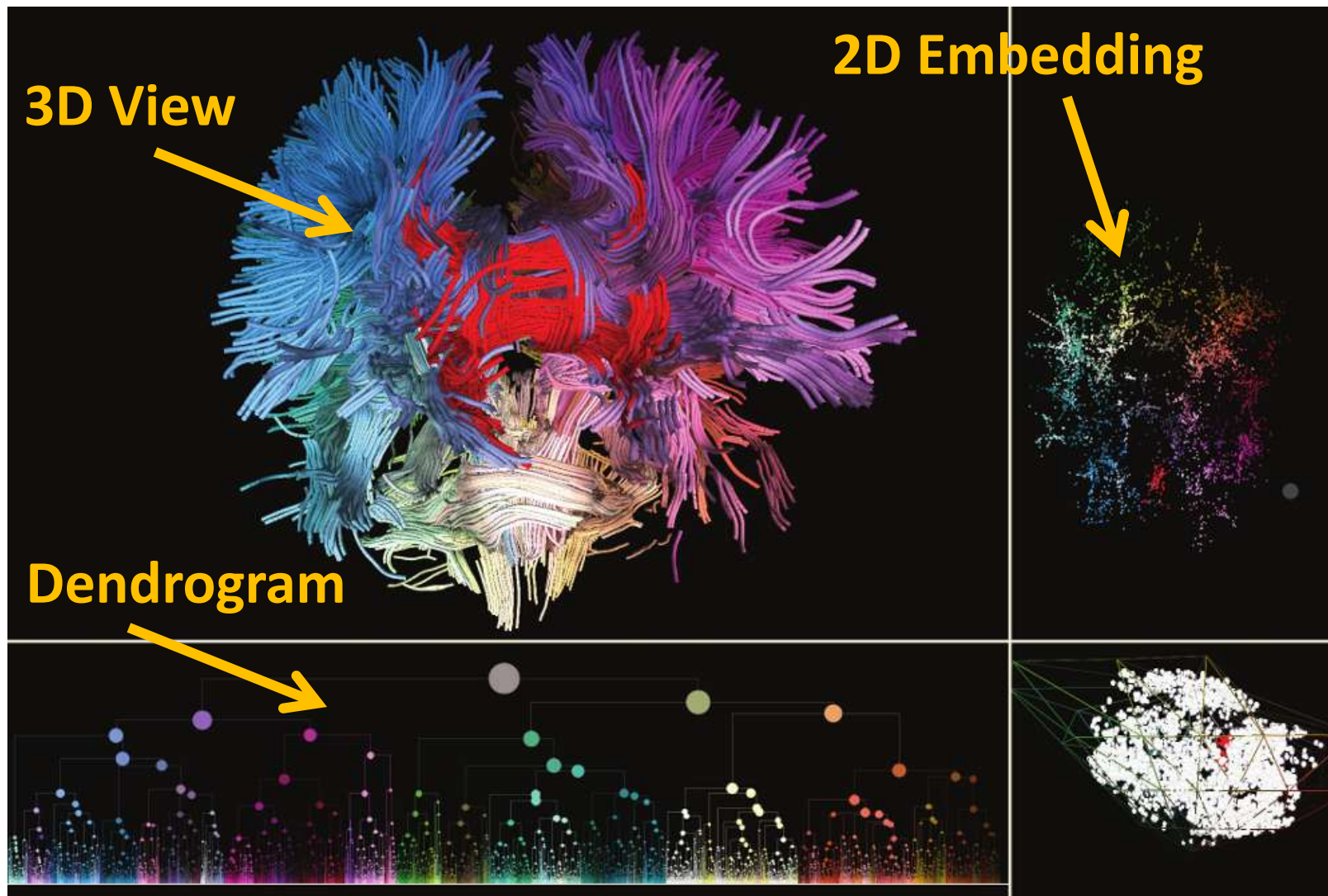


Image taken from [Jianu et al. 2009]

Summary: MDS

- **Multidimensional scaling** computes embeddings based on dissimilarities alone
 - Effort independent of dimensionality
 - Solution unique only up to rigid transformation and reflection
- **Gradient descent-based implementation**
 - Works with all dissimilarity matrices (including non-metric ones)
 - Defining different stress functions allows us to focus on absolute or relative errors
 - Random init, results can vary for the same input
 - Advanced optimization: SMACOF (outside our scope)
- **(Kernel) PCA can also compute embeddings from distances**
 - If dissimilarity measure corresponds to a squared Euclidean distance

Section 3.3: Manifold Learning

ISOMAP and Laplacian Eigenmaps

Introduction: Manifolds

- *Definition:* An **n -manifold** is a topological space that, at each point, has a neighborhood that is homeomorphic to n -dimensional Euclidean space.

- *Examples:*

- Curves (1-manifold)
- Surfaces (2-manifold)
- May not self-intersect

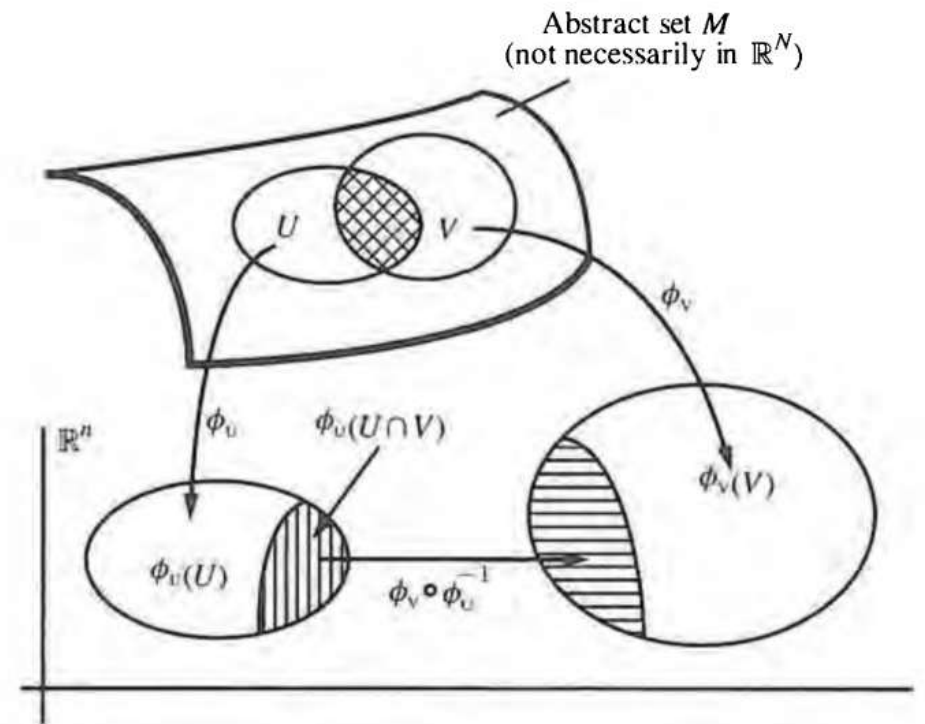
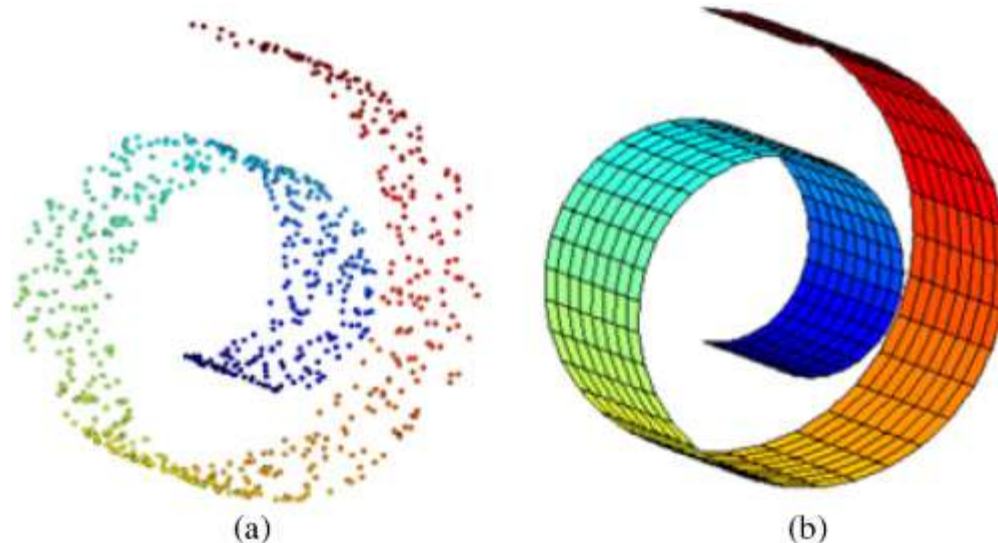


Figure from ncatlab.org

Motivation: Manifold Learning

- In practice, high-dimensional data is often restricted to a lower-dimensional manifold
- **Manifold learning** aims to infer that manifold from the observed samples
 - This motivated kernel PCA, but the kernels we discussed so far (e.g., RBF) are based on Euclidean distances, which is not suitable for manifold learning

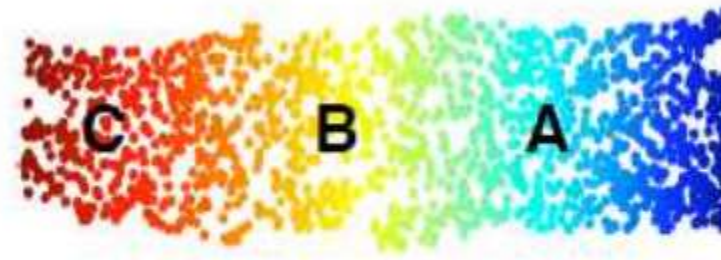


Distances on Nonlinear Manifolds

- On **nonlinear manifolds**, we should use **geodesic distances** (length of shortest connection on manifold), not Euclidean:



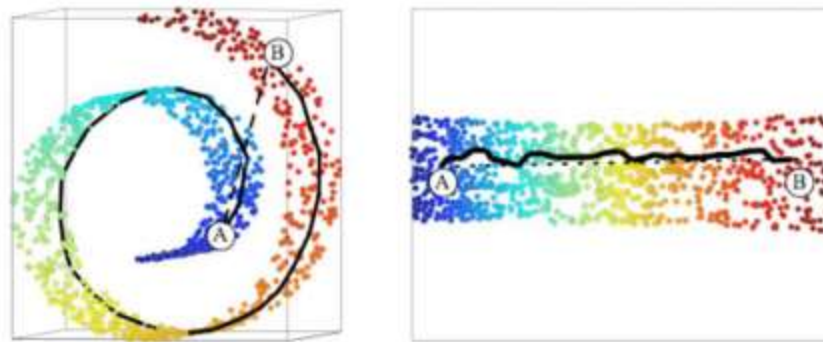
$$d(A,C) < d(A,B)$$



$$d(A,C) > d(A,B)$$

Idea: Estimating Geodesic Distance

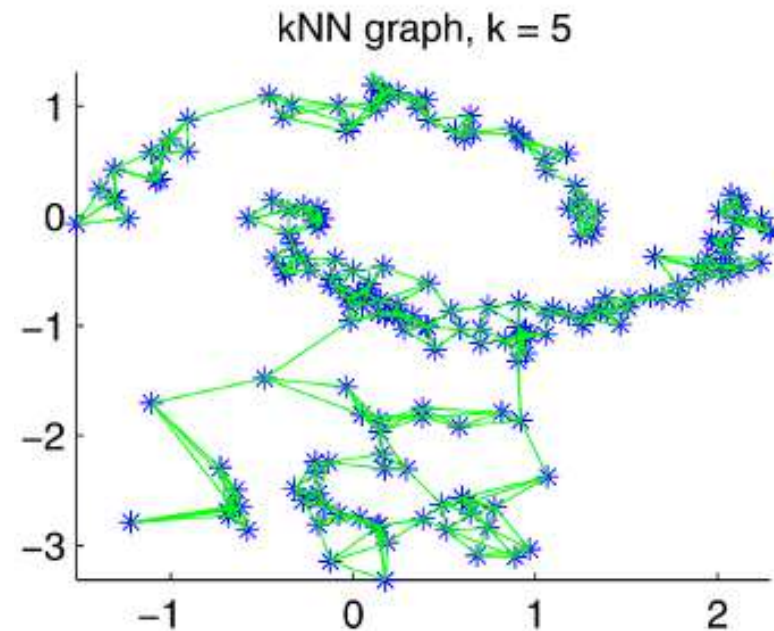
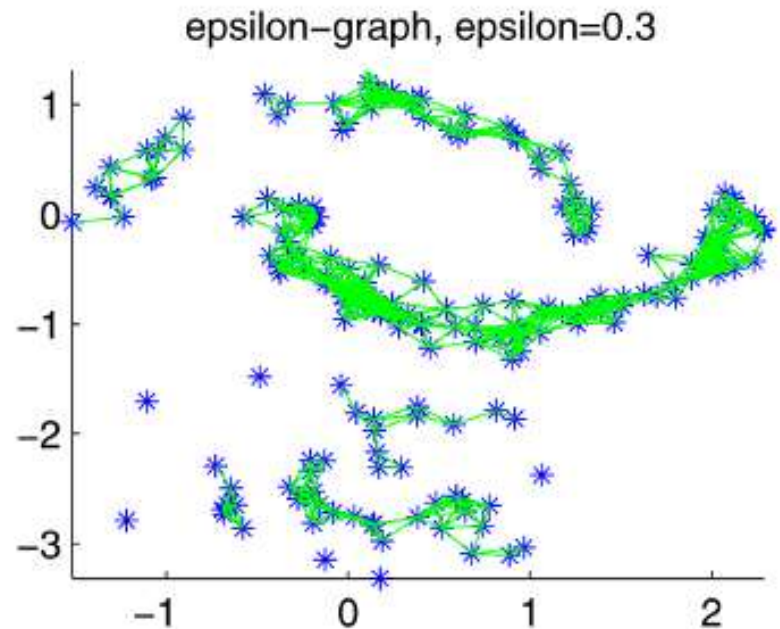
- We are usually not given the manifold, only a set of discrete input points
- **Estimate geodesic distances:**
 - Use Euclidean distance for nearby points
 - Connect distant points by short hops between nearby points, measure length of shortest path



Neighborhood Graphs

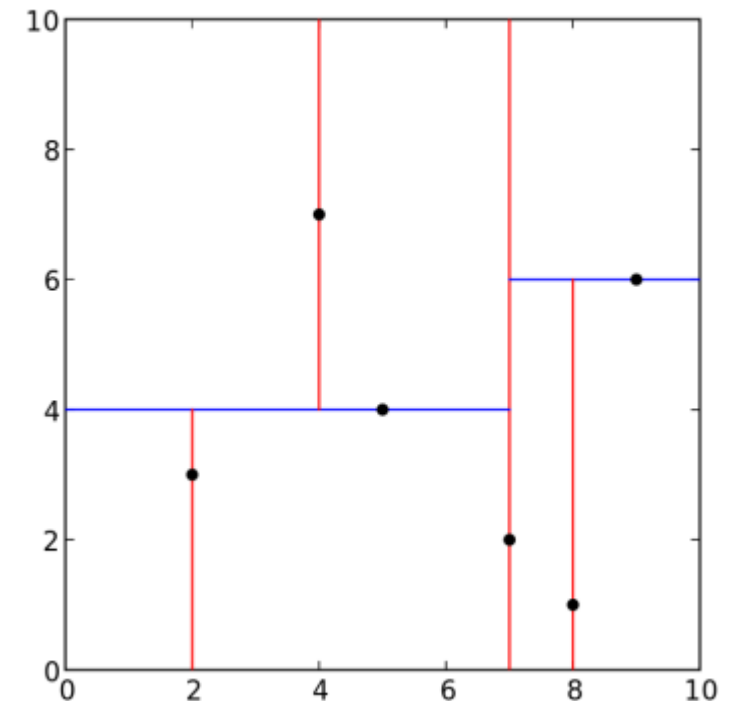
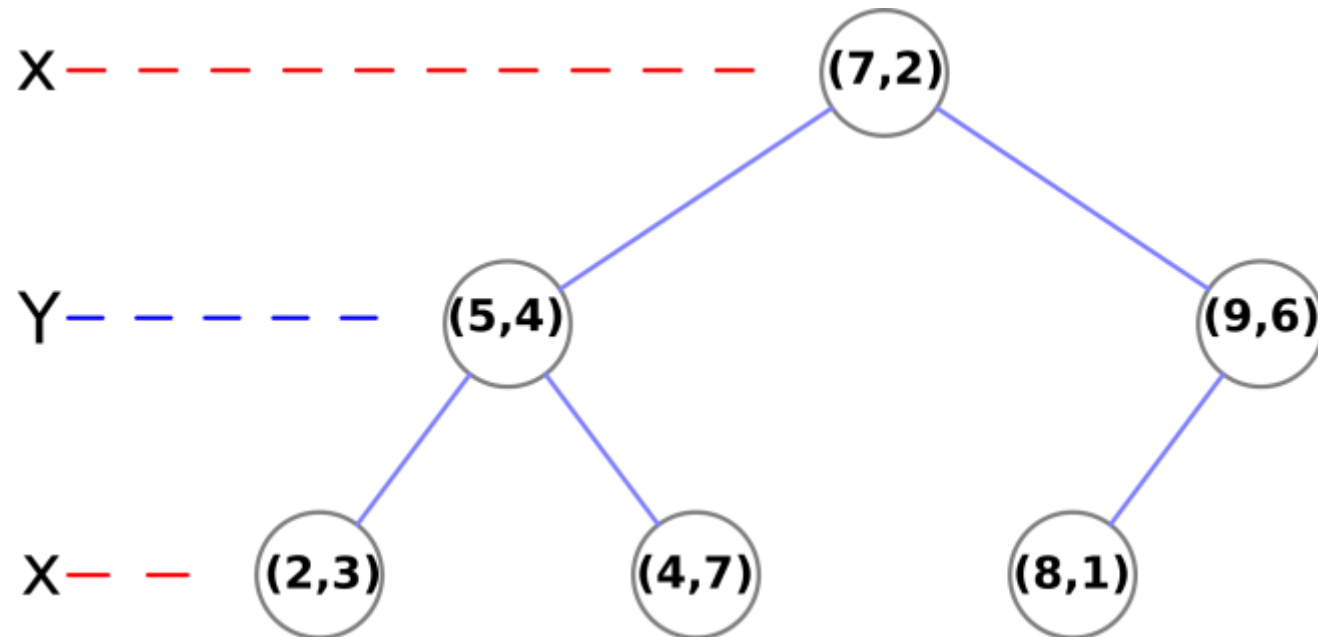
Neighborhood graph (V, E) : Each data point is a vertex V , edges connect local neighbors.

- **ϵ -graph**: Connect to all data points within distance ϵ
- **k NN**: Connect each data point to its k nearest neighbors



KD Trees

- **KD trees** [Bentley 1975] generalize balanced binary search trees to multiple dimensions
 - Splits at different levels alternate between dimensions
 - Can be used to reduce the effort for nearest neighbor search

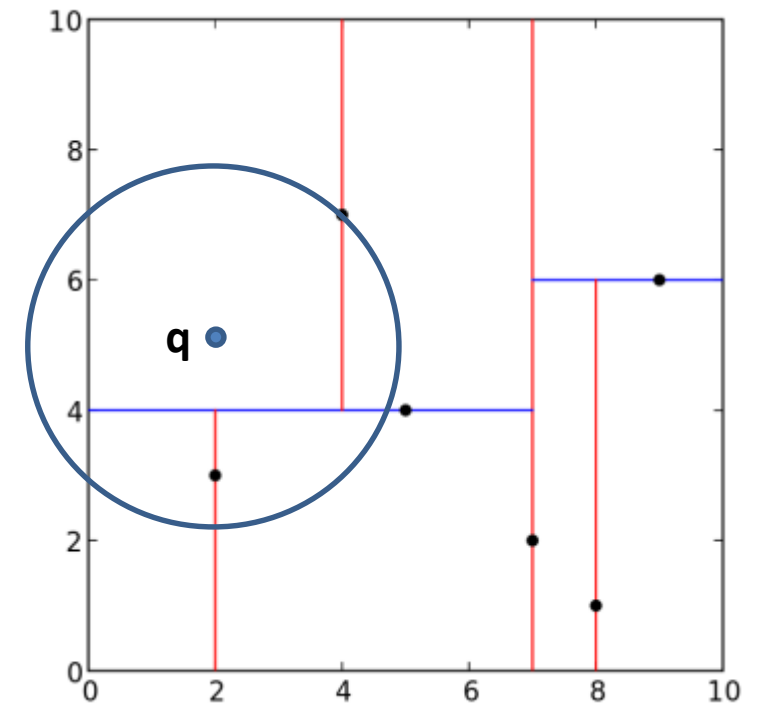
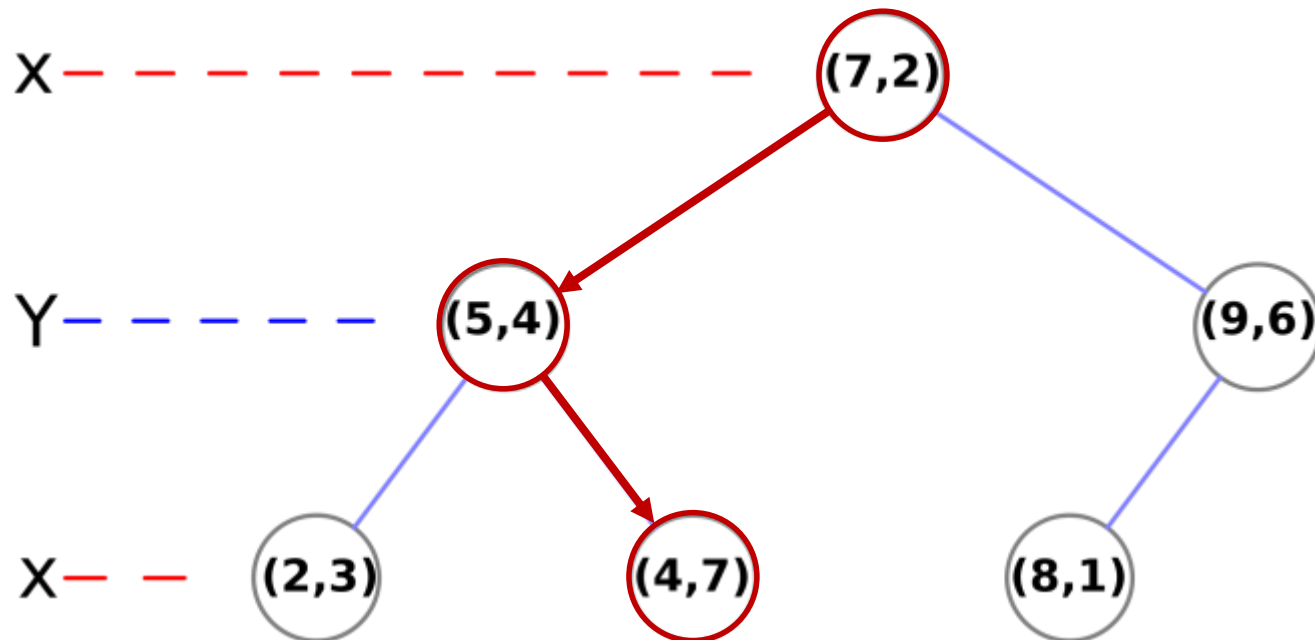


Nearest Neighbor Search in KD Trees

- 1. Downwards pass:** Starting at the root
 - traverse the tree in the direction of the query point $\mathbf{q}$
 - store the leaf as the current best neighbor $\mathbf{p}$

Example: Nearest Neighbor Search in KD Trees

- Downwards pass for $\mathbf{q} = (2,5)$
 - Initial best guess: $\mathbf{p} = (4,7), r = 2.83$

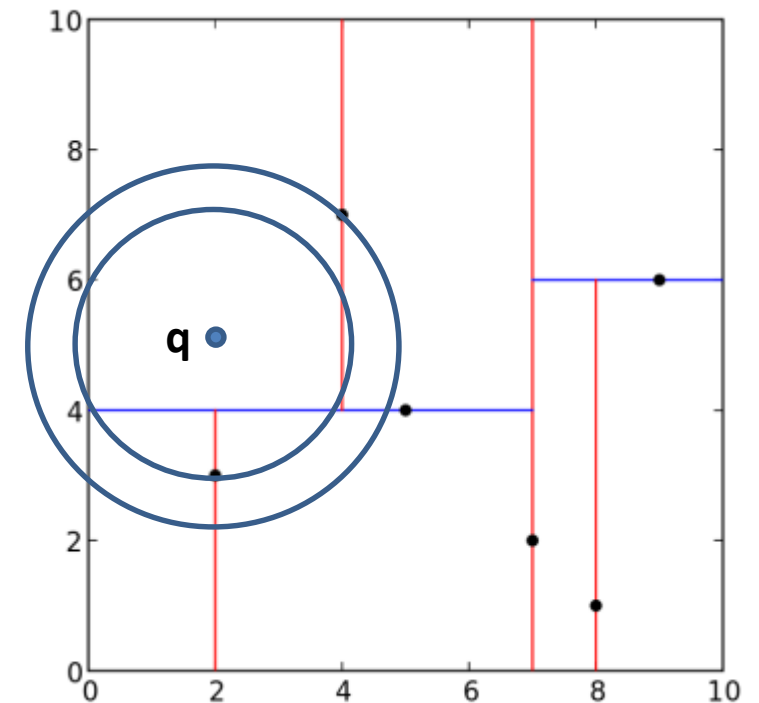
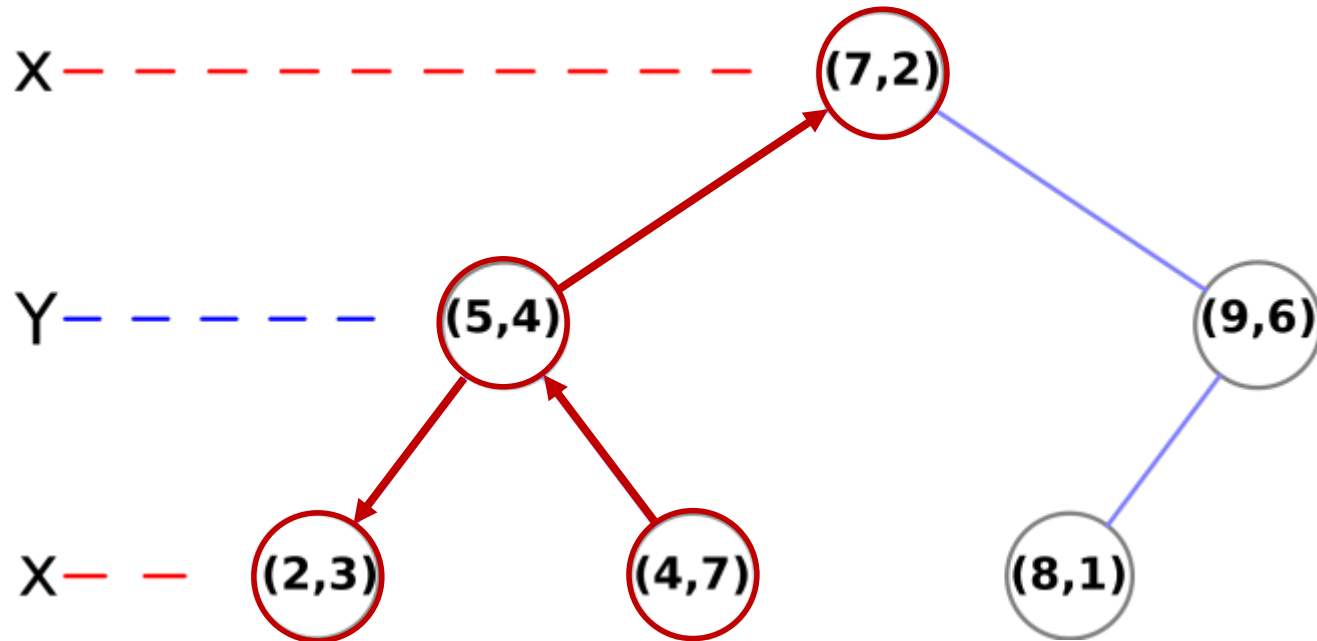


Nearest Neighbor Search in KD Trees

1. **Downwards pass:** Starting at the root
 - traverse the tree in the direction of the query point $\mathbf{q}$
 - store the leaf as the current best neighbor $\mathbf{p}$
2. **Upwards pass:** Starting at the leaf
 - if we encounter a closer point, it becomes the current best $\mathbf{p}$
 - if a sphere around $\mathbf{q}$ with radius $r = \|\mathbf{p} - \mathbf{q}\|$ crosses the splitting plane, we also need to traverse the other branch

Example: Nearest Neighbor Search in KD Trees

- Upwards pass for $\mathbf{q} = (2,5)$
 - Initial best guess: $\mathbf{p} = (4,7), r = 2.83$
 - Updated best guess: $\mathbf{p} = (2,3), r = 2$
 - Final answer: $\mathbf{p} = (2,3)$

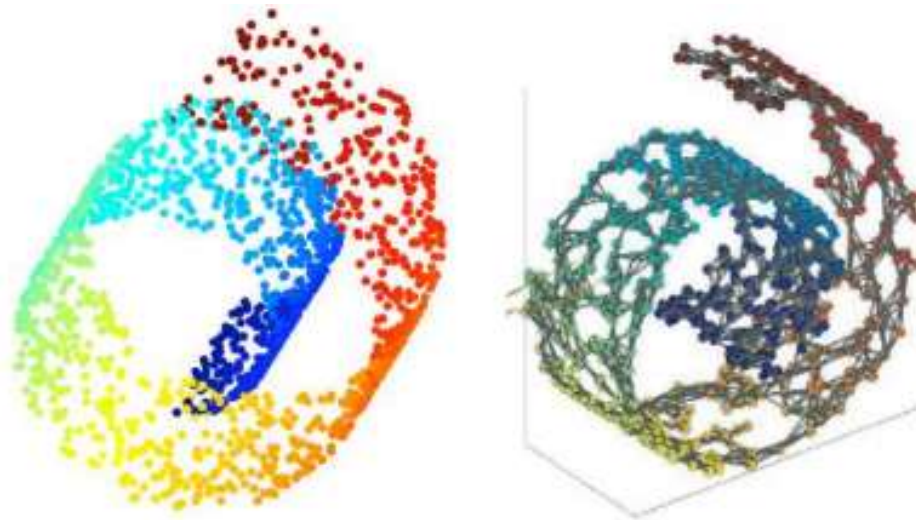


Nearest Neighbor Search in KD Trees

1. **Downwards pass:** Starting at the root
 - traverse the tree in the direction of the query point $\mathbf{q}$
 - store the leaf as the current best neighbor $\mathbf{p}$
2. **Upwards pass:** Starting at the leaf
 - if we encounter a closer point, it becomes the current best $\mathbf{p}$
 - if a sphere around $\mathbf{q}$ with radius $r = \|\mathbf{p} - \mathbf{q}\|$ crosses the splitting plane, we also need to traverse the other branch
- For a k nearest neighbor search, we maintain a priority queue of k closest points $\mathbf{p}_i$, search radius r accounts for the largest distance ($r = \infty$ while queue contains fewer than k points)

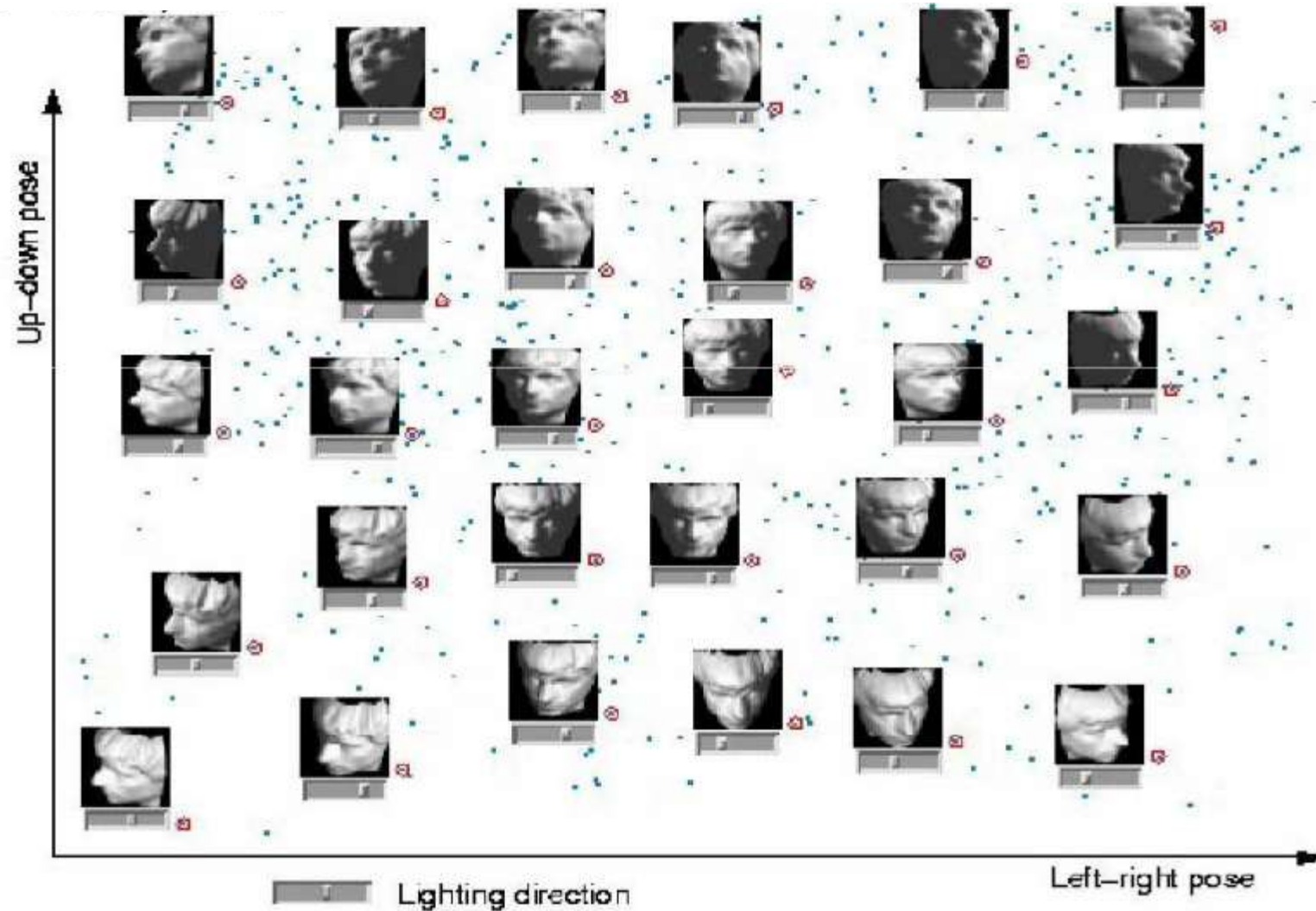
ISOMAP Algorithm

1. Build a neighborhood graph (e.g., k-NN) that approximates the manifold structure
 - Make sure it's connected
2. For the resulting graph, compute all-pairs shortest paths
3. Perform MDS based on resulting distance matrix



ISOMAP: Example result

- $n=698$, $p=4096$



ISOMAP: Intrinsic Dimensionality

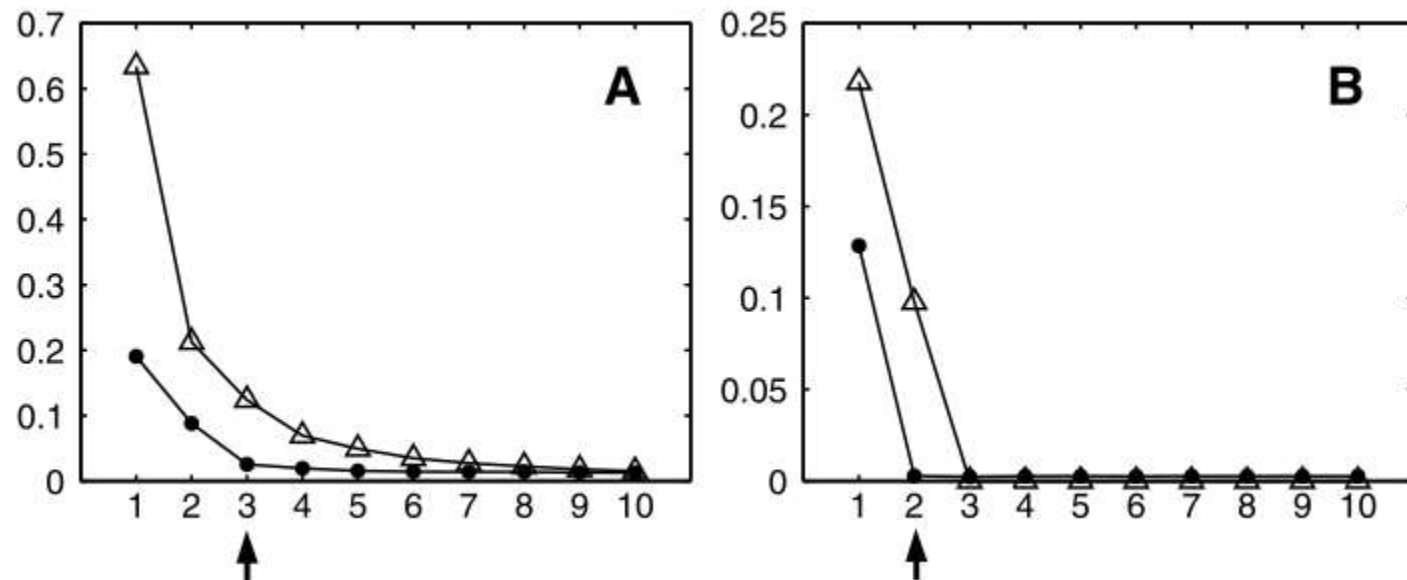
- **Estimating intrinsic dimensionality** is a difficult problem and a topic of ongoing research
 - It might vary within a given dataset
- Method proposed in ISOMAP paper: Plot “residual variance” vs. dimensionality

$$1 - R^2(\hat{D}_M, D_Y)$$

- $\hat{D}_M$: Algorithm-specific estimate of intrinsic distances
 - Euclidean for PCA, shortest path distance for ISOMAP
- D_Y : Distances in low-dimensional embedding
- R^2 : Squared Pearson correlation coefficient

Examples: Intrinsic Dimensionalities

- Results of plotting $1 - R^2(\hat{D}_M, D_Y)$ as a function of dimensionality in two examples:
 - A: Face images (pose and illumination)
 - B: Swiss roll data
 - Triangles: PCA / Circles: ISOMAP

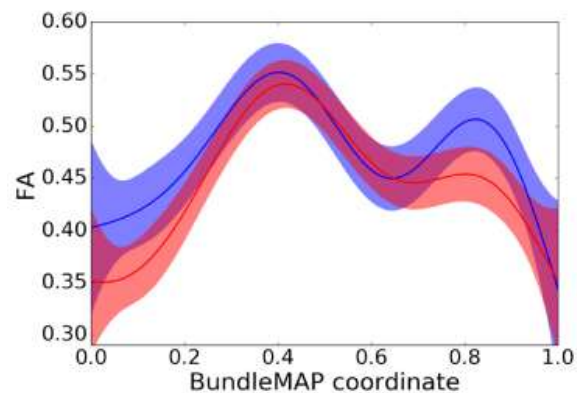


ISOMAP for Fiber Bundle Parametrization

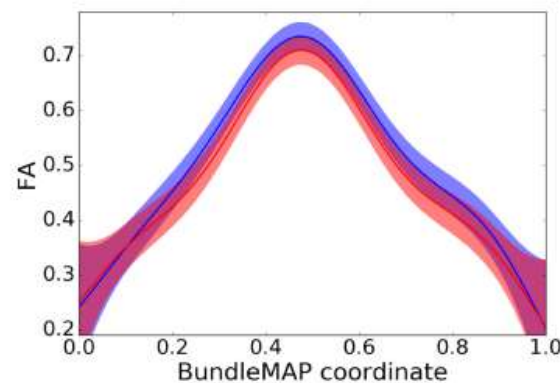
- Joint parametrization based on ISOMAP:



- FA as a function of position along the tract:



(a) Left IFO



(b) Genu of the *corpus callosum*



Laplacian Eigenmaps

- **Laplacian Eigenmaps** embed points based on the same neighborhood graphs as ISOMAP
 - Often, edge between points $\mathbf{x}_i$ and $\mathbf{x}_j$ is weighted according to their distance

$$w_{ij} = \exp\left(-\frac{\|\mathbf{x}_i - \mathbf{x}_j\|^2}{2\sigma^2}\right)$$

- Instead of preserving geodesic distances, we aim to keep neighbors on the manifold close to each other by minimizing

$$\sum_{i=1}^n \sum_{j=1}^n w_{ij} \|\mathbf{y}_i - \mathbf{y}_j\|^2$$

with a normalization that prevents a collapse of all $\mathbf{y}_i$ to a point

Graph Laplacian

Define:

- $n = |V|$ (number of data points)
- Affinity matrix **W** (symmetric, $n \times n$)
 - w_{ij} contains weight of edge between i and j
- Degree matrix **D** (diagonal, $n \times n$)
 - sum of adjacent edge weights of vertex i : $d_i = \sum_j w_{ij}$
- Graph Laplacian: **L** = **D** – **W**
 - “Unnormalized” Laplacian

Objective Function in Terms of Laplacian

- In 1D, defining $\mathbf{Y} \in \mathcal{R}^n$ with y_i as i th coefficient allows us to write

$$\sum_{i=1}^n \sum_{j=1}^n w_{ij} (y_i - y_j)^2 = 2 \mathbf{Y}^T \mathbf{L} \mathbf{Y}$$

- *Proof:*

$$\begin{aligned} \mathbf{Y}^T \mathbf{L} \mathbf{Y} &= \mathbf{Y}^T \mathbf{D} \mathbf{Y} - \mathbf{Y}^T \mathbf{W} \mathbf{Y} = \sum_i d_i y_i^2 - \sum_i \sum_j y_i y_j w_{ij} \\ &= \frac{1}{2} \left(\sum_i d_i y_i^2 - 2 \sum_i \sum_j y_i y_j w_{ij} + \sum_j d_j y_j^2 \right) \\ &= \frac{1}{2} \sum_i \sum_j w_{ij} (y_i - y_j)^2 \end{aligned}$$

Properties of the Graph Laplacian

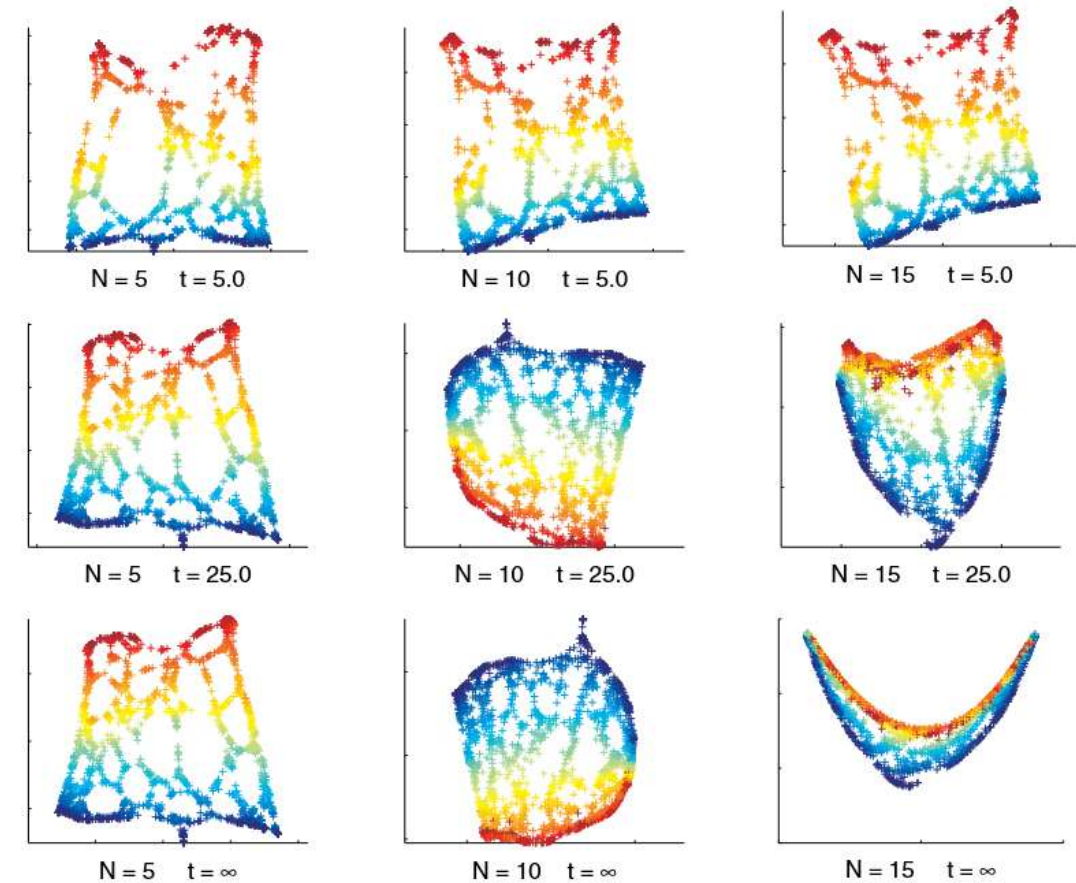
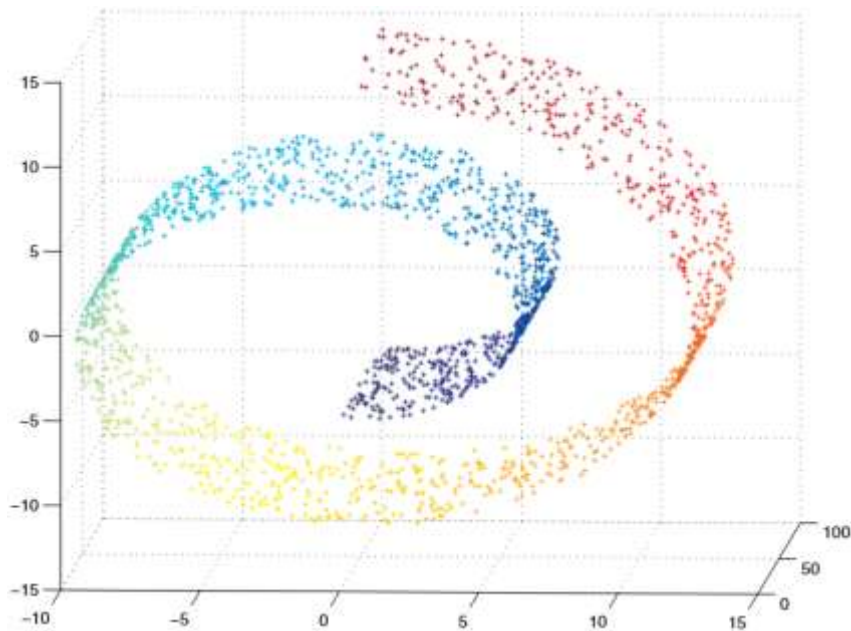
- **L** is symmetric and positive semi-definite
 - Symmetry: Direct consequence of definition
 - Semi-definiteness: Consequence of $\mathbf{Y}^T \mathbf{L} \mathbf{Y} = \frac{1}{2} \sum_i \sum_j w_{ij} (y_i - y_j)^2$ and non-negativity of w_{ij}
- The constant one vector **1** is an eigenvector of **L** with corresponding eigenvalue 0
 - Direct consequence of definition $\mathbf{L} = \mathbf{D} - \mathbf{W}$

Computing Laplacian Eigenmaps: 1D Case

- In 1D, Laplacian Eigenmaps minimize $\mathbf{Y}^T \mathbf{L} \mathbf{Y}$ under two constraints that prevent a collapse of all y_i to a single point:
 1. Orthogonality to $\mathbf{1}$: $\mathbf{Y}^T \mathbf{1} = 0$
 2. Normalization: $\mathbf{Y}^T \mathbf{Y} = 1$
- The properties from the previous slide allow us to perform this constrained minimization as an eigenvector problem
 - Min-max theorem: Since $\mathbf{L}$ is symmetric, the minimum of $\mathbf{Y}^T \mathbf{L} \mathbf{Y}$ under the constraint $\mathbf{Y}^T \mathbf{Y} = 1$ is given by the smallest eigenvalue of $\mathbf{L}$
 - Since $\mathbf{1}$ is eigenvector with eigenvalue 0, and $\mathbf{L}$ is positive semi-definite, we need to select $\mathbf{Y}$ as the *eigenvector* of $\mathbf{L}$ corresponding to the *second smallest eigenvalue*

Computing Laplacian Eigenmaps: General Case

- Laplacian Eigenmaps represent k -dimensional embedding as $\mathbf{Y} \in \mathcal{R}^{n \times k}$ and minimize $\text{tr}(\mathbf{Y}^T \mathbf{L} \mathbf{Y})$ subject to $\mathbf{Y}^T \mathbf{Y} = \mathbf{I}$
- $\mathbf{Y}$ contains k eigenvectors of $\mathbf{L}$ for smallest eigenvalues (except 0)



Summary: Manifold Learning

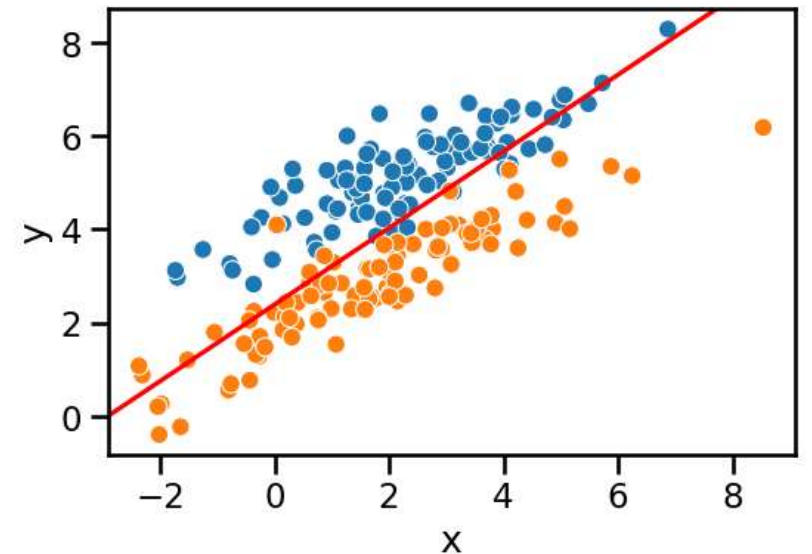
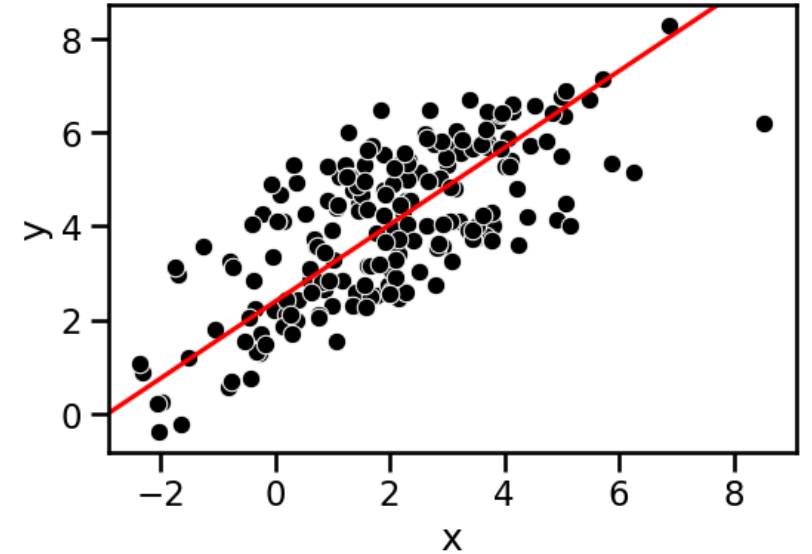
- PCA or MDS with Euclidean distances cannot obtain suitable results if data lies on a **nonlinear manifold** in high dimensional space
- **ISOMAP** combines MDS with estimated **geodesic distances** that are derived from a **neighborhood graph**
 - Introduces ability to “unfold” nonlinear structures
 - Aims to preserve all geodesic distances (local and global structure)
- **Laplacian Eigenmaps** compute an embedding from the eigenvectors of the (weighted) neighborhood graph
 - Focuses on preserving local structure (avoids separating neighbors)

Section 3.4:

Linear Discriminant Analysis

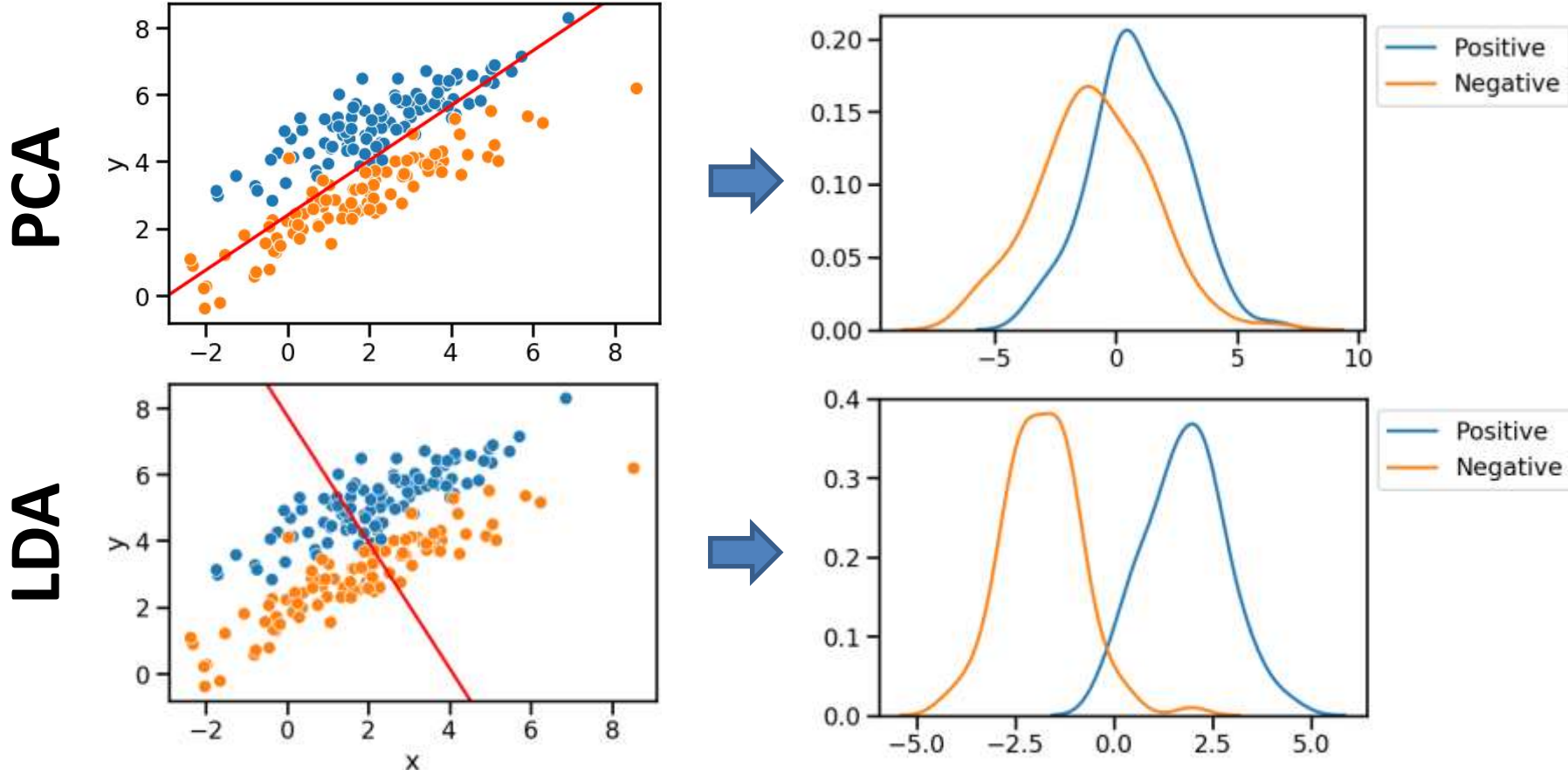
Motivation: Supervised Dim. Reduction

- The goal of preserving as much variance as possible in the data (PCA)...
- ...can be at odds with the alternative goal of preserving class separation
- **Question:** How can dimensionality reduction account for such **supervision**?



Goal of Supervised Dim. Reduction

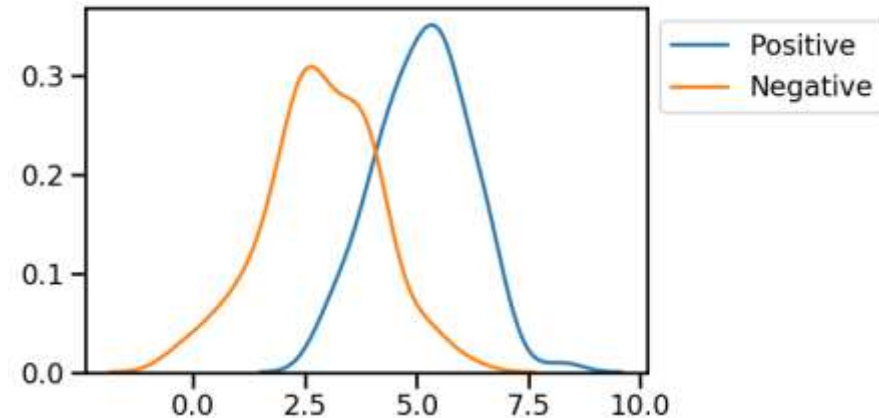
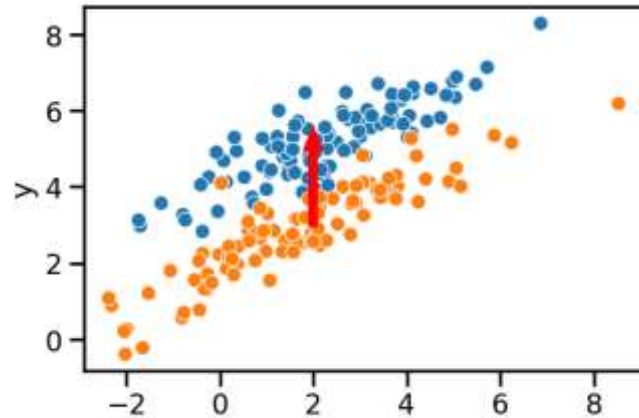
- **Supervised dimensionality reduction** aims to preserve the information that is relevant for separating given classes



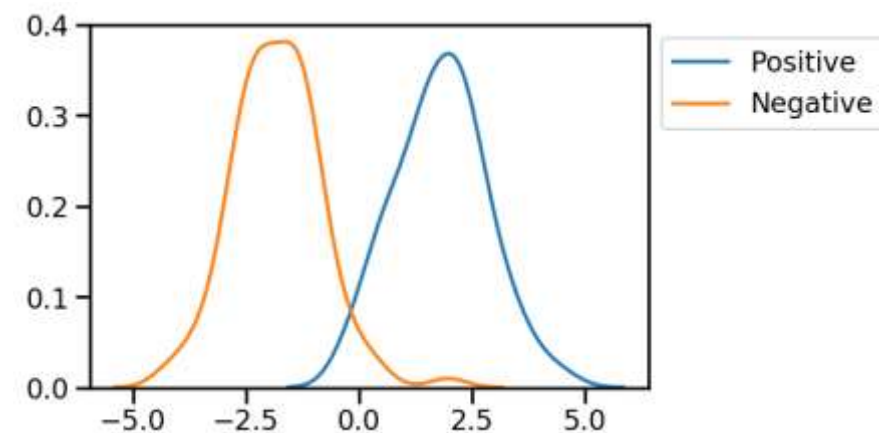
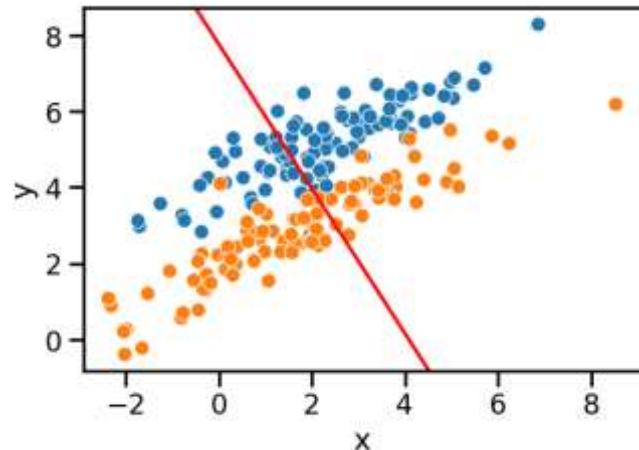
An Initial Idea

- Projecting onto the line between class centers maximizes their separation (distance), but might not minimize the overlap between the two distributions

Join Centers



LDA



Linear Discriminant Analysis (LDA)

- **Idea:** Given data $\mathbf{x}_i$ from two classes, **Fisher's discriminant** is the linear projection $y = \mathbf{w}^T \mathbf{x}$ with $\|\mathbf{w}\| = 1$ that yields the *largest class separation relative to the within-class variance*
 - We start with a 1D embedding derived from two classes, will generalize it later
- **Formalization:**
 - Denote mean of class c as $\bar{\mathbf{x}}_c = \frac{1}{|c|} \sum_{i \in c} \mathbf{x}_i$, its embedding as $\bar{y}_c = \mathbf{w}^T \bar{\mathbf{x}}_c$
 - Define (projection-space) **within-class variance** of class c as

$$s_c^2 = \sum_{i \in c} (y_i - \bar{y}_c)^2$$

- *Note:* In LDA, s_c^2 is called “variance” even though we do not divide by $|c| - 1$
- *Objective:* Select $\mathbf{w}$ with $\|\mathbf{w}\| = 1$ that maximizes

$$J(\mathbf{w}) = \frac{(\bar{y}_2 - \bar{y}_1)^2}{s_1^2 + s_2^2}$$

Two-Class LDA: Matrix Formulation

To compute $\mathbf{w}$, we reformulate $J(\mathbf{w}) = \frac{(\bar{y}_2 - \bar{y}_1)^2}{s_1^2 + s_2^2}$ with $y = \mathbf{w}^T \mathbf{x}$ based on a **between-class covariance** matrix

$$S_B = (\bar{\mathbf{x}}_2 - \bar{\mathbf{x}}_1)(\bar{\mathbf{x}}_2 - \bar{\mathbf{x}}_1)^T$$

class-specific **within-class covariance** matrices

$$S_c = \sum_{i \in c} (\mathbf{x}_i - \bar{\mathbf{x}}_c)(\mathbf{x}_i - \bar{\mathbf{x}}_c)^T$$

and a **total within-class covariance** $S_W = S_1 + S_2$ as

$$J(\mathbf{w}) = \frac{\mathbf{w}^T S_B \mathbf{w}}{\mathbf{w}^T S_W \mathbf{w}}$$

Solving Two-Class LDA

- **New goal:** Maximize $J(\mathbf{w}) = \frac{\mathbf{w}^T S_B \mathbf{w}}{\mathbf{w}^T S_W \mathbf{w}}$
- Necessary condition $\nabla_{\mathbf{w}} J(\mathbf{w}) = \mathbf{0}$ yields
$$(\mathbf{w}^T S_B \mathbf{w}) S_W \mathbf{w} = (\mathbf{w}^T S_W \mathbf{w}) S_B \mathbf{w}$$
- Observations:
 1. Since we will normalize $\|\mathbf{w}\| = 1$, we can disregard the scaling by $(\mathbf{w}^T S_B \mathbf{w})$ and $(\mathbf{w}^T S_W \mathbf{w})$
 2. By definition of S_B : $S_B \mathbf{w} \propto (\bar{\mathbf{x}}_2 - \bar{\mathbf{x}}_1)$
- **Conclusion:**

$$\mathbf{w} \propto S_W^{-1} (\bar{\mathbf{x}}_2 - \bar{\mathbf{x}}_1)$$

- *Note:* If S_W is isotropic (scaled identity), $\mathbf{w} \propto \bar{\mathbf{x}}_2 - \bar{\mathbf{x}}_1$
 - For this special case, we recover our initial idea of joining the class centers

LDA: Multi-Class Case

- We just defined a **one-dimensional embedding** that separates two classes
 - *Question:* Can we embed in higher dimension?
 - *Answer:* Yes, but it requires more classes.
- **Now:** Assume data $\mathbf{x}_i \in \mathbb{R}^p$ from K classes, look for $W \in \mathbb{R}^{p \times k}$ with $k < K$ so that the classes are well-separated in the embedding $\mathbf{y}_i = W^T \mathbf{x}_i$
 - *Same basic idea:* Maximize ratio of between-class over within-class variance
 - *But:* Between- and within-class variances are now described by variance-covariance matrices

Multi-Class LDA: Formalization

- Define (projection-space) between- and within-class covariance matrices as

$$S_B^\pi = \sum_{j=1}^K N_j (\bar{\mathbf{y}}_{c_j} - \bar{\mathbf{y}}) (\bar{\mathbf{y}}_{c_j} - \bar{\mathbf{y}})^T$$

$$S_W^\pi = \sum_{j=1}^K \sum_{i \in c_j} (\mathbf{y}_i - \bar{\mathbf{y}}_{c_j}) (\mathbf{y}_i - \bar{\mathbf{y}}_{c_j})^T$$

- N_j is the number of samples in class c_j ,
 $\bar{\mathbf{y}}_{c_j}$ is the mean of class c_j , $\bar{\mathbf{y}}$ is the grand (overall) mean
- $J(W)$ should be large when between-class covariance is large and within-class covariance is small. The popular choice
$$J(W) = \text{tr}\{(S_W^\pi)^{-1} S_B^\pi\}$$
generalizes our previous two-class definition

Multi-Class LDA: Dependency on W

Given the **between-class covariance** matrix

$$S_B = \sum_{j=1}^K N_j (\bar{\mathbf{x}}_j - \bar{\mathbf{x}})(\bar{\mathbf{x}}_j - \bar{\mathbf{x}})^T$$

and **total within-class covariance** matrix

$$S_W = \sum_{j=1}^K \sum_{i \in c_j} (\mathbf{x}_i - \bar{\mathbf{x}}_{c_j})(\mathbf{x}_i - \bar{\mathbf{x}}_{c_j})^T$$

in the original data space, $J(W) = \text{tr}\{(S_W^\pi)^{-1} S_B^\pi\}$ becomes

$$J(W) = \text{tr}\left\{(W^T S_W W)^{-1} (W^T S_B W)\right\}$$

Solving Multi-Class LDA

- **Goal:** Maximize

$$J(W) = \text{tr} \left\{ (W^T S_W W)^{-1} (W^T S_B W) \right\}$$

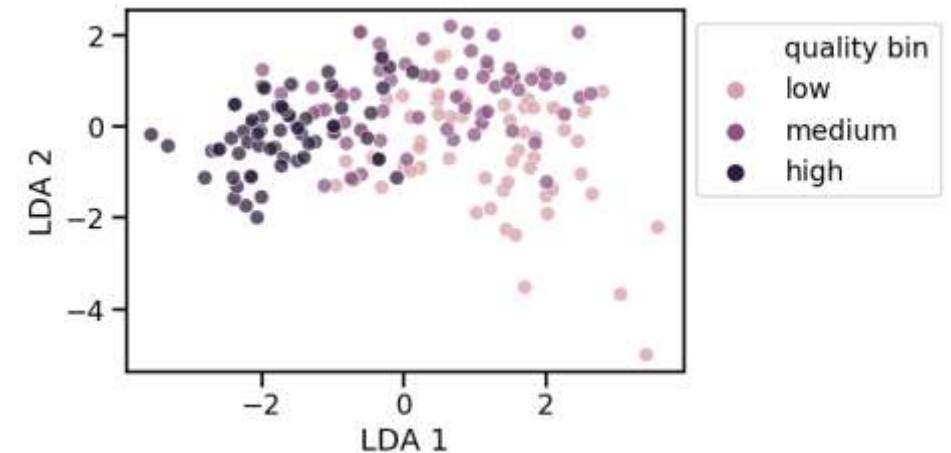
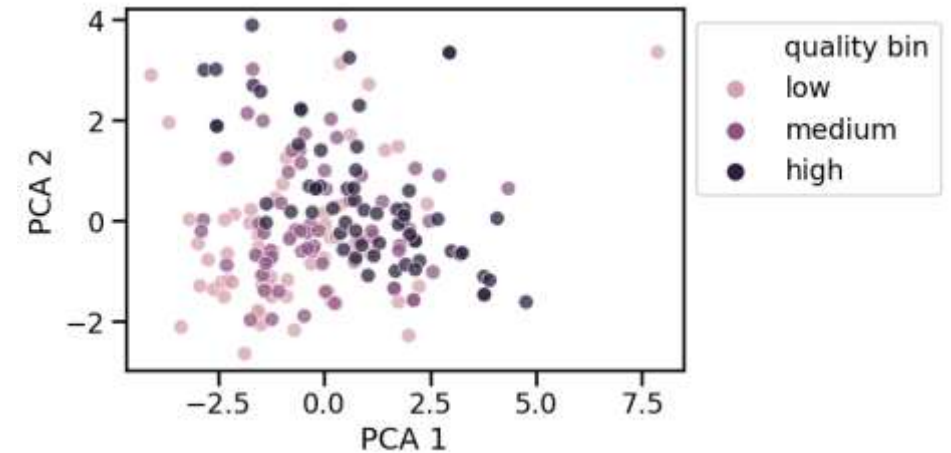
- For 1D case, this amounts to maximizing the generalized Rayleigh quotient

$$J(\mathbf{w}) = \frac{\mathbf{w}^T S_B \mathbf{w}}{\mathbf{w}^T S_W \mathbf{w}}$$

- Solution given by eigenvector corresponding to largest eigenvalue of $S_W^{-1} S_B$
- Subsequent columns in W should maximize the generalized Rayleigh quotient, while maintaining orthogonality to all previous ones
 - *Conclusion:* W given by eigenvectors for k largest eigenvalues

Example Result: LDA vs. PCA

- **Example result:**
 - Wine quality vs. lab measurements
 - **PCA projection** (after standardization) causes substantial overlap between quality levels
 - **LDA projection** provides improved class separation



Important Restriction of LDA

- **Note:** Given samples from K classes, LDA cannot be used to compute an embedding with $k > K - 1$ (meaningful) directions

- *Reason:* Between-class covariance matrix

$$S_B = \sum_{j=1}^K N_j (\bar{\mathbf{x}}_j - \bar{\mathbf{x}})(\bar{\mathbf{x}}_j - \bar{\mathbf{x}})^T$$

has rank no greater than $K - 1$

- All meaningful solutions of

$$J(W) = \text{tr} \left\{ (W^T S_W W)^{-1} (W^T S_B W) \right\}$$

should fall into the range of S_B

Summary: Linear Discriminant Analysis

- Linear Discriminant Analysis (LDA) is a standard approach for **supervised dimensionality reduction**
 - **Restriction:** Resulting dimension always lower than the number of classes
- **Remarks:**
 - LDA can also be used for supervised classification
 - If all classes have the same spread and equal prior probability, simply classify based on closest $\bar{\mathbf{y}}_{c_j}$
 - Like PCA, LDA can also be kernelized if nonlinear projections are required to separate the classes (but this is outside our scope)

Section 3.5:

t-SNE

Introduction

- The techniques we saw so far all attempt to preserve aspects that relate to (local and global) **distances**
 - *(kernel) PCA*: minimize sum of squared distances (in feature space)
 - *MDS*: minimize absolute or relative deviations from given distances
 - *ISOMAP*: estimated geodesic distances
 - *LDA*: between- vs. within-class distances
- If we are primarily interested in high-dimensional clusters, it can be beneficial to focus on preserving **(local) nearest neighbors**
 - **t-SNE** [van der Maaten/Hinton 2008]
 - abstracts away density and distance information
 - often reveals cluster structure more clearly than any other dimensionality reduction technique

Motivating Example: Dataset

- **Toy dataset:** 8x8 pixel representations of handwritten digits 0-5

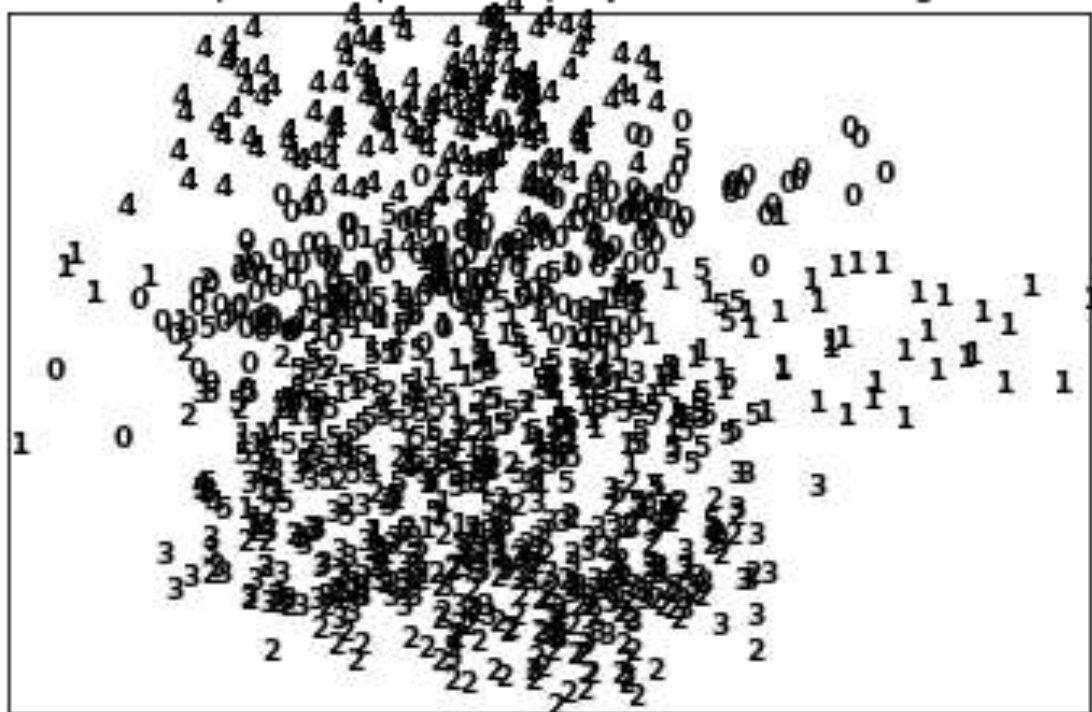
A selection from the 64-dimensional digits dataset

0	1	2	3	4	5	0	1	2	3	4	5	0	1	2	3	4	5	0	5
5	5	0	4	1	3	5	1	0	0	2	2	2	0	1	2	3	3	3	3
4	4	1	5	0	5	2	2	0	0	1	3	2	1	4	3	1	3	1	4
3	1	4	0	5	3	1	5	4	4	2	2	2	5	5	4	4	0	0	1
2	3	4	5	0	1	2	3	4	5	0	1	2	3	4	5	0	5	5	5
0	4	1	3	5	1	0	0	2	2	2	0	1	2	3	3	3	3	4	4
4	5	0	5	2	1	0	0	1	3	2	4	3	1	3	4	4	3	4	4
0	5	3	4	5	4	4	1	2	2	5	5	4	4	0	0	1	2	3	4
5	0	1	2	3	4	5	0	4	2	3	4	5	0	5	5	5	0	4	1
3	5	1	0	0	2	2	2	0	4	2	3	3	3	3	4	4	1	5	0
5	2	2	0	0	1	3	2	4	4	3	1	3	1	4	3	1	4	0	5
3	1	5	4	4	2	2	2	5	5	4	4	0	3	0	1	2	3	4	5
0	1	2	3	4	5	0	1	2	3	4	5	0	5	5	5	0	4	1	3
5	1	0	0	2	2	2	0	1	2	3	3	3	3	4	4	1	5	0	5
2	2	0	0	1	3	2	4	4	3	1	3	1	4	3	1	4	0	5	3
1	5	4	4	2	2	2	5	5	4	4	0	0	1	2	3	4	5	0	1
2	3	4	5	0	1	2	3	4	5	0	5	5	5	0	4	1	3	5	4
0	0	1	2	2	0	1	1	3	3	3	3	4	4	4	5	0	5	1	2
0	0	1	3	2	1	4	3	1	3	1	4	3	1	4	0	5	3	1	5
4	4	2	2	1	5	5	4	4	0	0	1	2	3	4	5	0	1	2	3

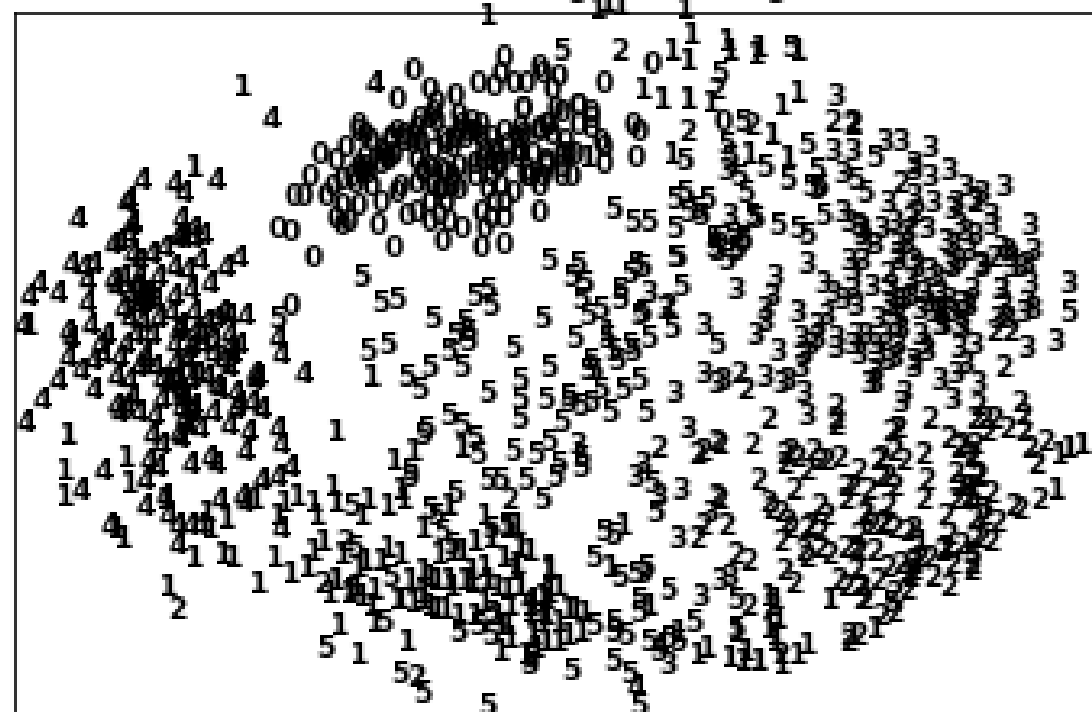
Motivating Example: PCA and MDS

- Embedding with PCA and MDS

Principal Components projection of the digits



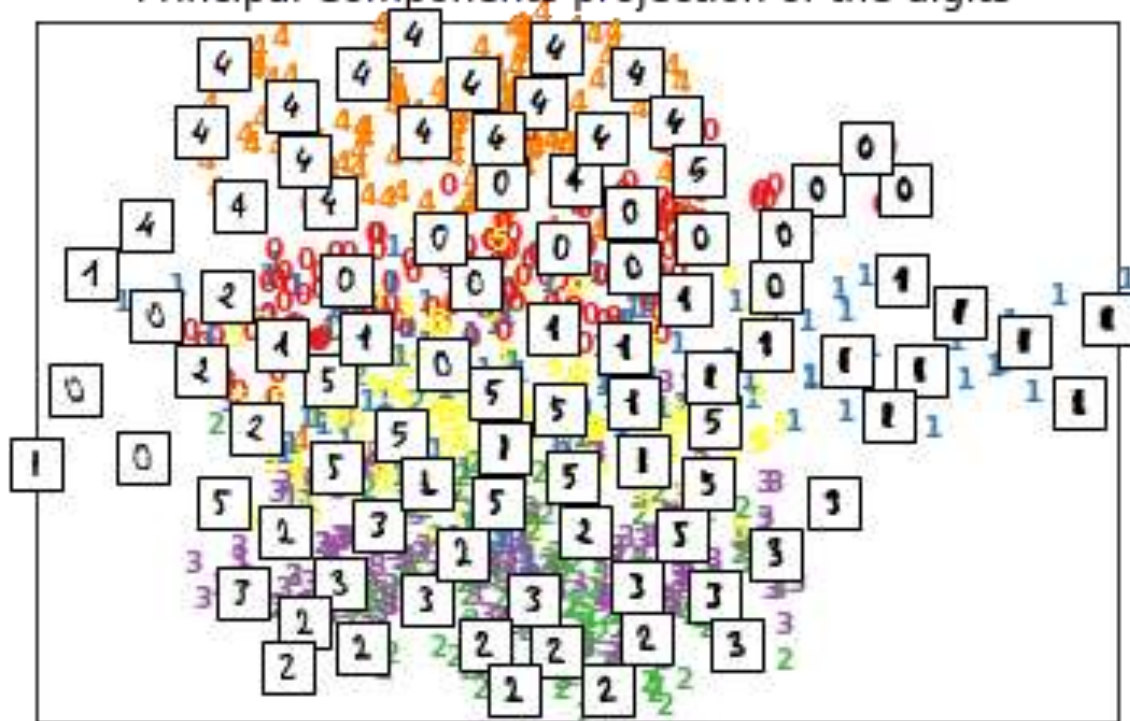
MDS embedding of the digits



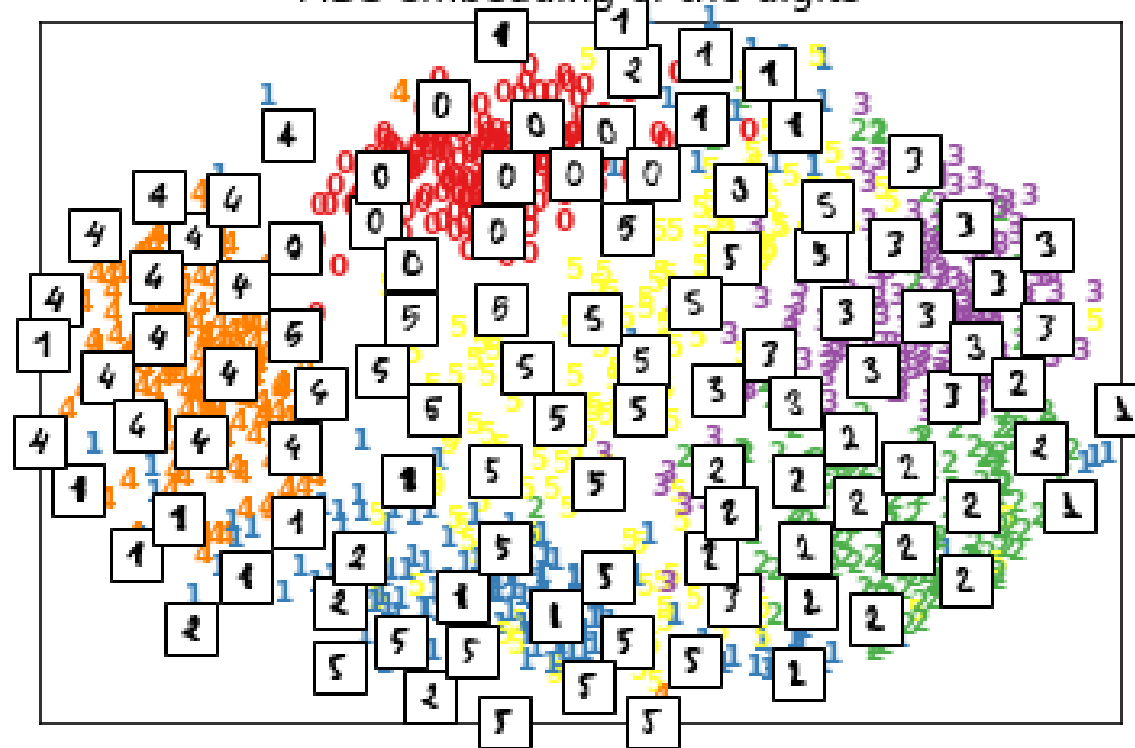
Motivating Example: PCA and MDS

- Embedding with PCA and MDS
 - Colored by class labels, with zoomed examples

Principal Components projection of the digits



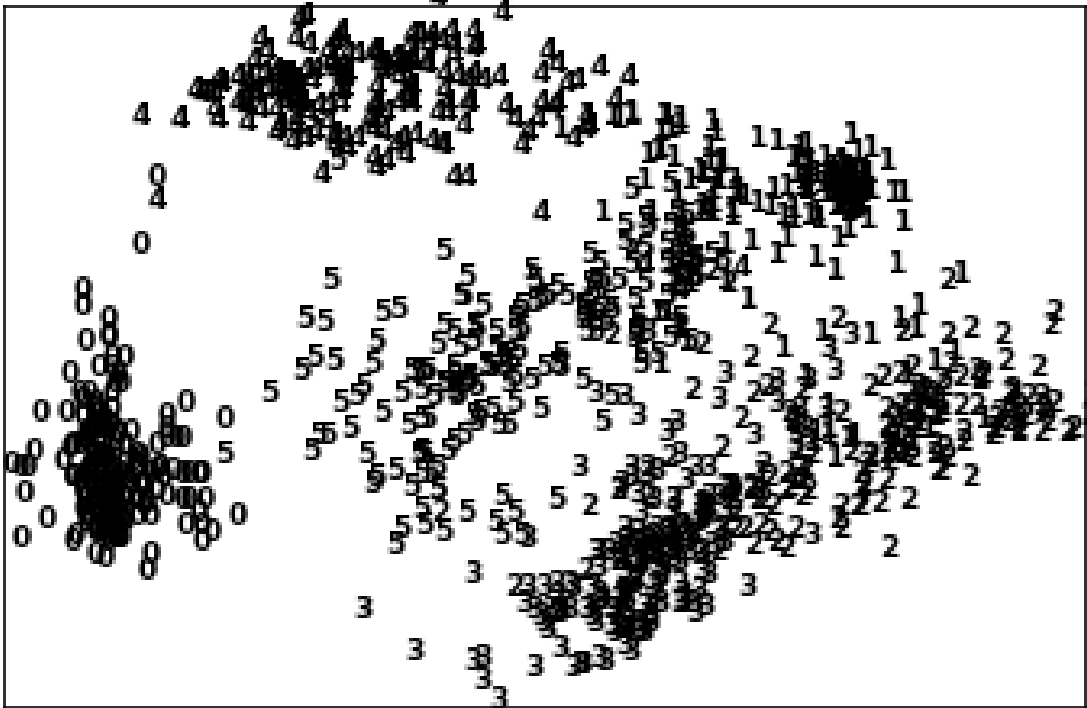
MDS embedding of the digits



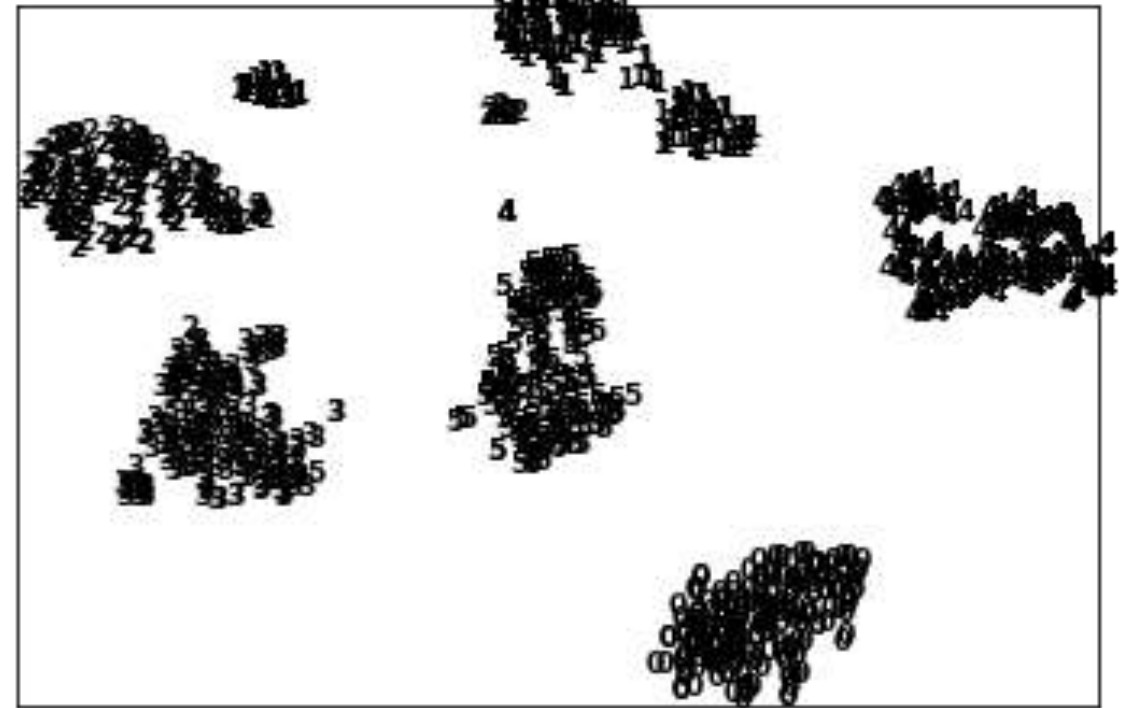
Motivating Example: ISOMAP and t-SNE

- Embedding with ISOMAP and t-SNE

Isomap projection of the digits

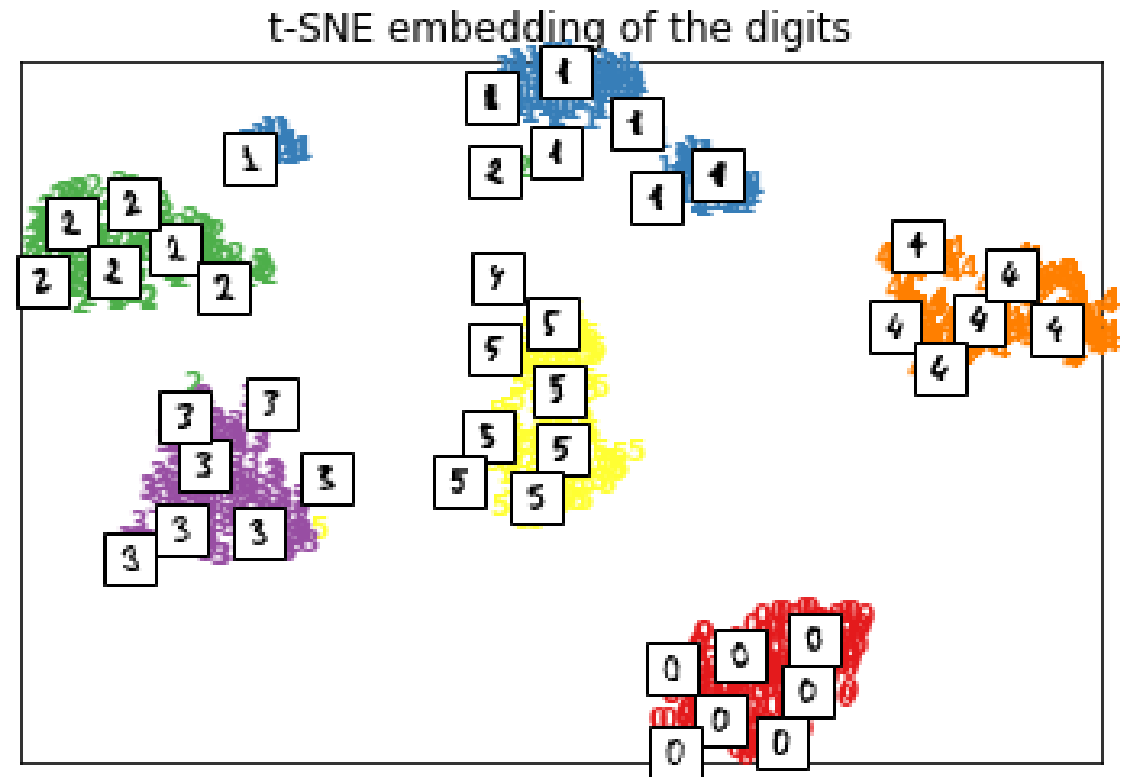
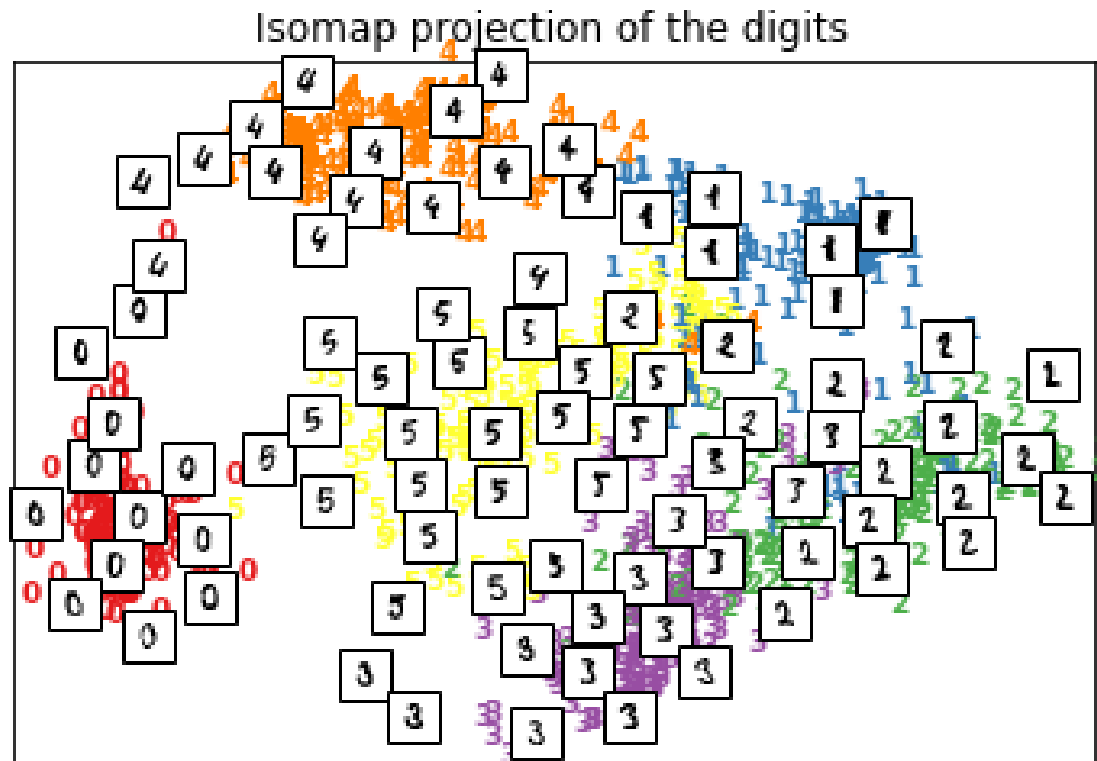


t-SNE embedding of the digits



Motivating Example: ISOMAP and t-SNE

- Embedding with ISOMAP and t-SNE
 - Colored by class labels, with zoomed examples



Basic Idea: SNE

- **Stochastic Neighborhood Embedding (SNE)** defines distance-based conditional probabilities that, given a local neighborhood scale σ_i , $\mathbf{x}_i$ would pick $\mathbf{x}_j$ as its neighbor

$$p_{j|i} = \frac{\exp(-\|\mathbf{x}_i - \mathbf{x}_j\|^2 / 2\sigma_i^2)}{\sum_{k \neq i} \exp(-\|\mathbf{x}_i - \mathbf{x}_k\|^2 / 2\sigma_i^2)}$$

- It then places lower-dimensional points $\mathbf{y}_i$ in such a way that similar probabilities are obtained

$$q_{j|i} = \frac{\exp(-\|\mathbf{y}_i - \mathbf{y}_j\|^2)}{\sum_{k \neq i} \exp(-\|\mathbf{y}_i - \mathbf{y}_k\|^2)}$$

KL Divergence

- Let P_i and Q_i denote the conditional probabilities given $\mathbf{x}_i$ and $\mathbf{y}_i$, respectively
- SNE aims to make them as similar as possible by minimizing the sum of KL divergences from Q_i to P_i

$$C = \sum_i \text{KL}(P_i || Q_i) = \sum_i \sum_j p_{j|i} \log \frac{p_{j|i}}{q_{j|i}}$$

- Observe asymmetry in cost function C :
 - Expensive to model nearby $\mathbf{x}$ with distant $\mathbf{y}$
 - Moderate cost to model distant $\mathbf{x}$ with nearby $\mathbf{y}$
- *Consequence*: SNE keeps neighbors close together, widely separated points “don’t care so much”

Bandwidth Selection and Perplexity

- Using a fixed σ does not work well when densities in high-dimensional space vary

- Same problem as with ϵ -graphs in ISOMAP

- Instead, user defines the desired **perplexity**

$$\text{Perp}(P_i) = 2^{H(P_i)}$$

$$\text{with } H(P_i) = -\sum_j p_{j|i} \log_2 p_{j|i}$$

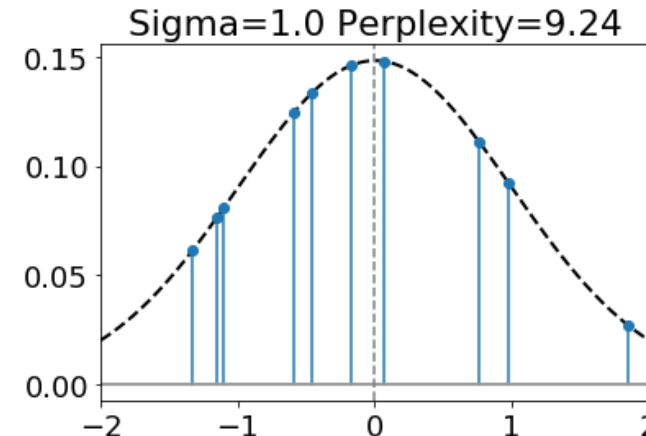
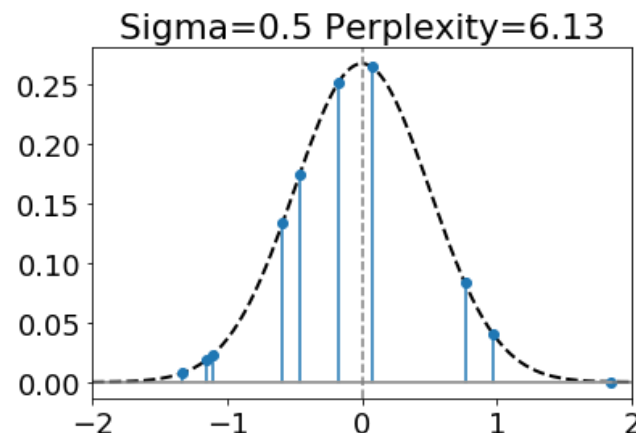
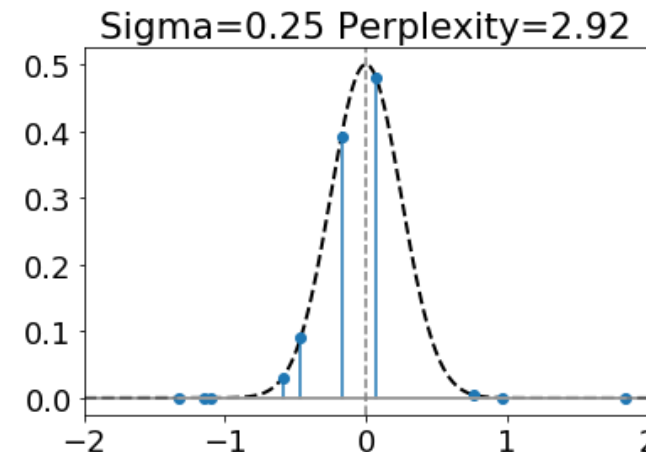
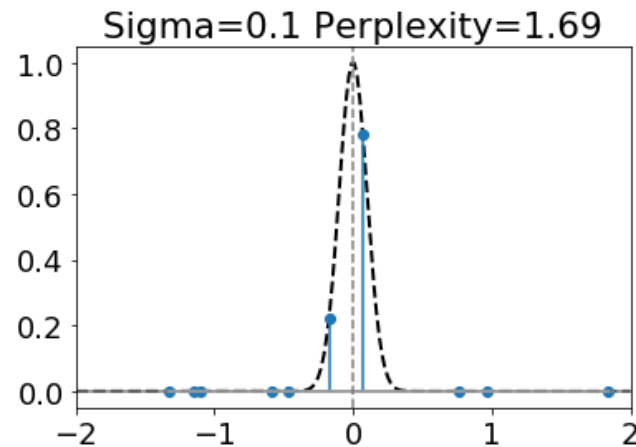
- Reflects the “effective number of neighbors”

- Uniform distribution over n points has perplexity n

- Key parameter in t-SNE. Should be large enough to capture relevant structures, but always (much) lower than the number of data points!

Finding σ Based on Perplexity

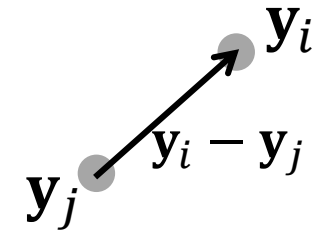
- For each $\mathbf{x}_i$, t-SNE automatically chooses σ_i to achieve user-specified perplexity
 - Perplexity increases monotonically with σ



Optimization

- Basic idea behind optimization is similar as in MDS:
 - Initialize $\mathbf{y}_i$ randomly
 - Iteratively shift $\mathbf{y}_i$ by some constant (“learning rate”) times the negative gradient of the KL divergence:

$$\frac{\partial \mathcal{C}}{\partial \mathbf{y}_i} = 2 \sum_j (p_{j|i} - q_{j|i} + p_{i|j} - q_{i|j})(\mathbf{y}_i - \mathbf{y}_j)$$



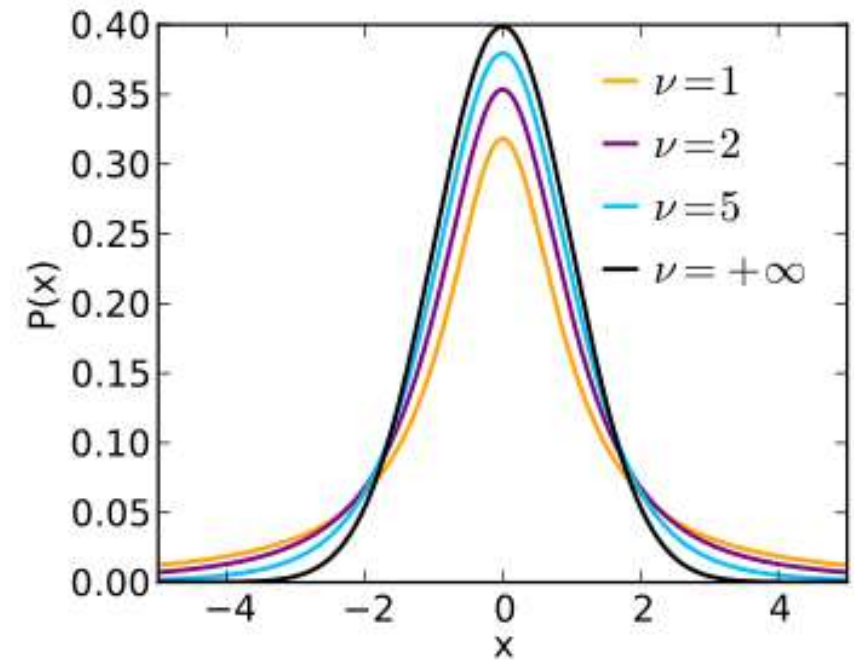
- In practice, additional tricks are used, such as
 - Decreasing learning rate with time
 - Randomly perturbing points to avoid local suboptima
 - Including a momentum term that continues to drive points in a similar direction as was taken previously
 - “Early compression” and “early exaggeration”

The Crowding Problem

- The “swiss roll” from the ISOMAP example can be embedded very well in 2D because it is intrinsically two-dimensional
- If intrinsic dimensionality is greater, 2D embeddings become *crowded*
 - Volume of m -dimensional sphere $\propto r^m$
 - If $m \gg 2$, 2D area around a point not nearly large enough to accommodate all points from a high-dimensional neighborhood
 - As a result, many points get collapsed together, making the resulting plot hard to read

Basic Idea of t-SNE

- t-SNE addresses the crowding problem by using **different probability distributions** in low and high dimension
 - “Heavy tails” allow points to move further apart in low dimensions compared to high dimensions
 - Student’s t-distribution is a well-known distribution with heavy tails
 - t-SNE uses t-distribution with $\nu = 1$ (Cauchy distribution)



Symmetric Cost Function

- t-SNE speeds up computation by defining the cost in terms of the (symmetric) probabilities $p_{ij} = p_{ji}$ of $\mathbf{x}_i$ and $\mathbf{x}_j$ being neighbors

$$C = \text{KL}(P \parallel Q) = \sum_i \sum_j p_{ij} \log \frac{p_{ij}}{q_{ij}}$$

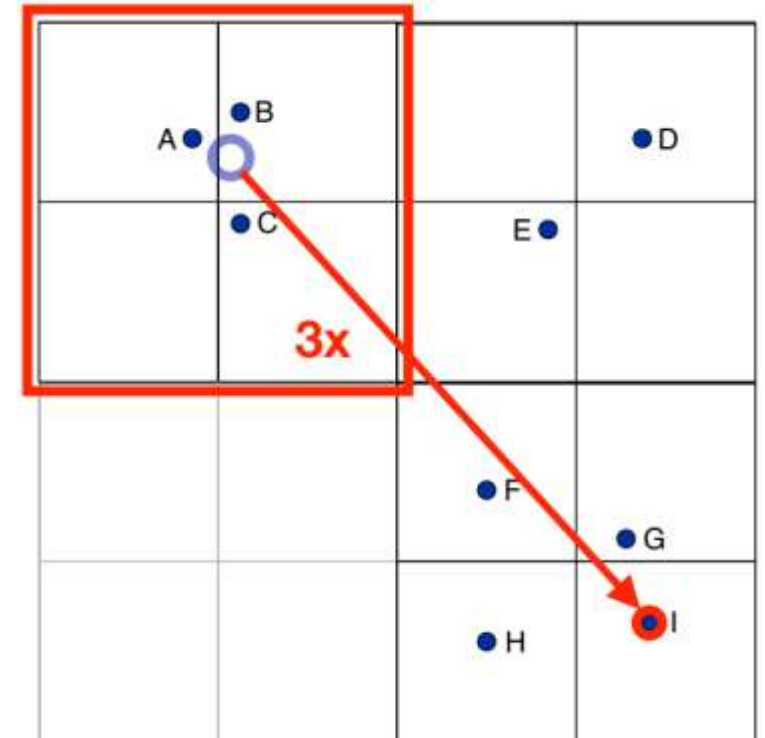
$$p_{ij} = \frac{p_{j|i} + p_{i|j}}{2n} \quad q_{ij} = \frac{\left(1 + \|\mathbf{y}_i - \mathbf{y}_j\|^2\right)^{-1}}{\sum_{k \neq l} \left(1 + \|\mathbf{y}_k - \mathbf{y}_l\|^2\right)^{-1}}$$

- p_{ij} can be pre-computed from Gaussians
- new q_{ij} faster to compute (no exponential)
- This leads to the new gradient

$$\frac{\partial C}{\partial \mathbf{y}_i} = 4 \sum_j (p_{ij} - q_{ij})(\mathbf{y}_i - \mathbf{y}_j) \left(1 + \|\mathbf{y}_i - \mathbf{y}_j\|^2\right)^{-1}$$

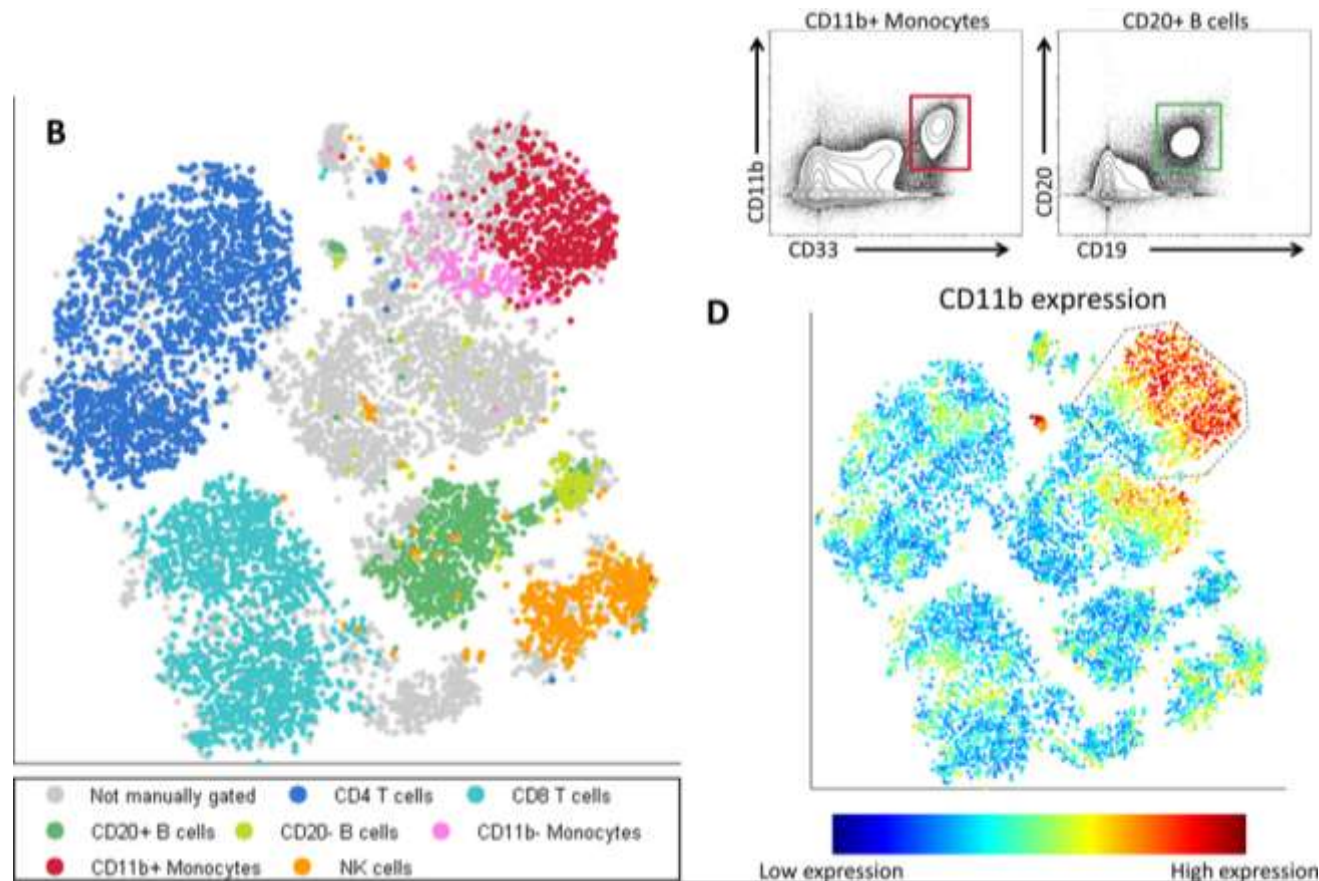
Barnes-Hut Approximation

- Since all pairs of points interact, evaluating the objective function and its gradient for n input points requires $\Theta(n^2)$ effort
- [van der Maaten 2014] introduces two approximations that reduce this to $O(n \log n)$:
 - Only compute $p_{i||j}$ for a fixed number of nearest neighbors (three times perplexity)
 - Approximate effect of remote clusters of points with a single value



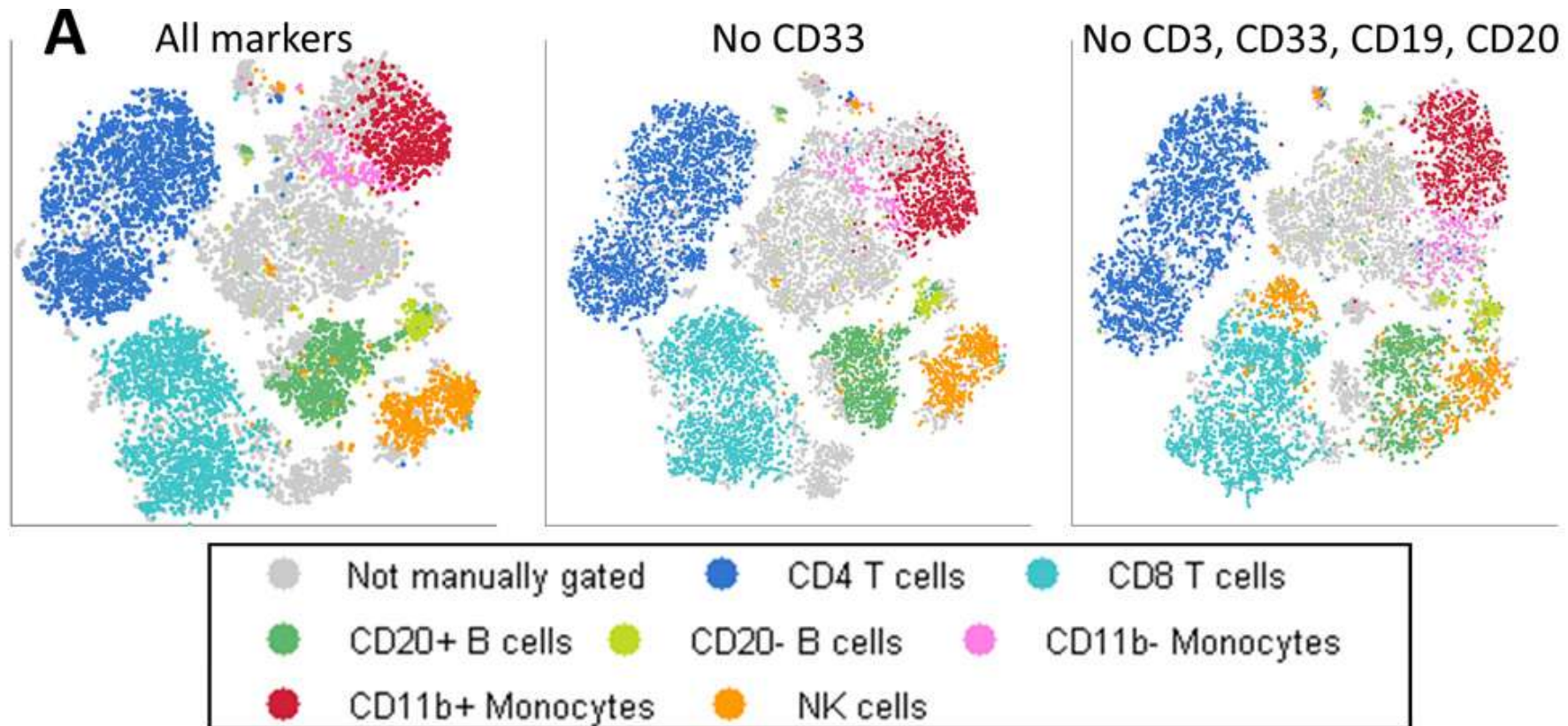
t-SNE in Biological Data Analysis

- [Amir et al., Nature Biotechnology 2013] use t-SNE to embed cells based on high-dimensional protein expression profiles



t-SNE in Biological Data Analysis

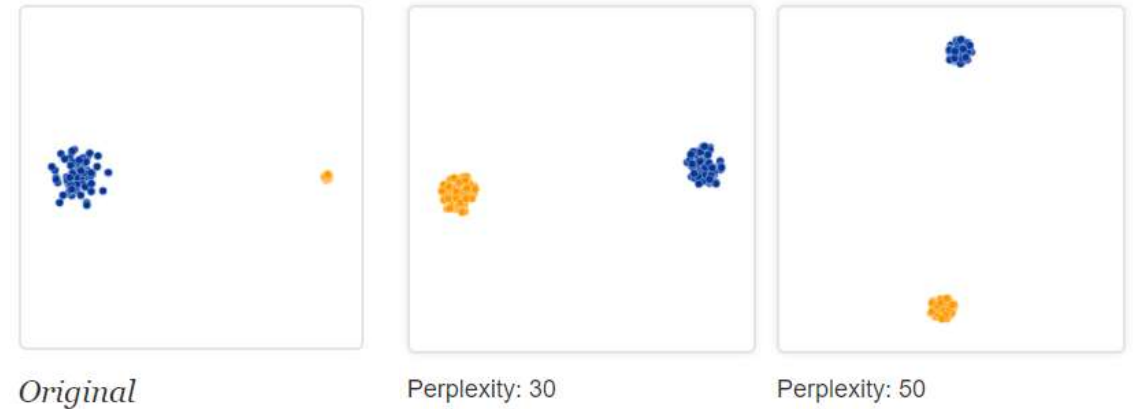
- *Remarkable finding:* Similar clustering still obtained after removing the known markers



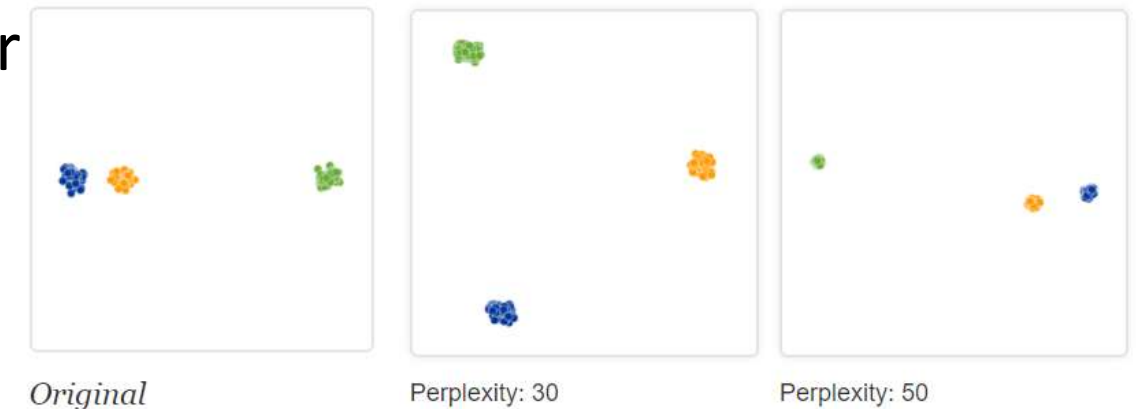
Caveats: What t-SNE Hides

- By design, t-SNE **does not preserve**

- **densities/cluster spreads:**
fixing perplexity intentionally
“spreads out” denser regions
to show them in more detail

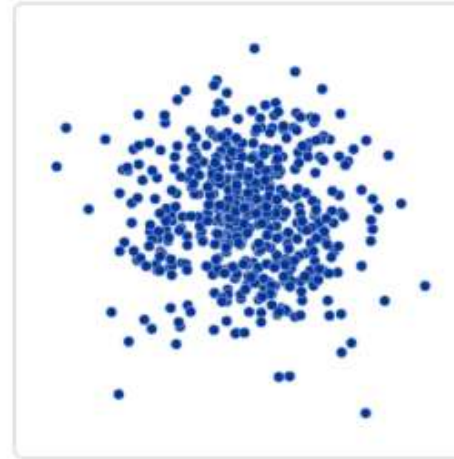


- **distances between clusters:** may get compressed by KL divergence (effect: makes better use of available screen space)

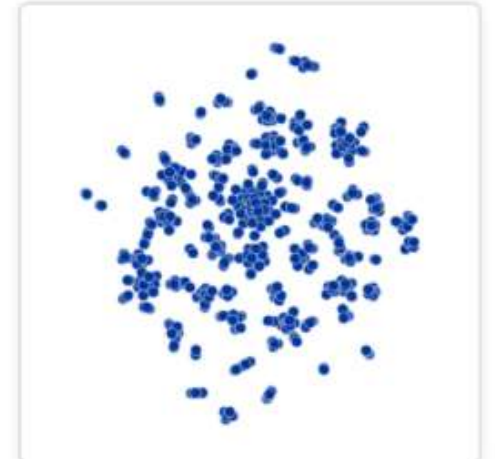


Caveats: What t-SNE May Hallucinate

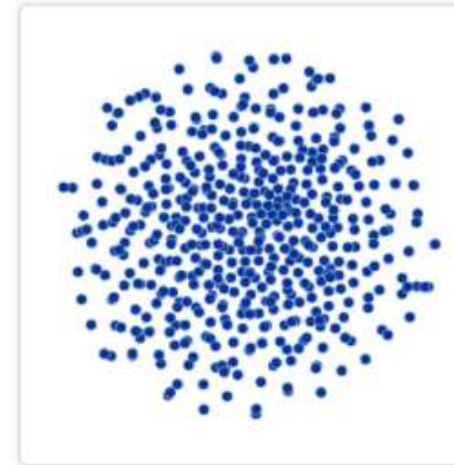
- Cluster-forming forces might **amplify small-scale random variations** in the data
 - *Example*: 500 samples from 100-dimensional Gaussian



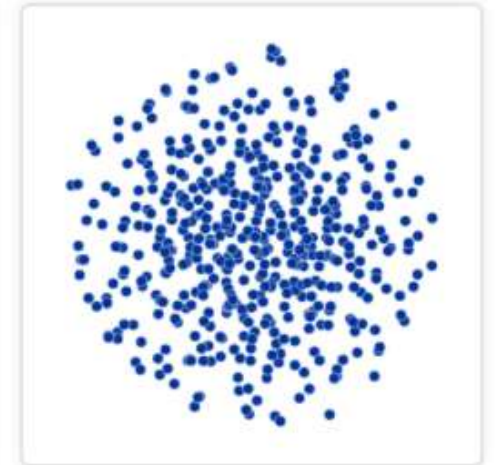
Original



Perplexity: 2



Perplexity: 30
Step: 5,000



Perplexity: 50
Step: 5,000

Summary: t-SNE

- **t-SNE** is a method for nonlinear dimensionality reduction that focuses on preserving nearest neighbors
 - particularly good at **separating clusters**
 - to an extent that it might suggest clusters where none exist
 - specifically designed for **visualization**
 - questionable for feature extraction, formal clustering, etc.
 - distorts distances, densities, relative sizes
- Results of t-SNE depend on **many parameters**
 - Try different values of perplexity and initializations, make sure optimization has stabilized
 - “Invisible” parameters from the optimization (learning rate, momentum, etc.) can also have an effect
 - Beware of application papers that do not account for this

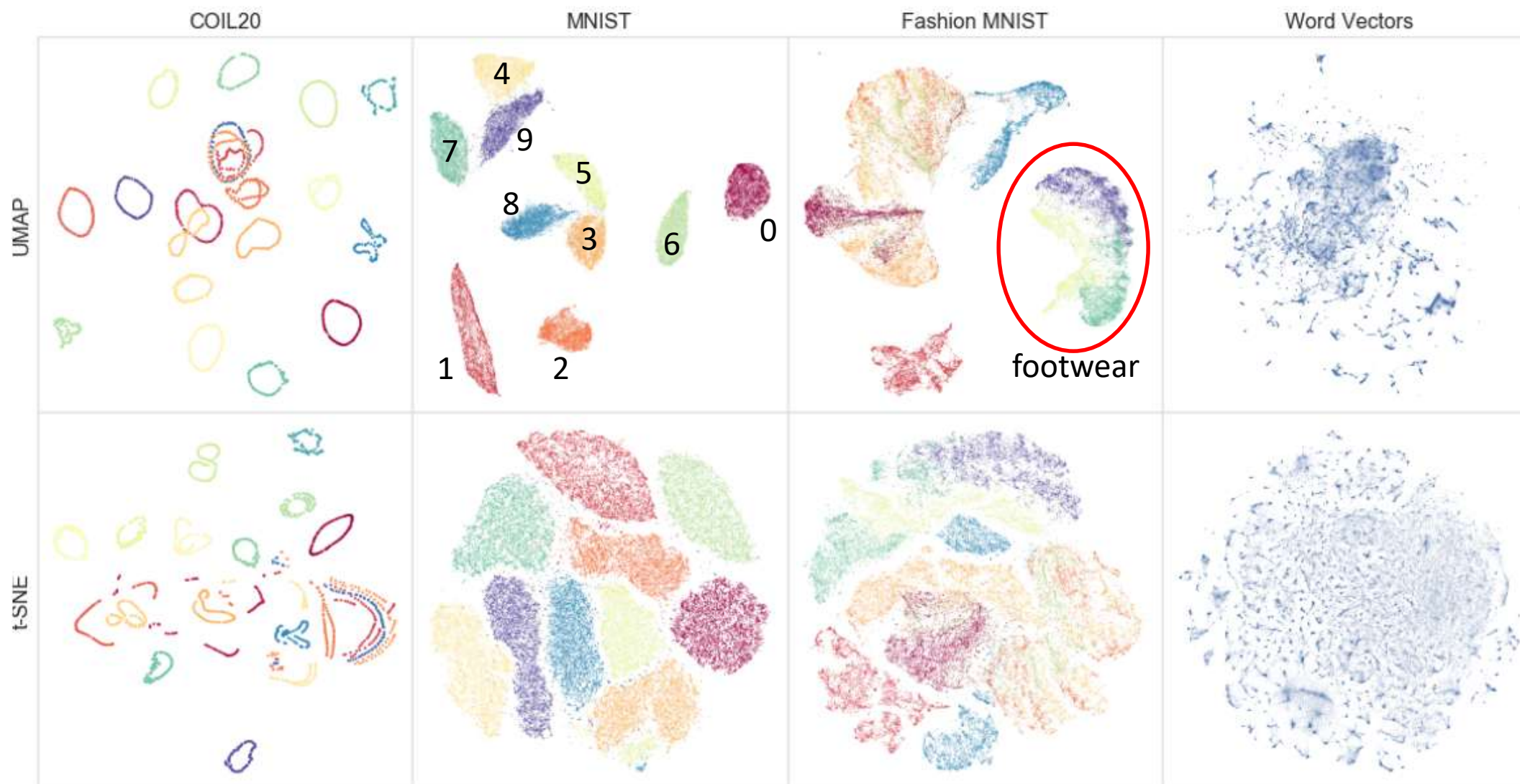
Section 3.6:

UMAP

Introduction to UMAP

- **UMAP** (Uniform Manifold Approximation and Embedding) [McInnes et al. 2018] is a more recent alternative to t-SNE
 - Derivation aims to preserve fuzzy topological representations, formulated in the language of algebraic topology and category theory
 - Algorithm is quite similar to t-SNE, differences are mostly details
 - Efficient computation, due to random sampling
 - UMAP is often said to preserve global in addition to local structure, but that is mostly due to initialization, can also be done with t-SNE
- Powerful **software package** has made UMAP popular
 - many additional features, including (semi-)supervised dimensionality reduction and out-of-sample projection

Examples: t-SNE vs UMAP



Modeling Neighbors in High Dimension

- Based on the set $\mathcal{N}_i$ of the k nearest neighbors of point $\mathbf{x}_i$, UMAP assigns similarity scores

$$p_{j|i} = \begin{cases} \exp\left(-\frac{\|\mathbf{x}_i - \mathbf{x}_j\| - \rho_i}{\sigma_i}\right) & \text{if } x_j \in \mathcal{N}_i \\ 0 & \text{else} \end{cases}$$

- where ρ_i is the distance of $\mathbf{x}_i$ to its nearest neighbor
- where σ_i is adapted so that $\sum_j p_{j|i} = \log_2 k$
- Empirically, k can be chosen to be lower than perplexity in t-SNE; binary weights often give similar results in practice [Böhm et al. 2020]
- Similarities are symmetrized via a fuzzy union operation:

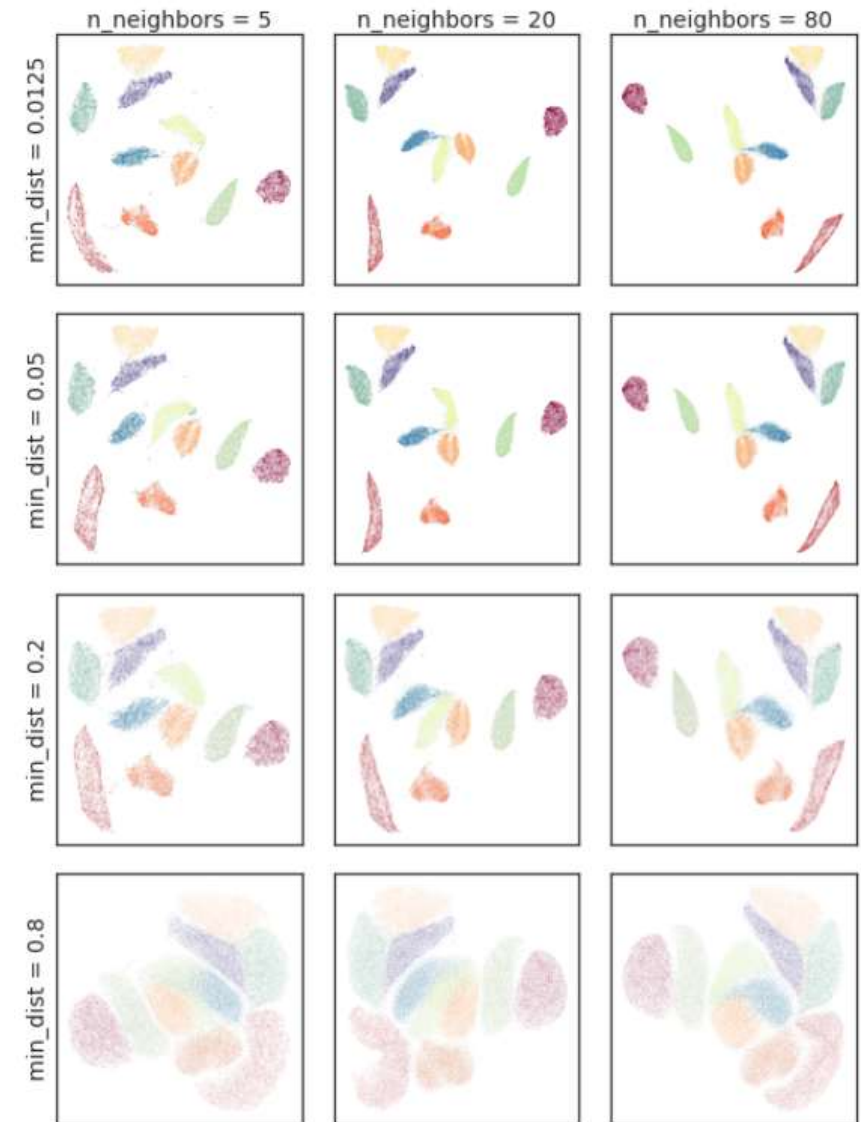
$$p_{ij} = p_{j|i} + p_{i|j} - p_{j|i} p_{i|j}$$

Modeling Neighbors in the Embedding

- UMAP quantifies the similarity of points i and j in the embedding as

$$q_{ij} = \left(1 + a \|\mathbf{y}_i - \mathbf{y}_j\|_2^{2b}\right)^{-1}$$

- Values a, b affect how closely points will be packed in the embedding
- In UMAP, a, b are set indirectly via a new hyperparameter: min-dist
 - UMAP default 0.1 leads to $a \approx 1.929, b \approx 0.7915$
 - Setting $a = 1, b = 1$ corresponds to Cauchy distribution used in t-SNE
 - [McInnes et al. 2018]: “essentially aesthetic parameter”



Cost Function in UMAP

- Without normalizing p_{ij} and q_{ij} to sum to one, UMAP aims to match them by minimizing their cross-entropy

$$C = \sum_i \sum_{j \neq i} \left(\underbrace{p_{ij} \ln \frac{p_{ij}}{q_{ij}}}_{\text{attraction:}} + \underbrace{(1 - p_{ij}) \ln \left(\frac{1 - p_{ij}}{1 - q_{ij}} \right)}_{\text{repulsion:}} \right)$$

keeps nearby points together (like t-SNE) pushes distant points apart

- Since the p_{ij} do not depend on the embedding, UMAP equivalently minimizes

$$\tilde{C} = - \sum_i \sum_{j \neq i} (p_{ij} \ln q_{ij} + (1 - p_{ij}) \ln(1 - q_{ij}))$$

Efficient Optimization in UMAP

- UMAP uses Laplacian Eigenmaps for initialization
- UMAP performs a **stochastic gradient descent** that involves subsampling to keep computational effort low:
 - In each epoch, each point i is updated according to its **attraction to point j** with probability p_{ij}
 - *Negative sampling*: In each update, the **repelling term** is included for m randomly sampled points
 - *Note*: This effectively makes the repelling term much weaker than $\tilde{C}$ suggests

$$\tilde{C} = - \sum_i \sum_{j \neq i} (p_{ij} \ln q_{ij} + (1 - p_{ij}) \ln(1 - q_{ij}))$$

Role of Initialization

- [Kobak/Linderman 2021]: Preservation of global structure in t-SNE and UMAP primarily depends on initialization
 - Recommend use of informative initialization also for t-SNE, but also state: „t-SNE with non-random initialization should not be considered a new algorithm or even an extension of the original t-SNE“

Table 1 | Performance of t-SNE and UMAP with random and informative initialization using datasets and evaluation metrics from Becht et al.

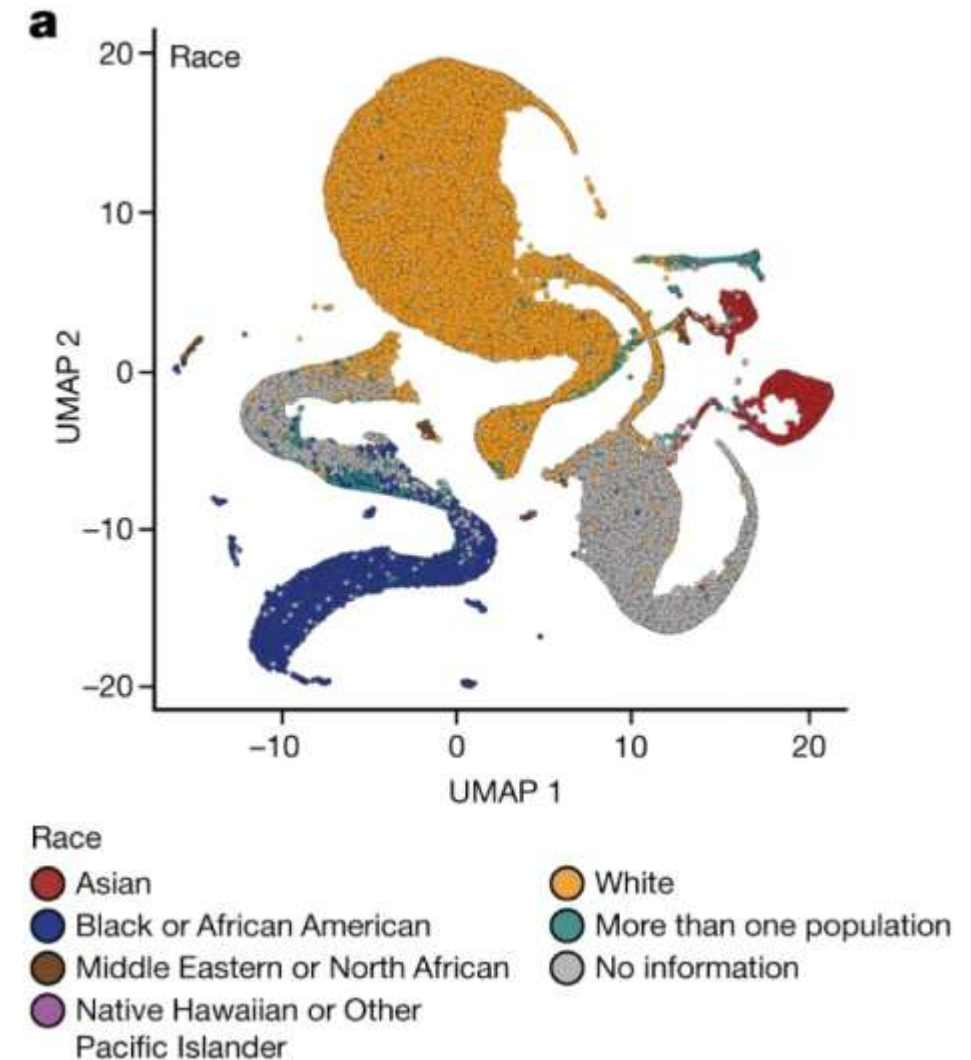
Dataset	Preservation of pairwise distances			Reproducibility of large-scale structures ^a		
	Samusik et al. ⁸	Wong et al. ⁹	Han et al. ¹⁰	Samusik et al. ⁸	Wong et al. ⁹	Han et al. ¹⁰
UMAP, LE initialization	0.70	0.57	0.30	0.94	0.98	0.49
UMAP, random initialization	0.41	0.38	0.14	0.24	0.21	0.22
t-SNE, PCA initialization	0.59	0.66	0.28	0.95	0.98	0.92
t-SNE, random initialization	0.32	0.36	0.18	0.29	0.33	0.06

^aFor the reproducibility metric, the average over three random subsamples of size $n=200,000$ is reported. Bold numbers denote the maximum value in each column.



A Recent UMAP Controversy

- In a 2024 Nature paper, the **All of Us Research Program Genomics Investigators** used the following UMAP figure of whole-genome sequence information to illustrate the diversity in their dataset
 - Do you see any issues with this?



Summary: UMAP

- **UMAP** is a widely used alternative to t-SNE
 - follows the same main goal of **preserving nearest neighbors**, but expresses this with more sophisticated mathematical concepts
 - only considering the k nearest neighbors and negative sampling during optimization increase **computational efficiency**
 - Modern implementations of t-SNE fully competitive when embedding to 2D
 - is often said to **better preserve global structure**, but this is due to a more informative initialization, which should also be used for t-SNE
- **Main parameters** are number of neighbors and min distance
 - Default parameters lead to more compact clusters / more whitespace

Summary: Dimensionality Reduction

- Dimensionality reduction projects high-dimensional objects to low-dimensional space
 - **PCA:** Takes Euclidean input and finds linear projection that preserves maximum variance / minimizes sum of squared distances
 - **Kernel PCA:** Better preserves nonlinear structures
 - **MDS:** Can be applied even if data is non geometric or dissimilarities are non-Euclidean
 - **ISOMAP / Laplacian Eigenmaps:** Aim to preserve nonlinear manifold structure that is represented in a neighborhood graph
 - **LDA:** Preserves separability of given class labels
 - **t-SNE / UMAP:** Embeddings that focus on preserving nearest neighbors and equalize densities

References (Methods)

- Ghojogh, Crowley, Karray, Ghodsi: **Elements of Dimensionality Reduction and Manifold Learning**. Springer, 2023
- Hastie, Tibshirani, Friedman: **The Elements of Statistical Learning**. 2nd edition, Springer, 2011
- Bishop: **Pattern Recognition and Machine Learning**. Springer, 2006
- Tenenbaum, de Silva, Langford: **A Global Geometric Framework for Nonlinear Dimensionality Reduction**, Science 290: 2319–2323, 2000
- van der Maaten, Hinton: **Visualizing Data using t-SNE**. Journal of Machine Learning Research 9:2579-2605, 2008
- McInnes, Healy, Melville: **UMAP: Uniform Manifold Approximation and Projection for Dimension Reduction**. arXiv 2018

References (Applications)

- Jianu, Demiralp, Laidlaw: **Exploring 3D DTI Fiber Tracts with Linked 2D Representations**. IEEE TVCG 15(6):1449-1456, 2009
- Mohammad Khatami, Tobias Schmidt-Wilcke, Pia C. Sundgren, Amin Abbasloo, Bernhard Schölkopf, Thomas Schultz: **BundleMAP: Anatomically Localized Classification, Regression, and Hypothesis Testing in Diffusion MRI**. *Pattern Recognition (2017)*, 63(593-600)
- Amir et al: **viSNE enables visualization of high dimensional single-cell data and reveals phenotypic heterogeneity of leukemia**. Nature Biotechnology 31(6):545-552, 2013

Exam-like Questions

1. When using Principal Component Analysis (PCA) for dimensionality reduction, suggest a criterion for deciding the target dimensionality.
2. What effect can we achieve by using different stress functionals in Multidimensional Scaling (MDS)?
3. Name or describe a type of neighborhood graph that we could use in ISOMAP, and explain under which conditions that type of neighborhood graph could be a suitable choice.
4. What is the main difference between Stochastic Neighborhood Embedding (SNE) and t-SNE?